



(12) **United States Patent**
Deshpande

(10) **Patent No.:** **US 12,389,045 B2**
(45) **Date of Patent:** **Aug. 12, 2025**

(54) **SYSTEMS AND METHODS FOR SIGNALING NEURAL NETWORK POST-FILTER APPLICATION SPECIFIC PURPOSE INFORMATION IN VIDEO CODING**

2022/0256227 A1* 8/2022 Rezazadegan Tavakoli A61B 5/4839
2022/0394308 A1* 12/2022 Li H04N 19/82
2023/0112309 A1* 4/2023 Hannuksela H04N 19/184
375/240.29

(71) Applicant: **SHARP KABUSHIKI KAISHA**, Sakai (JP)

OTHER PUBLICATIONS

(72) Inventor: **Sachin G. Deshpande**, Vancouver, WA (US)

JVET-AA2006-v1 “Additional SEI messages for VSEI (Draft 2)” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29, 27th Meeting, by teleconference, Jul. 13-22, 2022.

(73) Assignee: **Sharp Kabushiki Kaisha**, Osaka (JP)

JVET-Z0082-v2 “AHG11: Content-adaptive neural network post-filter” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29, 26th Meeting, by teleconference, Apr. 20-29, 2022.

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

“Test Model of Incremental Compression of Neural Networks for Multimedia Content Description and Analysis (INCTM),” ISO/IEC JTC 1/SC 29/WG 04, N0179. Feb. 4, 2022.

(21) Appl. No.: **18/214,780**

JVET-Z0052-v1 “AHG9: NNR post-filter SEI message” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29, 26th Meeting, by teleconference, Apr. 20-29, 2022.

(22) Filed: **Jun. 27, 2023**

JVET-Y2006 “Additional SEI messages for VSEI (Draft 6)” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29, 25th Meeting, by teleconference, Jan. 12-21, 2022.

(65) **Prior Publication Data**

US 2025/0008163 A1 Jan. 2, 2025

(Continued)

(51) **Int. Cl.**

H04N 19/85 (2014.01)

H04N 19/70 (2014.01)

H04N 19/80 (2014.01)

Primary Examiner — William C Vaughn, Jr.

Assistant Examiner — Astewaye Gettu Zewede

(74) *Attorney, Agent, or Firm* — J. Fish Law, PLLC

(52) **U.S. Cl.**

CPC **H04N 19/85** (2014.11); **H04N 19/70** (2014.11); **H04N 19/80** (2014.11)

(57)

ABSTRACT

(58) **Field of Classification Search**

CPC H04N 19/85; H04N 19/70; H04N 19/80

USPC 375/240

See application file for complete search history.

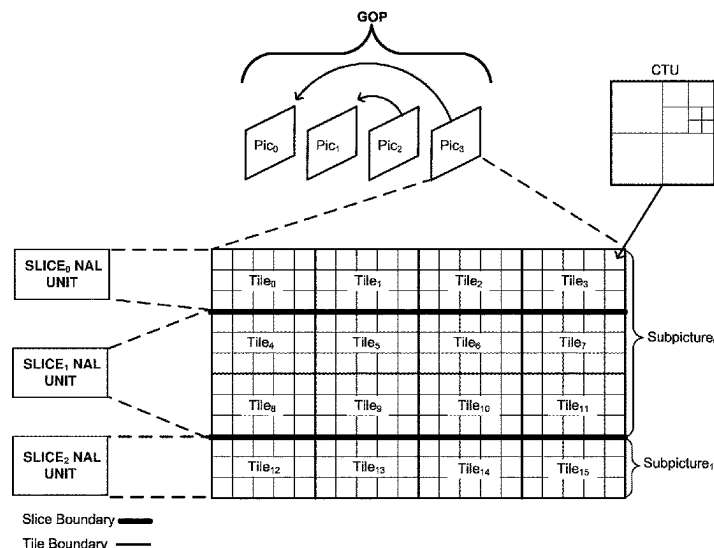
A device may be configured to perform filtering based on information included in a neural network post-filter characteristics message. In one example, the neural network post-filter characteristics message includes a syntax element indicating whether a purpose of a neural network post-filter is specified by an application. In one example, the neural network post-filter characteristics message includes a syntax element specifying a universal resource identifier indicating an application specified purpose.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2017/0289556 A1* 10/2017 Hendry H04N 21/85406

4 Claims, 8 Drawing Sheets



(56)

References Cited

OTHER PUBLICATIONS

Rec. ITU-T H.274 “Versatile supplemental enhancement information messages for coded video bitstreams” (Aug. 2020).

Rec. ITU-T H.273 “Coding-independent code point for video signal type identification” (Jul. 2021).

“New level and systems-related supplemental enhancement information for VVC (Draft 5)” 30th Meeting of ISO/IEC JTC1/SC29/WG11 Apr. 21-28, 2023, Antalya, TR, document JVET-AD2005-v1. ITU-T H.266 “Versatile Video Coding” (Apr. 2022).

Algorithm Description of Enhanced Compression Model 7 (ECM 7), ISO/IEC JTC1/SC29 Document: JVET-AB2025, Oct. 20-28, 2022, Mainz, DE.

ITU-T H.265 “High efficiency video coding” (Dec. 2016).

ITU-T H.264 “Advanced video coding for generic audiovisual services” (Oct. 2016).

ISO/IEC FDIS 15938-17. “Information technology—Multimedia content description interface—Part 17: Compression of neural networks for multimedia content description and analysis” ISO/IEC 15938-17:2020(E). 2021.

Additional SEI messages for VSEI (Draft 4) 30th Meeting of ISO/IEC JTC1/SC29/WG11 Apr. 21-28, 2023, Antalya, TR, document JVET-AD2006-v1.

* cited by examiner

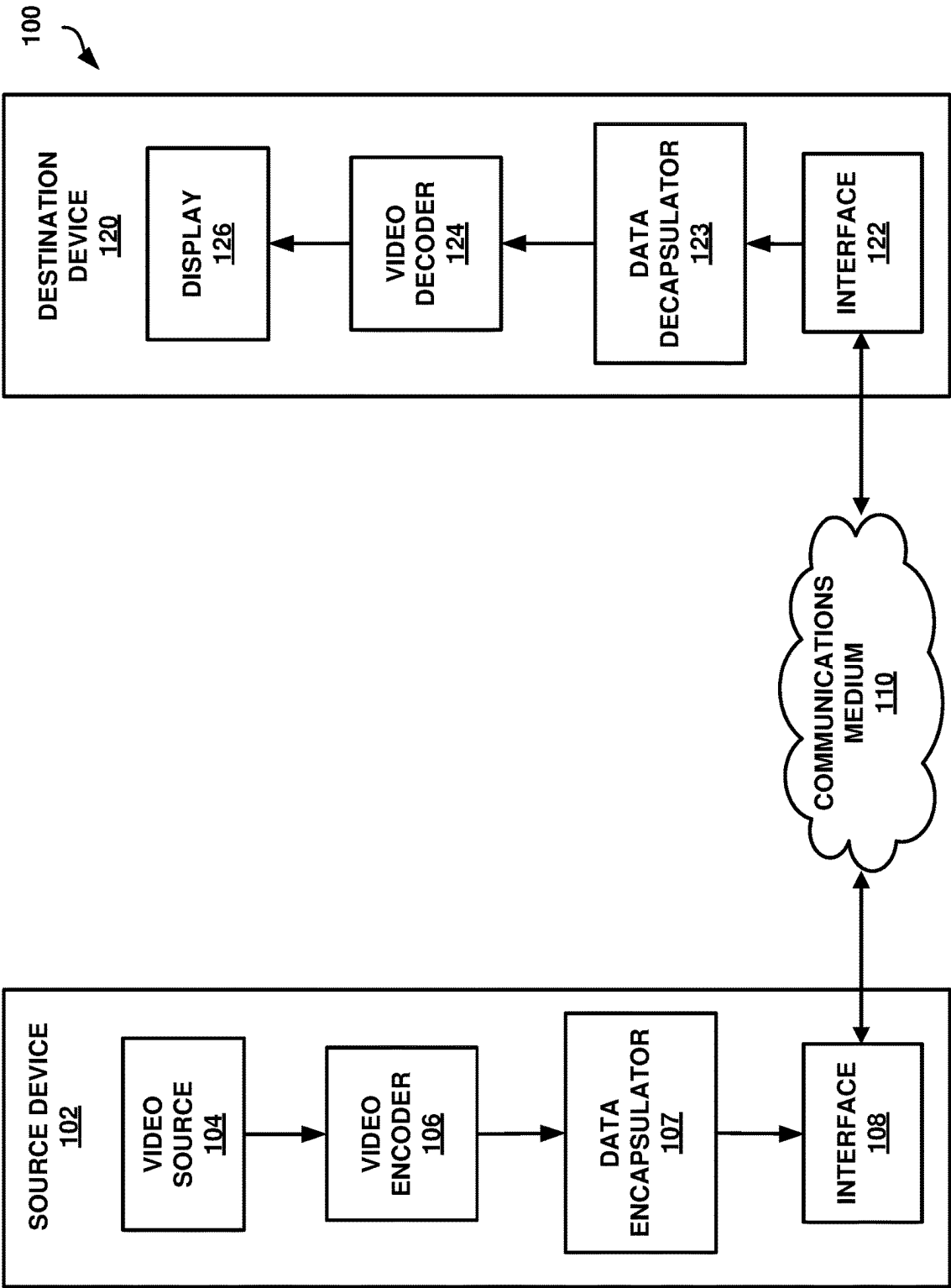


FIG. 1

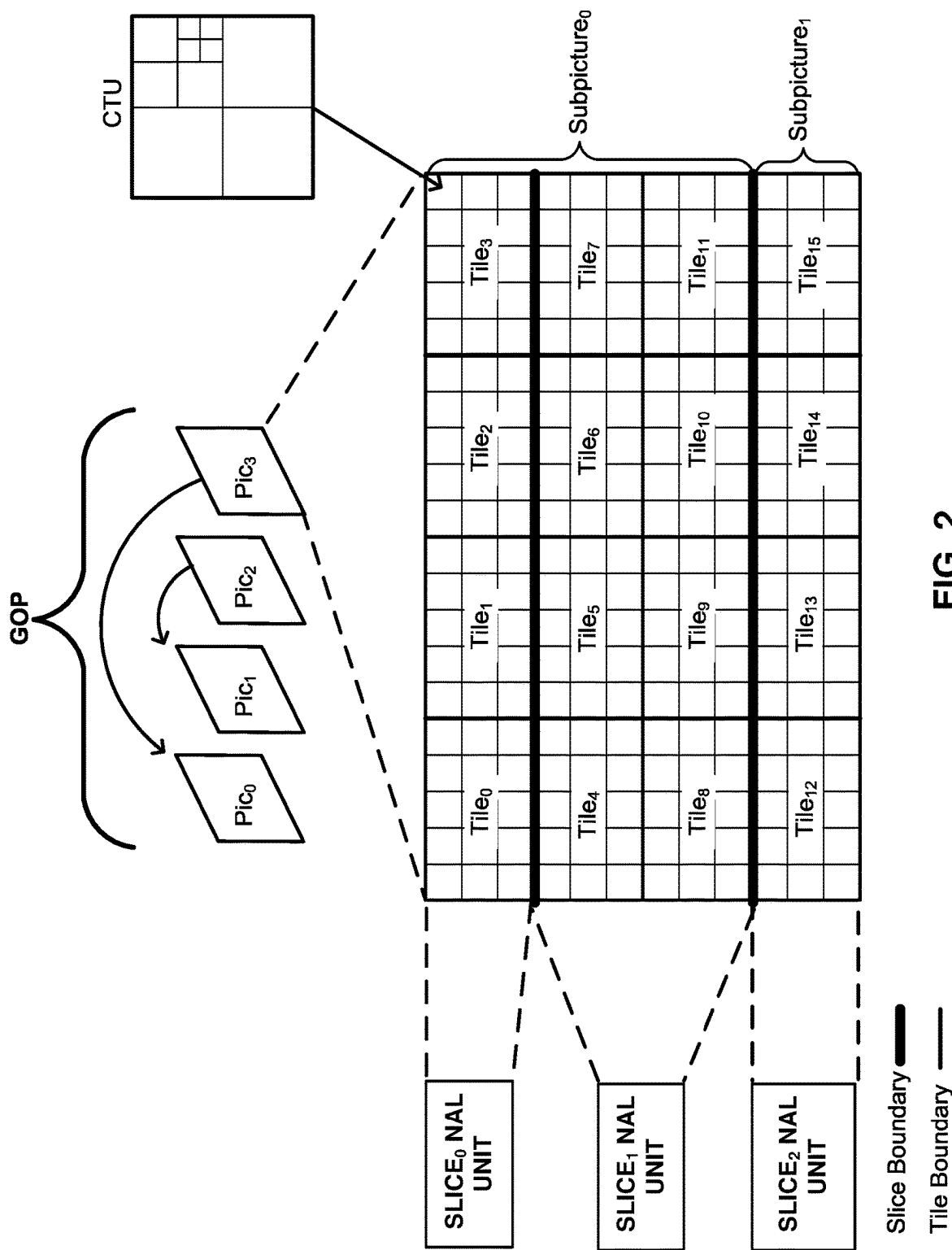


FIG. 2

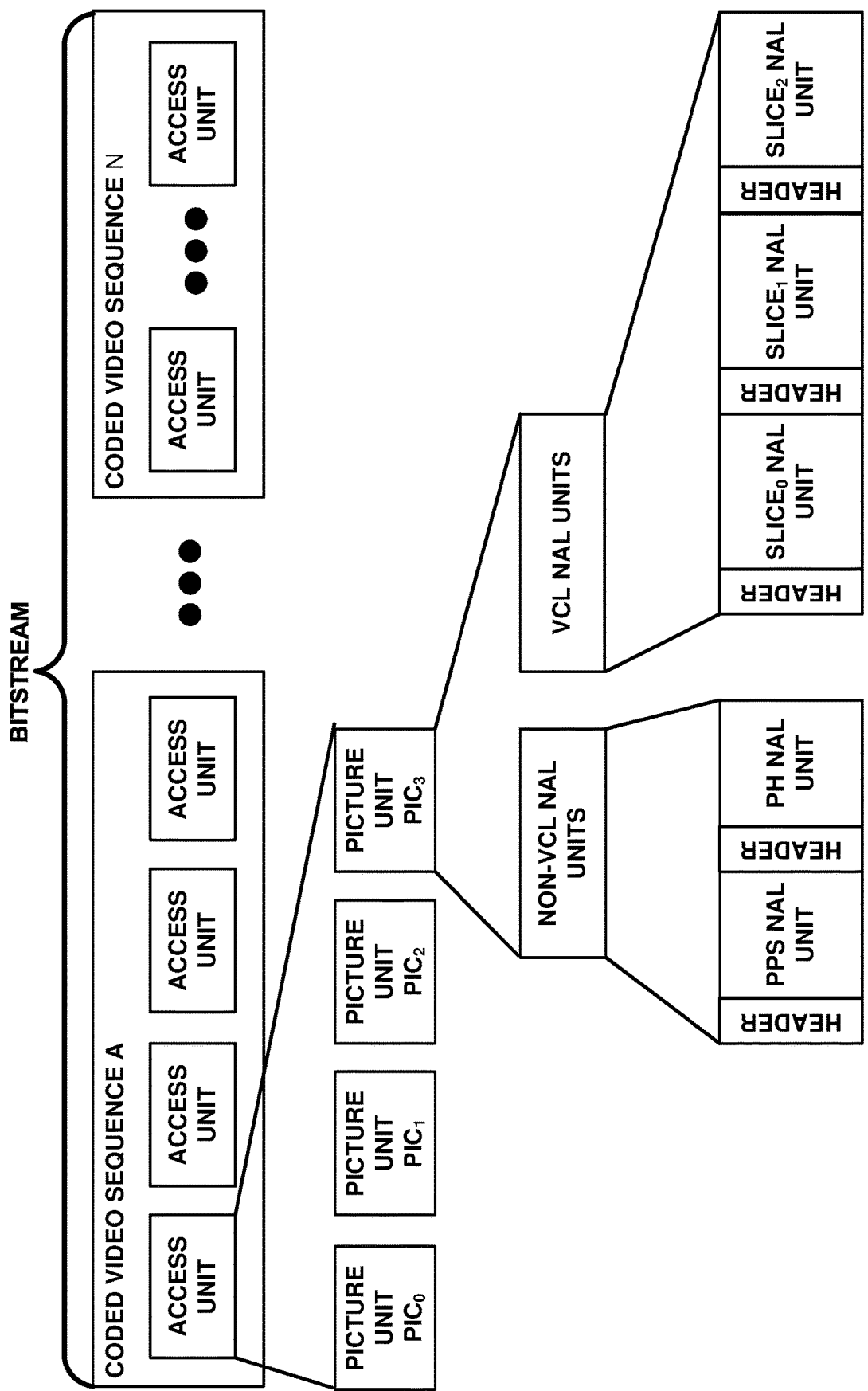


FIG. 3

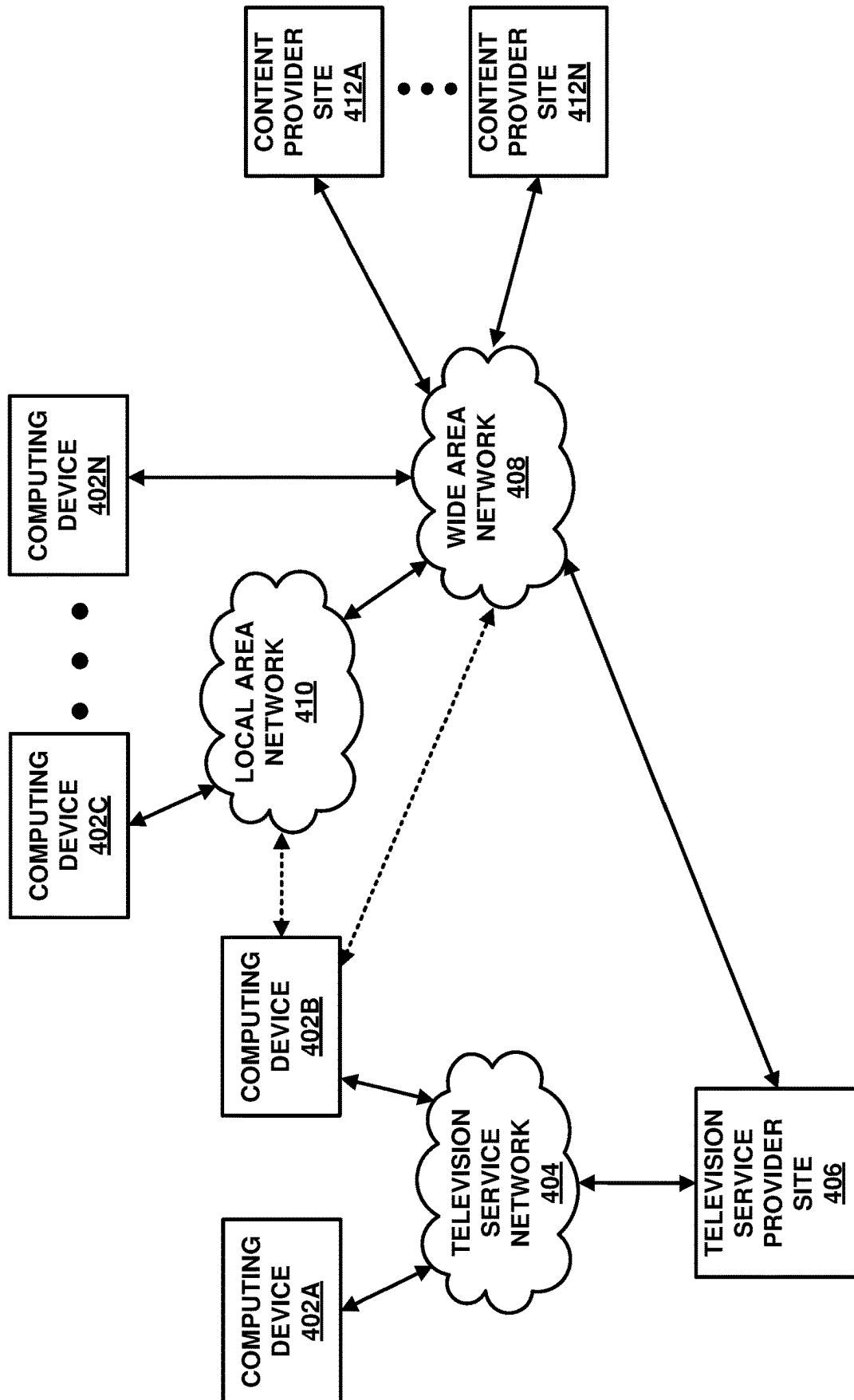
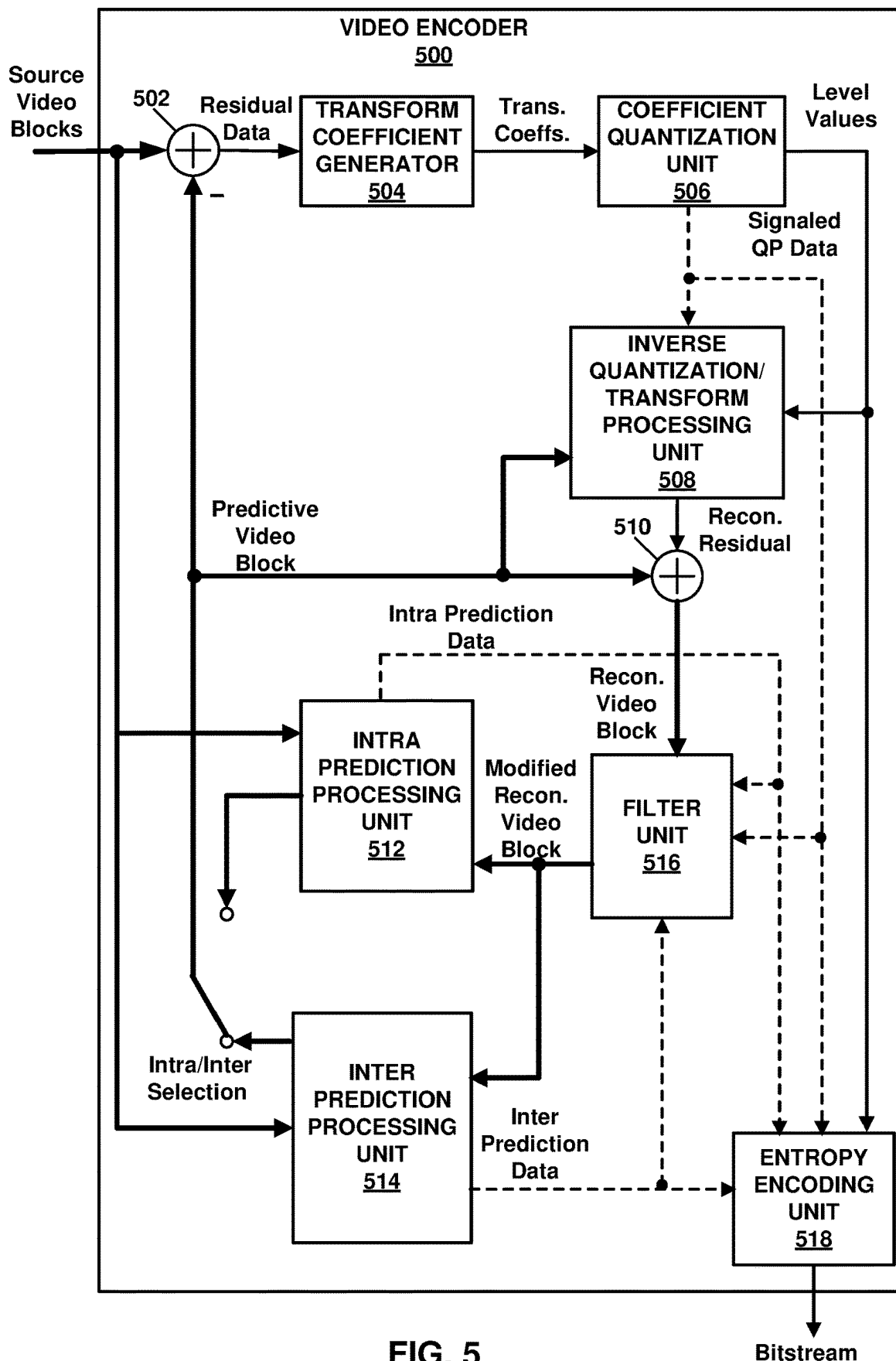


FIG. 4



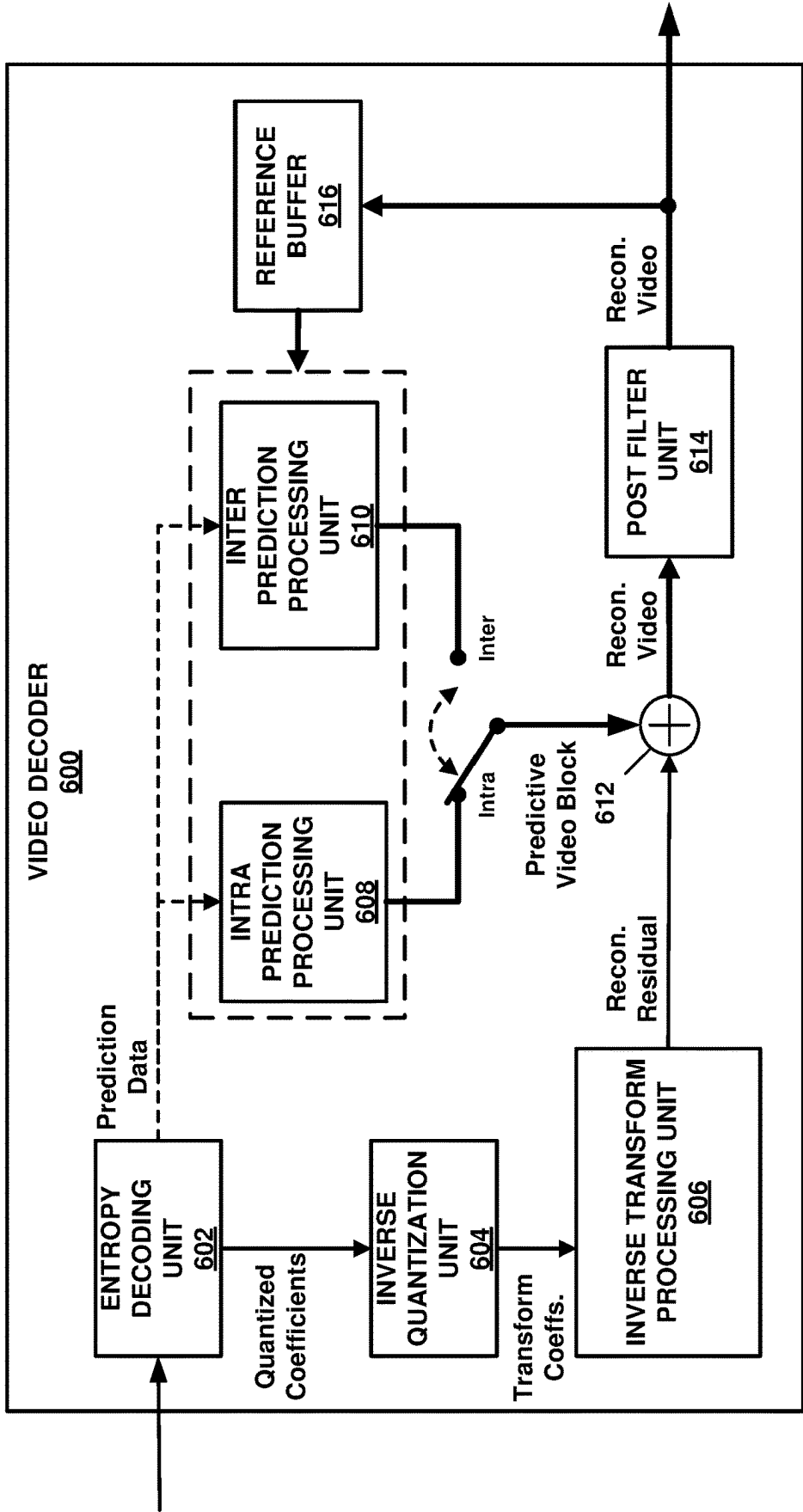
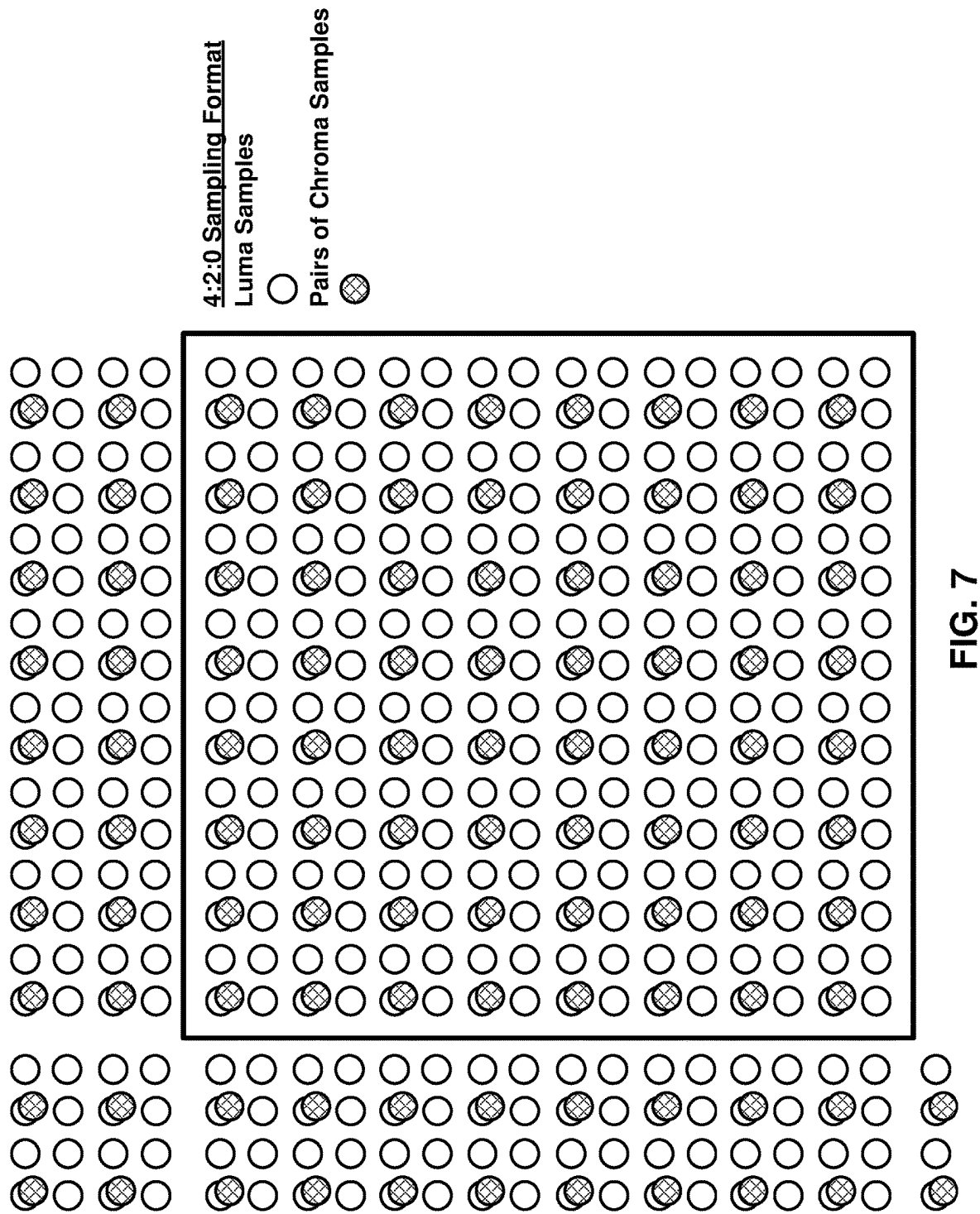


FIG. 6



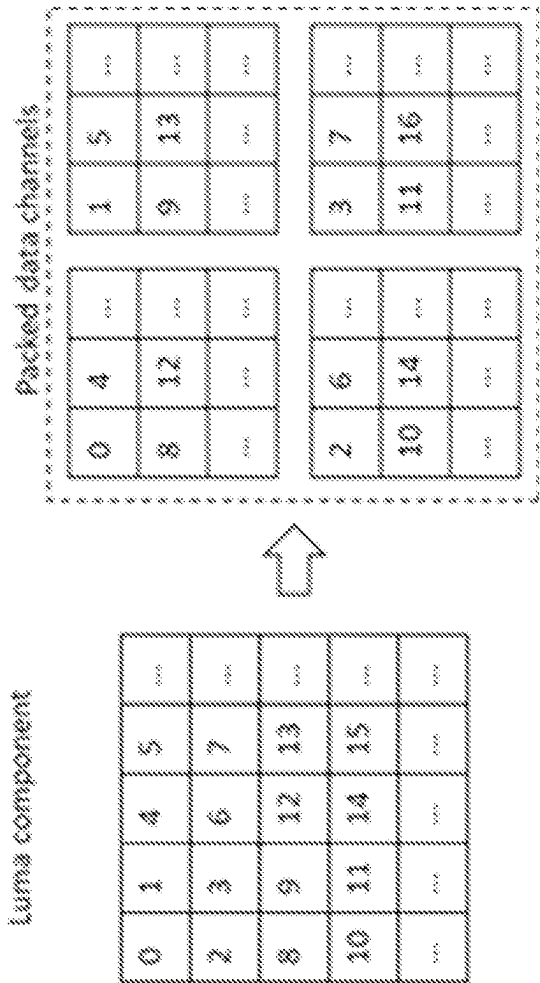


FIG. 8

1

SYSTEMS AND METHODS FOR SIGNALING NEURAL NETWORK POST-FILTER APPLICATION SPECIFIC PURPOSE INFORMATION IN VIDEO CODING

TECHNICAL FIELD

This disclosure relates to video coding and more particularly to techniques for signaling neural network post-filter parameter information for coded video.

BACKGROUND

Digital video capabilities can be incorporated into a wide range of devices, including digital televisions, laptop or desktop computers, tablet computers, digital recording devices, digital media players, video gaming devices, cellular telephones, including so-called smartphones, medical imaging devices, and the like. Digital video may be coded according to a video coding standard. Video coding standards define the format of a compliant bitstream encapsulating coded video data. A compliant bitstream is a data structure that may be received and decoded by a video decoding device to generate reconstructed video data. Video coding standards also define the decoding process and decoders that follow the decoding process can be said to be conforming decoders. Video coding standards may incorporate video compression techniques. Examples of video coding standards include ISO/IEC MPEG-4 Visual and ITU-T H.264 (also known as ISO/IEC MPEG-4 AVC), High-Efficiency Video Coding (HEVC), and Versatile video coding (VVC). HEVC is described in High Efficiency Video Coding, Rec. ITU-T H.265, November 2019, which is referred to herein as ITU-T H.265. VVC is described in Versatile Video Coding, Rec. ITU-T H.266, April 2022, which is incorporated by reference, and referred to herein as ITU-T H.266. Extensions and improvements for ITU-T H.266 are currently being considered for the development of next generation video coding standards. For example, the ITU-T Video Coding Experts Group (VCEG) and ISO/IEC (Moving Picture Experts Group (MPEG) (collectively referred to as the Joint Video Exploration Team (JVET)) are working to standardized enhanced video coding technology beyond the capabilities of the VVC standard. The Enhanced Compression Model 7 (ECM 7), Algorithm Description of Enhanced Compression Model 7 (ECM 7), ISO/IEC JTC1/SC29 Document: JVET-AB2025, Oct. 20-28, 2022, Mainz, DE, which is incorporated by reference herein, describes the coding features that were under coordinated test model study by as potentially enhancing video coding technology beyond the capabilities of ITU-T H.266. It should be noted that the coding features of ECM 7 are implemented in ECM reference software. As used herein, the term ECM may collectively refer to algorithms included in ECM 7 and implementations of ECM reference software.

Video compression techniques enable data requirements for storing and transmitting video data to be reduced. Video compression techniques may reduce data requirements by exploiting the inherent redundancies in a video sequence. Video compression techniques may sub-divide a video sequence into successively smaller portions (i.e., groups of pictures within a video sequence, a picture within a group of pictures, regions within a picture, sub-regions within regions, etc.). Intra prediction coding techniques (e.g., spatial prediction techniques within a picture) and inter prediction techniques (i.e., inter-picture techniques (temporal)) may be used to generate difference values between a unit of

2

video data to be coded and a reference unit of video data. The difference values may be referred to as residual data. Residual data may be coded as quantized transform coefficients. Syntax elements may relate residual data and a reference coding unit (e.g., intra-prediction mode indices, and motion information). Residual data and syntax elements may be entropy coded. Entropy encoded residual data and syntax elements may be included in data structures forming a compliant bitstream.

SUMMARY

In general, this disclosure describes various techniques for coding video data. In particular, this disclosure describes techniques for signaling neural network post-filter parameter information in video coding. It should be noted that although techniques of this disclosure are described with respect to ITU-T H.264, ITU-T H.265, ITU-T H.266, and ECM, the techniques of this disclosure are generally applicable to video coding. For example, the coding techniques described herein may be incorporated into video coding systems, (including video coding systems based on future video coding standards) including video block structures, intra prediction techniques, inter prediction techniques, transform techniques, filtering techniques, and/or entropy coding techniques other than those included in ITU-T H.264, ITU-T H.265, ITU-T H.266, and ECM. Thus, reference to ITU-T H.264, ITU-T H.265, ITU-T H.266, and/or ECM is for descriptive purposes and should not be construed to limit the scope of the techniques described herein. Further, it should be noted that incorporation by reference of documents herein is for descriptive purposes and should not be construed to limit or create ambiguity with respect to terms used herein. For example, in the case where an incorporated reference provides a different definition of a term than another incorporated reference and/or as the term is used herein, the term should be interpreted in a manner that broadly includes each respective definition and/or in a manner that includes each of the particular definitions in the alternative.

In one example, a method of coding video data comprises signaling a neural network post-filter characteristics message, signaling a first syntax element in the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and signaling a second syntax element in the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, a device comprises one or more processors configured to signal a neural network post-filter characteristics message, signal a first syntax element in the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and signal a second syntax element in the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, a non-transitory computer-readable storage medium comprises instructions stored thereon that, when executed, cause one or more processors of a device to signal a neural network post-filter characteristics message, signal a first syntax element in the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and signal a second syntax element in the neural network post-

filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, an apparatus comprises means for signaling a neural network post-filter characteristics message, means for signaling a first syntax element in the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and means for signaling a second syntax element in the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, a method of decoding video data comprises receiving a neural network post-filter characteristics message, parsing a first syntax element from the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and parsing a second syntax element from the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, a device comprises one or more processors configured to receive a neural network post-filter characteristics message, parse a first syntax element from the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and parse a second syntax element from the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, a non-transitory computer-readable storage medium comprises instructions stored thereon that, when executed, cause one or more processors of a device to receive a neural network post-filter characteristics message, parse a first syntax element from the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and parse a second syntax element from the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one example, an apparatus comprises means for receiving a neural network post-filter characteristics message, means for parsing a first syntax element from the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and means for parsing a second syntax element from the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

The details of one or more examples are set forth in the accompanying drawings and the description below. Other features, objects, and advantages will be apparent from the description and drawings, and from the claims.

BRIEF DESCRIPTION OF DRAWINGS

FIG. 1 is a block diagram illustrating an example of a system that may be configured to encode and decode video data according to one or more techniques of this disclosure.

FIG. 2 is a conceptual diagram illustrating coded video data and corresponding data structures according to one or more techniques of this disclosure.

FIG. 3 is a conceptual diagram illustrating a data structure encapsulating coded video data and corresponding metadata according to one or more techniques of this disclosure.

FIG. 4 is a conceptual drawing illustrating an example of components that may be included in an implementation of a

system that may be configured to encode and decode video data according to one or more techniques of this disclosure.

FIG. 5 is a block diagram illustrating an example of a video encoder that may be configured to encode video data according to one or more techniques of this disclosure.

FIG. 6 is a block diagram illustrating an example of a video decoder that may be configured to decode video data according to one or more techniques of this disclosure.

FIG. 7 is a conceptual diagram illustrating an example of a video component sampling format that may be utilized in accordance with one or more techniques of this disclosure.

FIG. 8 is a conceptual drawing illustrating an example of packed data channels of a luma component according to one or more techniques of this disclosure.

DETAILED DESCRIPTION

Video content includes video sequences comprised of a series of frames (or pictures). A series of frames may also be referred to as a group of pictures (GOP). Each video frame or picture may be divided into one or more regions. Regions may be defined according to a base unit (e.g., a video block) and sets of rules defining a region. For example, a rule defining a region may be that a region must be an integer number of video blocks arranged in a rectangle. Further, video blocks in a region may be ordered according to a scan pattern (e.g., a raster scan). As used herein, the term video block may generally refer to an area of a picture or may more specifically refer to the largest array of sample values that may be predictively coded, sub-divisions thereof, and/or corresponding structures. Further, the term current video block may refer to an area of a picture being encoded or decoded. A video block may be defined as an array of sample values. It should be noted that in some cases pixel values may be described as including sample values for respective components of video data, which may also be referred to as color components, (e.g., luma (Y) and chroma (Cb and Cr) components or red, green, and blue components). It should be noted that in some cases, the terms pixel value and sample value are used interchangeably. Further, in some cases, a pixel or sample may be referred to as a pel. A video sampling format, which may also be referred to as a chroma format, may define the number of chroma samples included in a video block with respect to the number of luma samples included in a video block. For example, for the 4:2:0 sampling format, the sampling rate for the luma component is twice that of the chroma components for both the horizontal and vertical directions. It should be noted that in some cases, the terms luma and luminance are used interchangeably.

A video encoder may perform predictive encoding on video blocks and sub-divisions thereof. Video blocks and sub-divisions thereof may be referred to as nodes. ITU-T H.264 specifies a macroblock including 16×16 luma samples. That is, in ITU-T H.264, a picture is segmented into macroblocks. ITU-T H.265 specifies an analogous Coding Tree Unit (CTU) structure (which may be referred to as a largest coding unit (LCU)). In ITU-T H.265, pictures are segmented into CTUs. In ITU-T H.265, for a picture, a CTU size may be set as including 16×16, 32×32, or 64×64 luma samples. In ITU-T H.265, a CTU is composed of respective Coding Tree Blocks (CTB) for each component of video data (e.g., luma (Y) and chroma (Cb and Cr)). It should be noted that video having one luma component and the two corresponding chroma components may be described as having two channels, i.e., a luma channel and a chroma channel. Further, in ITU-T H.265, a CTU may be partitioned

according to a quadtree (QT) partitioning structure, which results in the CTBs of the CTU being partitioned into Coding Blocks (CB). That is, in ITU-T H.265, a CTU may be partitioned into quadtree leaf nodes. According to ITU-T H.265, one luma CB together with two corresponding chroma CBs and associated syntax elements are referred to as a coding unit (CU). In ITU-T H.265, a minimum allowed size of a CB may be signaled. In ITU-T H.265, the smallest minimum allowed size of a luma CB is 8×8 luma samples. In ITU-T H.265, the decision to code a picture area using intra prediction or inter prediction is made at the CU level.

In ITU-T H.265, a CU is associated with a prediction unit structure having its root at the CU. In ITU-T H.265, prediction unit structures allow luma and chroma CBs to be split for purposes of generating corresponding reference samples. That is, in ITU-T H.265, luma and chroma CBs may be split into respective luma and chroma prediction blocks (PBs), where a PB includes a block of sample values for which the same prediction is applied. In ITU-T H.265, a CB may be partitioned into 1, 2, or 4 PBs. ITU-T H.265 supports PB sizes from 64×64 samples down to 4×4 samples. In ITU-T H.265, intra prediction data (e.g., intra prediction mode syntax elements) or inter prediction data (e.g., motion data syntax elements) corresponding to a PB is used to produce reference and/or predicted sample values for the PB. ITU-T H.266 specifies a CTU having a maximum size of 128×128 luma samples. In ITU-T H.266, CTUs are partitioned according to a quadtree plus multi-type tree (QTMT or QT+MTT) structure. The QTMT structure in ITU-T H.266 enables quadtree leaf nodes to be further partitioned by a binary tree (BT) structure. That is, in ITU-T H.266, quadtree leaf nodes may be recursively divided vertically or horizontally. Further, in ITU-T H.266, in addition to indicating binary splits, the multi-type tree may indicate so-called ternary (or triple tree (TT)) splits. A ternary split divides a block vertically or horizontally into three blocks. In the case of a vertical TT split, a block is divided at one quarter of its width from the left edge and at one quarter its width from the right edge and in the case of a horizontal TT split a block is at one quarter of its height from the top edge and at one quarter of its height from the bottom edge.

As described above, each video frame or picture may be divided into one or more regions. For example, according to ITU-T H.265, each video frame or picture may be partitioned to include one or more slices and further partitioned to include one or more tiles, where each slice includes a sequence of CTUs (e.g., in raster scan order) and where a tile is a sequence of CTUs corresponding to a rectangular area of a picture. It should be noted that a slice, in ITU-T H.265, is a sequence of one or more slice segments starting with an independent slice segment and containing all subsequent dependent slice segments (if any) that precede the next independent slice segment (if any). A slice segment, like a slice, is a sequence of CTUs. Thus, in some cases, the terms slice and slice segment may be used interchangeably to indicate a sequence of CTUs arranged in a raster scan order. Further, it should be noted that in ITU-T H.265, a tile may consist of CTUs contained in more than one slice and a slice may consist of CTUs contained in more than one tile. However, ITU-T H.265 provides that one or both of the following conditions shall be fulfilled: (1) All CTUs in a slice belong to the same tile; and (2) All CTUs in a tile belong to the same slice.

With respect to ITU-T H.266, slices are required to consist of an integer number of complete tiles or an integer number of consecutive complete CTU rows within a tile,

instead of only being required to consist of an integer number of CTUs. It should be noted that in ITU-T H.266, the slice design does not include slice segments (i.e., no independent/dependent slice segments). Thus, in ITU-T H.266, a picture may include a single tile, where the single tile is contained within a single slice or a picture may include multiple tiles where the multiple tiles (or CTU rows thereof) may be contained within one or more slices. In ITU-T H.266, the partitioning of a picture into tiles is specified by specifying respective heights for tile rows and respective widths for tile columns. Thus, in ITU-T H.266 a tile is a rectangular region of CTUs within a particular tile row and a particular tile column position. Further, it should be noted that ITU-T H.266 provides where a picture may be partitioned into subpictures, where a subpicture is a rectangular region of a CTUs within a picture. The top-left CTU of a subpicture may be located at any CTU position within a picture with subpictures being constrained to include one or more slices. Thus, unlike a tile, a subpicture is not necessarily limited to a particular row and column position. It should be noted that subpictures may be useful for encapsulating regions of interest within a picture and a sub-bitstream extraction process may be used to only decode and display a particular region of interest. That is, as described in further detail below, a bitstream of coded video data includes a sequence of network abstraction layer (NAL) units, where a NAL unit encapsulates coded video data, (i.e., video data corresponding to a slice of picture) or a NAL unit encapsulates metadata used for decoding video data (e.g., a parameter set) and a sub-bitstream extraction process forms a new bitstream by removing one or more NAL units from a bitstream.

FIG. 2 is a conceptual diagram illustrating an example of a picture within a group of pictures partitioned according to tiles, slices, and subpictures. It should be noted that the techniques described herein may be applicable to tiles, slices, subpictures, sub-divisions thereof and/or equivalent structures thereto. That is, the techniques described herein may be generally applicable regardless of how a picture is partitioned into regions. For example, in some cases, the techniques described herein may be applicable in cases where a tile may be partitioned into so-called bricks, where a brick is a rectangular region of CTU rows within a particular tile. Further, for example, in some cases, the techniques described herein may be applicable in cases where one or more tiles may be included in so-called tile groups, where a tile group includes an integer number of adjacent tiles. In the example illustrated in FIG. 2, Pic₃ is illustrated as including 16 tiles (i.e., Tile₀ to Tile₁₅) and three slices (i.e., Slice₀ to Slice₂). In the example illustrated in FIG. 2, Slice₀ includes four tiles (i.e., Tile₀ to Tile₃), Slice₁ includes eight tiles (i.e., Tile₄ to Tile₁₁), and Slice₂ includes four tiles (i.e., Tile₁₂ to Tile₁₅). Further, as illustrated in the example of FIG. 2, Pic₃ is illustrated as including two subpictures (i.e., Subpicture₀ and Subpicture₁), where Subpicture₀ includes Slice₀ and Slice₁ and where Subpicture₁ includes Slice₂. As described above, subpictures may be useful for encapsulating regions of interest within a picture and a sub-bitstream extraction process may be used in order to selectively decode (and display) a region interest. For example, referring to FIG. 2, Subpicture₀ may correspond to an action portion of a sporting event presentation (e.g., a view of the field) and Subpicture₁ may correspond to a scrolling banner displayed during the sporting event presentation. By using organizing a picture into subpictures in this manner, a viewer may be able to disable the display of the scrolling banner. That is, through a sub-bitstream extraction

process Slice₂ NAL unit may be removed from a bitstream (and thus not decoded and/or displayed) and Slice₀ NAL unit and Slice₁ NAL unit may be decoded and displayed. The encapsulation of slices of a picture into respective NAL unit data structures and sub-bitstream extraction are described in further detail below.

As described above, a video sampling format, which may also be referred to as a chroma format, may define the number of chroma samples included in a CU with respect to the number of luma samples included in a CU. For example, for the 4:2:0 sampling format, the sampling rate for the luma component is twice that of the chroma components for both the horizontal and vertical directions. As a result, for a CU formatted according to the 4:2:0 format, the width and height of an array of samples for the luma component are twice that of each array of samples for the chroma components. FIG. 7 is a conceptual diagram illustrating an example of a coding unit formatted according to a 4:2:0 sample format. FIG. 7 illustrates the relative position of chroma samples with respect to luma samples within a CU. As described above, a CU is typically defined according to the number of horizontal and vertical luma samples. Thus, as illustrated in FIG. 7, a 16×16 CU formatted according to the 4:2:0 sample format includes 16×16 samples of luma components and 8×8 samples for each chroma component. Further, in the example illustrated in FIG. 7, the relative position of chroma samples with respect to luma samples for video blocks neighboring the 16×16 CU are illustrated. For a CU formatted according to the 4:2:2 format, the width of an array of samples for the luma component is twice that of the width of an array of samples for each chroma component, but the height of the array of samples for the luma component is equal to the height of an array of samples for each chroma component. Further, for a CU formatted according to the 4:4:4 format, an array of samples for the luma component has the same width and height as an array of samples for each chroma component.

Table 1 illustrates how a chroma format is specified in ITU-T H.266 based on the value of syntax element chroma_format_idc. Further, Table 1 illustrates how the variables SubWidthC and SubHeightC are specified derived depending on the chroma format. SubWidthC and SubHeightC are utilized, for example, for deblocking. With respect to Table 1, ITU-T H.266 provides the following:

In monochrome sampling there is only one sample array, which is nominally considered the luma array.

In 4:2:0 sampling, each of the two chroma arrays has half the height and half the width of the luma array.

In 4:2:2 sampling, each of the two chroma arrays has the same height and half the width of the luma array.

In 4:4:4 sampling, each of the two chroma arrays has the same height and width as the luma array.

TABLE 1

chroma_format_idc	Chroma format	SubWidthC	SubHeightC
0	Monochrome	1	1
1	4:2:0	2	2
2	4:2:2	2	1
3	4:4:4	1	1

For intra prediction coding, an intra prediction mode may specify the location of reference samples within a picture. In ITU-T H.265, defined possible intra prediction modes include a planar (i.e., surface fitting) prediction mode, a DC (i.e., flat overall averaging) prediction mode, and 33 angular prediction modes (predMode: 2-34). In ITU-T H.266,

defined possible intra-prediction modes include a planar prediction mode, a DC prediction mode, and 65 angular prediction modes. Further, in ITU-T H.266, additional intra prediction tools, such as, for example, intra subpartition mode and matrix-based intra prediction are enabled. It should be noted that planar and DC prediction modes may be referred to as non-directional prediction modes and that angular prediction modes may be referred to as directional prediction modes. It should be noted that the techniques described herein may be generally applicable regardless of the number of defined possible prediction modes.

For inter prediction coding, a reference picture is determined and a motion vector (MV) identifies samples in the reference picture that are used to generate a prediction for a current video block. For example, a current video block may be predicted using reference sample values located in one or more previously coded picture(s) and a motion vector is used to indicate the location of the reference block relative to the current video block. A motion vector may describe, for example, a horizontal displacement component of the motion vector (i.e., MV_x), a vertical displacement component of the motion vector (i.e., MV_y), and a resolution for the motion vector (e.g., one-quarter pixel precision, one-half pixel precision, one-pixel precision, two-pixel precision, four-pixel precision). Previously decoded pictures, which may include pictures output before or after a current picture, may be organized into one or more to reference pictures lists and identified using a reference picture index value. Further, in inter prediction coding, uni-prediction refers to generating a prediction using sample values from a single reference picture and bi-prediction refers to generating a prediction using respective sample values from two reference pictures. That is, in uni-prediction, a single reference picture and corresponding motion vector are used to generate a prediction for a current video block and in bi-prediction, a first reference picture and corresponding first motion vector and a second reference picture and corresponding second motion vector are used to generate a prediction for a current video block. In bi-prediction, respective sample values are combined (e.g., added, rounded, and clipped, or averaged according to weights) to generate a prediction. Pictures and regions thereof may be classified based on which types of prediction modes may be utilized for encoding video blocks thereof. That is, for regions having a B type (e.g., a B slice), bi-prediction, uni-prediction, and intra prediction modes may be utilized, for regions having a P type (e.g., a P slice), uni-prediction, and intra prediction modes may be utilized, and for regions having an I type (e.g., an I slice), only intra prediction modes may be utilized. As described above, reference pictures are identified through reference indices. For example, for a P slice, there may be a single reference picture list, RefPicList0 and for a B slice, there may be a second independent reference picture list, RefPicList1, in addition to RefPicList0. It should be noted that for uni-prediction in a B slice, one of RefPicList0 or RefPicList1 may be used to generate a prediction. Further, it should be noted that during the decoding process, at the onset of decoding a picture, reference picture list(s) are generated from previously decoded pictures stored in a decoded picture buffer (DPB).

Further, a coding standard may support various modes of motion vector prediction. Motion vector prediction enables the value of a motion vector for a current video block to be derived based on another motion vector. For example, a set of candidate blocks having associated motion information may be derived from spatial neighboring blocks and temporal neighboring blocks to the current video block. Further,

generated (or default) motion information may be used for motion vector prediction. Examples of motion vector prediction include advanced motion vector prediction (AMVP), temporal motion vector prediction (TMVP), so-called “merge” mode, and “skip” and “direct” motion inference. Further, other examples of motion vector prediction include advanced temporal motion vector prediction (ATMVP) and Spatial-temporal motion vector prediction (STMVP). Further, in ITU-T H.266, the following inter prediction modes are enabled: the affine motion model, adaptive motion vector resolution, bi-directional optical flow, decoder side-motion vector refinement and geometric partitioning mode.

As described above, for inter prediction coding, reference samples in a previously coded picture are used for coding video blocks in a current picture. Previously coded pictures which are available for use as reference when coding a current picture are referred as reference pictures. It should be noted that the decoding order does not necessarily correspond with the picture output order, i.e., the temporal order of pictures in a video sequence. In ITU-T H.266, when a picture is decoded it is stored to a decoded picture buffer (DPB) (which may be referred to as frame buffer, a reference buffer, a reference picture buffer, or the like). In ITU-T H.266, pictures stored to the DPB are removed from the DPB when they have been output and are no longer needed for coding subsequent pictures. In ITU-T H.266, a determination of whether pictures should be removed from the DPB is invoked once per picture, after decoding a slice header, i.e., at the onset of decoding a picture. For example, referring to FIG. 2, Pic₂ is illustrated as referencing Pic₁. Similarly, Pic₃ is illustrated as referencing Pic₀. With respect to FIG. 2, assuming the picture number corresponds to the decoding order, the DPB would be populated as follows: after decoding Pic₀, the DPB would include {Pic₀}; at the onset of decoding Pic₁, the DPB would include {Pic₀}; after decoding Pic₁, the DPB would include {Pic₀, Pic₁}; at the onset of decoding Pic₂, the DPB would include {Pic₀, Pic₁}. Pic₂ would then be decoded with reference to Pic₁ and after decoding Pic₂, the DPB would include {Pic₀, Pic₁, Pic₂}. At the onset of decoding Pic₃, pictures Pic₀ and Pic₁ would be marked for removal from the DPB, as they are not needed for decoding Pic₃ (or any subsequent pictures, not shown) and assuming Pic₁ and Pic₂ have been output, the DPB would be updated to include {Pic₀}. Pic₃ would then be decoded by referencing Pic₀. The process of marking pictures for removal from a DPB may be referred to as reference picture set (RPS) management.

As described above, intra prediction data or inter prediction data is used to produce reference sample values for a block of sample values. The difference between sample values included in a current PB, or another type of picture area structure, and associated reference samples (e.g., those generated using a prediction) may be referred to as residual data. Residual data may include respective arrays of difference values corresponding to each component of video data. Residual data may be in the pixel domain. A transform, such as, a discrete cosine transform (DCT), a discrete sine transform (DST), an integer transform, a wavelet transform, or a conceptually similar transform, may be applied to an array of difference values to generate transform coefficients. It should be noted that in ITU-T H.265 and ITU-T H.266, a CU is associated with a transform tree structure having its root at the CU level. The transform tree is partitioned into one or more transform units (TUs). That is, an array of difference values may be partitioned for purposes of generating transform coefficients (e.g., four 8×8 transforms may be applied to a 16×16 array of residual values). For each component of

video data, such sub-divisions of difference values may be referred to as Transform Blocks (TBs). It should be noted that in some cases, a core transform and subsequent secondary transforms may be applied (in the video encoder) to generate transform coefficients. For a video decoder, the order of transforms is reversed.

A quantization process may be performed on transform coefficients or residual sample values directly (e.g., in the case, of palette coding quantization). Quantization approximates transform coefficients by amplitudes restricted to a set of specified values. Quantization essentially scales transform coefficients in order to vary the amount of data required to represent a group of transform coefficients. Quantization may include division of transform coefficients (or values resulting from the addition of an offset value to transform coefficients) by a quantization scaling factor and any associated rounding functions (e.g., rounding to the nearest integer). Quantized transform coefficients may be referred to as coefficient level values. Inverse quantization (or “dequantization”) may include multiplication of coefficient level values by the quantization scaling factor, and any reciprocal rounding or offset addition operations. It should be noted that as used herein the term quantization process in some instances may refer to division by a scaling factor to generate level values and multiplication by a scaling factor to recover transform coefficients in some instances. That is, a quantization process may refer to quantization in some cases and inverse quantization in some cases. Further, it should be noted that although in some of the examples below quantization processes are described with respect to arithmetic operations associated with decimal notation, such descriptions are for illustrative purposes and should not be construed as limiting. For example, the techniques described herein may be implemented in a device using binary operations and the like. For example, multiplication and division operations described herein may be implemented using bit shifting operations and the like.

Quantized transform coefficients and syntax elements (e.g., syntax elements indicating a coding structure for a video block) may be entropy coded according to an entropy coding technique. An entropy coding process includes coding values of syntax elements using lossless data compression algorithms. Examples of entropy coding techniques include content adaptive variable length coding (CAVLC), context adaptive binary arithmetic coding (CABAC), probability interval partitioning entropy coding (PIPE), and the like. Entropy encoded quantized transform coefficients and corresponding entropy encoded syntax elements may form a compliant bitstream that can be used to reproduce video data at a video decoder. An entropy coding process, for example, CABAC, may include performing a binarization on syntax elements. Binarization refers to the process of converting a value of a syntax element into a series of one or more bits. These bits may be referred to as “bins.” Binarization may include one or a combination of the following coding techniques: fixed length coding, unary coding, truncated unary coding, truncated Rice coding, Golomb coding, k-th order exponential Golomb coding, and Golomb-Rice coding. For example, binarization may include representing the integer value of 5 for a syntax element as 00000101 using an 8-bit fixed length binarization technique or representing the integer value of 5 as 11110 using a unary coding binarization technique. As used herein each of the terms fixed length coding, unary coding, truncated unary coding, truncated Rice coding, Golomb coding, k-th order exponential Golomb coding, and Golomb-Rice coding may refer to general implementations of these techniques and/or more

specific implementations of these coding techniques. For example, a Golomb-Rice coding implementation may be specifically defined according to a video coding standard. In the example of CABAC, for a particular bin, a context provides a most probable state (MPS) value for the bin (i.e., an MPS for a bin is one of 0 or 1) and a probability value of the bin being the MPS or the least probably state (LPS). For example, a context may indicate, that the MPS of a bin is 0 and the probability of the bin being 1 is 0.3. It should be noted that a context may be determined based on values of previously coded bins including bins in the current syntax element and previously coded syntax elements. For example, values of syntax elements associated with neighboring video blocks may be used to determine a context for a current bin.

As described above, video content includes video sequences comprised of a series of pictures and each picture may be divided into one or more regions. In ITU-T H.266, a coded representation of a picture comprises VCL NAL units of a particular layer within an AU and contains all CTUs of the picture. For example, referring again to FIG. 2, the coded representation of Pic₃ is encapsulated in three coded slice NAL units (i.e., Slice₀ NAL unit, Slice₁ NAL unit, and Slice₂ NAL unit). It should be noted that the term video coding layer (VCL) NAL unit is used as a collective term for coded slice NAL units, i.e., VCL NAL is a collective term which includes all types of slice NAL units. As described above, and in further detail below, a NAL unit may encapsulate metadata used for decoding video data. A NAL unit encapsulating metadata used for decoding a video sequence is generally referred to as a non-VCL NAL unit. Thus, in ITU-T H.266, a NAL unit may be a VCL NAL unit or a non-VCL NAL unit. It should be noted that a VCL NAL unit includes slice header data, which provides information used for decoding the particular slice. Thus, in ITU-T H.266, information used for decoding video data, which may be referred to as metadata in some cases, is not limited to being included in non-VCL NAL units. ITU-T H.266 provides where a picture unit (PU) is a set of NAL units that are associated with each other according to a specified classification rule, are consecutive in decoding order, and contain exactly one coded picture and where an access unit (AU) is a set of PUs that belong to different layers and contain coded pictures associated with the same time for output from the DPB. ITU-T H.266 further provides where a layer is a set of VCL NAL units that all have a particular value of a layer identifier and the associated non-VCL NAL units. Further, in ITU-T H.266, a PU consists of zero or one PH NAL units, one coded picture, which comprises of one or more VCL NAL units, and zero or more other non-VCL NAL units. Further, in ITU-T H.266, a coded video sequence (CVS) is a sequence of AUs that consists, in decoding order, of a CVSS AU, followed by zero or more AUs that are not CVSS AUs, including all subsequent AUs up to but not including any subsequent AU that is a CVSS AU, where a coded video sequence start (CVSS) AU is an AU in which there is a PU for each layer in the CVS and the coded picture in each present picture unit is a coded layer video sequence start (CLVSS) picture. In ITU-T H.266, a coded layer video sequence (CLVS) is a sequence of PUs within the same layer that consists, in decoding order, of a CLVSS PU, followed by zero or more PUs that are not CLVSS PUs, including all subsequent PUs up to but not including any subsequent PU that is a CLVSS PU. This is, in ITU-T H.266, a bitstream may be described as including a sequence of AUs forming one or more CVSs.

Multi-layer video coding enables a video presentation to be decoded/displayed as a presentation corresponding to a base layer of video data and decoded/displayed one or more additional presentations corresponding to enhancement layers of video data. For example, a base layer may enable a video presentation having a basic level of quality (e.g., a High Definition rendering and/or a 30 Hz frame rate) to be presented and an enhancement layer may enable a video presentation having an enhanced level of quality (e.g., an Ultra High Definition rendering and/or a 60 Hz frame rate) to be presented. An enhancement layer may be coded by referencing a base layer. That is, for example, a picture in an enhancement layer may be coded (e.g., using inter-layer prediction techniques) by referencing one or more pictures (including scaled versions thereof) in a base layer. It should be noted that layers may also be coded independent of each other. In this case, there may not be inter-layer prediction between two layers. Each NAL unit may include an identifier indicating a layer of video data the NAL unit is associated with. As described above, a sub-bitstream extraction process may be used to only decode and display a particular region of interest of a picture. Further, a sub-bitstream extraction process may be used to only decode and display a particular layer of video. Sub-bitstream extraction may refer to a process where a device receiving a compliant or conforming bitstream forms a new compliant or conforming bitstream by discarding and/or modifying data in the received bitstream. For example, sub-bitstream extraction may be used to form a new compliant or conforming bitstream corresponding to a particular representation of video (e.g., a high quality representation).

In ITU-T H.266, each of a video sequence, a GOP, a picture, a slice, and CTU may be associated with metadata that describes video coding properties and some types of metadata are encapsulated in non-VCL NAL units. ITU-T H.266 defines parameter sets that may be used to describe video data and/or video coding properties. In particular, ITU-T H.266 includes the following four types of parameter sets: video parameter set (VPS), sequence parameter set (SPS), picture parameter set (PPS), and adaption parameter set (APS), where a SPS applies to zero or more entire CVSs, a PPS applies to zero or more entire coded pictures, an APS applies to zero or more slices, and a VPS may be optionally referenced by a SPS. A PPS applies to one or more individual coded picture(s) that refers to it. In ITU-T H.266, parameter sets may be encapsulated as a non-VCL NAL unit and/or may be signaled as a message. ITU-T H.266 also includes a picture header (PH) which is encapsulated as a non-VCL NAL unit when signaled in its own NAL unit, or as part of a VCL NAL unit when signaled in the slice header of a coded slice. In ITU-T H.266, a picture header applies to all slices of a coded picture. ITU-T H.266 further enables decoding capability information (DCI) and supplemental enhancement information (SEI) messages to be signaled. In ITU-T H.266, DCI and SEI messages assist in processes related to decoding, display or other purposes, however, DCI and SEI messages may not be required for constructing the luma or chroma samples according to a decoding process. In ITU-T H.266, DCI and SEI messages may be signaled in a bitstream using non-VCL NAL units. Further, DCI and SEI messages may be conveyed by some mechanism other than by being present in the bitstream (i.e., signaled out-of-band).

FIG. 3 illustrates an example of a bitstream including multiple CVSs, where a CVS includes AUs, and AUs include picture units. The example illustrated in FIG. 3 corresponds to an example of encapsulating the slice NAL

13

units illustrated in the example of FIG. 2 in a bitstream. In the example illustrated in FIG. 3, the corresponding picture unit for Pic₃ includes the three VCL NAL coded slice NAL units, i.e., Slice₀ NAL unit, Slice₁ NAL unit, and Slice₂ NAL unit and two non-VCL NAL units, i.e., a PPS NAL Unit and a PH NAL unit. It should be noted that in FIG. 3, HEADER is a NAL unit header (i.e., not to be confused with a slice header). Further, it should be noted that in FIG. 3, other non-VCL NAL units, which are not illustrated may be included in the CVSSs, e.g., SPS NAL units, VPS NAL units, SEI message NAL units, etc. Further, it should be noted that in other examples, a PPS NAL Unit used for decoding Pic₃ may be included elsewhere in the bitstream, e.g., in the picture unit corresponding to Pic₀ or may be provided by an external mechanism. In ITU-T H.266, a PH syntax structure may be present in the slice header of a VCL NAL unit or in a PH NAL unit of the current PU.

It should be noted that during the development of ITU-T H.266, Convolutional Neural Networks (CNN)-based techniques, showing potential in artifact removal and objective quality improvement, were investigated, but it was decided not to include such techniques in ITU-T H.266. However, CNN based techniques are currently being considered for extensions and/or improvements for ITU-T H.266. Some CNN based-techniques relate to post-filtering. For example, “AHG11: Content-adaptive neural network post-filter,” 26th Meeting of ISO/IEC JTC1/SC29/WG11 20-29 Apr. 2022, Teleconference, document JVET-Z0082-v2, (referred to herein as JVET-Z0082) describes a content adaptive neural network based post-filter. It should be noted that in JVET-Z0082 the content adaption is achieved by overfitting the NN post-filter on test video. Further, it should be noted that the result of the overfitting process in JVET-Z0082 is a weight-update. JVET-Z0082 describes where the weight-update is coded with ISO/IEC FDIS 15938-17. Information technology—Multimedia content description interface—Part 17: Compression of neural networks for multimedia content description and analysis and Test Model of Incremental Compression of Neural Networks for Multimedia Content Description and Analysis (INCTM), N0179. February 2022, which may be collectively referred to as the MPEG NNR (Neural Network Representation) or Neural Network Coding (NNC) standard. JVET-Z0082 further describes where the coded weight-update signaled within the video bitstream as an NNR post-filter SEI message. “AHG9: NNR post-filter SEI message,” 26th Meeting of ISO/IEC JTC1/SC29/WG11 20-29 Apr. 2022, Teleconference, document JVET-Z0052-v1, (referred to herein as JVET-Z0052) describes the NNR post-filter SEI message utilized by JVET-Z0082. Elements of the NN post-filter described in JVET-Z0082 and the NNR post-filter SEI message described in JVET-Z0052 were adopted in “Additional SEI messages for VSEI (Draft 2)” 27th Meeting of ISO/IEC JTC1/SC29/WG11 13-22 Jul. 2022, Teleconference, document JVET-AA2006-v2, (referred to herein as JVET-AA2006). JVET-AA2006 provides versatile supplemental enhancement information messages for coded video bitstreams (VSEI). JVET-AA2006 specifies the syntax and semantics for a Neural network post-filter characteristics SEI message and for a Neural-network post-filter activation SEI message. A Neural network post-filter characteristics SEI message specifies a neural network that may be used as a post-processing filter. The use of specified post-processing filters for specific pictures is indicated with Neural-network post-filter activation SEI messages. “Additional SEI messages for VSEI (Draft 4)” 30th Meeting of ISO/IEC JTC1/SC29/WG11 21-28 Apr. 2023, Antalya, TR, document JVET-

14

AD2006-v1, (referred to herein as JVET-AD2006) further specifies the syntax and semantics for a Neural network post-filter characteristics SEI message and for a Neural-network post-filter activation SEI message. JVET-AD2006 is described in further detail below. The techniques described herein provide techniques for signaling of neural network post-filter messages.

With respect to the equations used herein, the following arithmetic operators may be used:

- + Addition
- − Subtraction
- * Multiplication, including matrix multiplication
- x^y Exponentiation. Specifies x to the power of y. In other contexts, such notation is used for superscripting not intended for interpretation as exponentiation.
- / Integer division with truncation of the result toward zero. For example, 7/4 and −7/−4 are truncated to 1 and −1.
- ÷ Used to denote division in mathematical equations where no truncation or rounding is intended.

$$\frac{x}{y}$$

Used to denote division in mathematical equations where no truncation or rounding is intended.

Further, the following mathematical functions may be used:

Log 2(x) the base-2 logarithm of x;

$$\text{Min}(x, y) = \begin{cases} x; & x \leq y \\ y; & x > y \end{cases};$$

$$\text{Max}(x, y) = \begin{cases} x; & x \geq y \\ y; & x < y \end{cases}$$

Ceil(x) the smallest integer greater than or equal to x.

With respect to the example syntax used herein, the following definitions of logical operators may be applied:

x && y Boolean logical “and” of x and y

x|y Boolean logical “or” of x and y

! Boolean logical “not”

x?:y:z If x is TRUE or not equal to 0, evaluates to the value of y; otherwise, evaluates to the value of z.

Further, the following relational operators may be applied:

> Greater than

>= Greater than or equal to

< Less than

<= Less than or equal to

— Equal to

!= Not equal to

Further, it should be noted that in the syntax descriptors used herein, the following descriptors may be applied:

b(8): byte having any pattern of bit string (8 bits). The parsing process for this descriptor is specified by the return value of the function read_bits(8).

f(n): fixed-pattern bit string using n bits written (from left to right) with the left bit first. The parsing process for this descriptor is specified by the return value of the function read_bits(n).

se(v): signed integer 0-th order Exp-Golomb-coded syntax element with the left bit first.

tb(v): truncated binary using up to maxVal bits with maxVal defined in the semantics of the syntax element.

15

tu(v): truncated unary using up to maxVal bits with maxVal defined in the semantics of the syntax element.

u(n): unsigned integer using n bits. When n is “v” in the syntax table, the number of bits varies in a manner dependent on the value of other syntax elements. The parsing process for this descriptor is specified by the return value of the function read_bits(n) interpreted as a binary representation of an unsigned integer with most significant bit written first.

ue(v): unsigned integer 0-th order Exp-Golomb-coded syntax element with the left bit first.

st(v): null-terminated string encoded as universal coded character set (UCS) transmission format-8 (UTF-8) characters as specified in ISO/IEC 10646. The parsing process is specified as follows: st(v) begins at a byte-aligned position in the bitstream and reads and returns a series of bytes from the bitstream, beginning at the current position and continuing up to but not including the next byte-aligned byte that is equal to 0x00, and advances the bitstream pointer by (stringLength+1)*8 bit positions, where stringLength is equal to the number of bytes returned.

As described above, ITU-T H.266 defines NAL unit header semantics that specify the type of Raw Byte Sequence Payload (RBSP) data structure included in the NAL unit. Table 2 illustrates the syntax of the NAL unit header provided in ITU-T H.266.

TABLE 2

nal_unit_header() {	Descriptor
forbidden_zero_bit	f(1)
nuh_reserved_zero_bit	u(1)
nuh_layer_id	u(6)
nal_unit_type	u(5)
nuh_temporal_id_plus1	u(3)
}	

ITU-T H.266 provides the following definitions for the respective syntax elements illustrated in Table 2.

forbidden_zero_bit shall be equal to 0.

nuh_reserved_zero_bit shall be equal to 0. The value 1 of nuh_reserved_zero_bit could be specified in the future by ITU-T/ISO/IEC. Although the value of nuh_reserved_zero_bit is required to be equal to 0 in this version of this Specification, decoders conforming to this version of this Specification shall allow the value of nuh_reserved_zero_bit equal to 1 to appear in the syntax and shall ignore (i.e. remove from the bitstream and discard) NAL units with nuh_reserved_zero_bit equal to 1.

nuh_layer_id specifies the identifier of the layer to which a VCL NAL unit belongs or the identifier of a layer to which a non-VCL NAL unit applies. The value of nuh_layer_id shall be in the range of 0 to 55, inclusive. Other values for nuh_layer_id are reserved for future use by ITU-T/ISO/IEC. Although the value of nuh_layer_id is required to be the range of 0 to 55, inclusive, in this version of this Specification, decoders conforming to this version of this Specification shall allow the value of nuh_layer_id to be greater than 55 to appear in the syntax and shall ignore (i.e. remove from the bitstream and discard) NAL units with nuh_layer_id greater than 55.

The value of nuh_layer_id shall be the same for all VCL NAL units of a coded picture. The value of nuh_layer_id of

16

a coded picture or a PU is the value of the nuh_layer_id of the VCL NAL units of the coded picture or the PU.

When nal_unit_type is equal to PH_NUT, or FD_NUT, nuh_layer_id shall be equal to the nuh_layer_id of associated VCL NAL unit.

When nal_unit_type is equal to EOS_NUT, nuh_layer_id shall be equal to one of the nuh_layer_id values of the layers present in the CVS.

NOTE—The value of nuh_layer_id for DCI, OPI, VPS, AUD, and EOB NAL units is not constrained.

nuh_temporal_id_plus1 minus 1 specifies a temporal identifier for the NAL unit.

The value of nuh_temporal_id_plus1 shall not be equal to 0.

The variable TemporalId is derived as follows:

$$TemporalId = nuh_temporal_id_plus1 - 1$$

When nal_unit_type is in the range of IDR_W_RADL to RSV_IRAP_11, inclusive, TemporalId shall be equal to 0.

When nal_unit_type is equal to STSA_NUT and vps_independent_layer_flag[GeneralLayerIdx[nuh_layer_id]] is equal to 1, TemporalId shall be greater than 0.

The value of TemporalId shall be the same for all VCL NAL units of an AU. The value of TemporalId of a coded picture, a PU, or an AU is the value of the TemporalId of the VCL NAL units of the coded picture, PU, or AU. The value of TemporalId of a sublayer representation is the greatest value of TemporalId of all VCL NAL units in the sublayer representation.

The value of TemporalId for non-VCL NAL units is constrained as follows:

If nal_unit_type is equal to DCI_NUT, OPI_NUT, VPS_NUT, or SPS_NUT, TemporalId shall be equal to 0 and the TemporalId of the AU containing the NAL unit shall be equal to 0.

Otherwise, if nal_unit_type is equal to PH_NUT, TemporalId shall be equal to the TemporalId of the PU containing the NAL unit.

Otherwise, if nal_unit_type is equal to EOS_NUT or EOB_NUT, TemporalId shall be equal to 0.

Otherwise, if nal_unit_type is equal to AUD_NUT, FD_NUT, PREFIX_SEI_NUT, or SUFFIX_SEI_NUT, TemporalId shall be equal to the TemporalId of the AU containing the NAL unit.

Otherwise, when nal_unit_type is equal to PPS_NUT, PREFIX_APS_NUT, or SUFFIX_APS_NUT, TemporalId shall be greater than or equal to the TemporalId of the PU containing the NAL unit.

NOTE—When the NAL unit is a non-VCL NAL unit, the value of TemporalId is equal to the minimum value of the TemporalId values of all AUs to which the non-VCL NAL unit applies. When nal_unit_type is equal to PPS_NUT, PREFIX_APS_NUT, or SUFFIX_APS_NUT, TemporalId could be greater than or equal to the TemporalId of the containing AU, as all PPSs and APSs could be included in the beginning of the bitstream (e.g., when they are transported out-of-band, and the receiver places them at the beginning of the bitstream), wherein the first coded picture has TemporalId equal to 0.

nal_unit_type specifies the NAL unit type, i.e., the type of RBSP data structure contained in the NAL unit as specified in Table 3.

NAL units that have nal_unit_type in the range of UNSPEC28 . . . UNSPEC31, inclusive, for which semantics

are not specified, shall not affect the decoding process specified in this Specification.

NOTE—NAL unit types in the range of UNSPEC_28 . . . UNSPEC_31 could be used as determined by the application. No decoding process for these values of nal_unit_type is specified in this Specification. Since different applications might use these NAL unit types for different purposes, particular care is expected to be exercised in the design of encoders that generate NAL units with these nal_unit_type values, and in the design of decoders that interpret the content of NAL units with these nal_unit_type values. This Specification does not define any management for these values. These nal_unit_type values might only be

suitable for use in contexts in which “collisions” of usage (i.e., different definitions of the meaning of the NAL unit content for the same nal_unit_type value) are unimportant, or not possible, or are managed—e.g., defined or managed in the controlling application or transport specification, or by controlling the environment in which bitstreams are distributed.

For purposes other than determining the amount of data in the DUs of the bitstream, decoders shall ignore (remove from the bitstream and discard) the contents of all NAL units that use reserved values of nal_unit_type. NOTE—This requirement allows future definition of compatible extensions to this Specification.

TABLE 3

nal_unit_type	Name of nal_unit_type	Content of NAL unit and RBSP syntax structure	NAL unit type class
0	TRAIL_NUT	Coded slice of a trailing picture or subpicture* slice_layer_rbsp()	VCL
1	STSA_NUT	Coded slice of an STSA picture or subpicture* slice_layer_rbsp()	VCL
2	RADL_NUT	Coded slice of a RADL picture or subpicture* slice_layer_rbsp()	VCL
3	RASL_NUT	Coded slice of a RASL picture or subpicture* slice_layer_rbsp()	VCL
4 . . . 6	RSV_VCL_4 . . . RSV_VCL_6	Reserved non-IRAP VCL NAL unit types	VCL
7	IDR_W_RADL	Coded slice of an IDR picture or subpicture* slice_layer_rbsp()	VCL
8	IDR_N_LP	Coded slice of a CRA picture or subpicture* slice_layer_rbsp()	VCL
9	CRA_NUT	Coded slice of a CRA picture or subpicture* slice_layer_rbsp()	VCL
10	GDR_NUT	Coded slice of a GDR picture or subpicture* slice_layer_rbsp()	VCL
11	RSV_IRAP_11	Reserved IRAP VCL NAL unit type	VCL
12	OPI_NUT	Operating point information operating_point_information_rbsp()	non-VCL
13	DCI_NUT	Decoding capability information decoding_capability_information_rbsp()	non-VCL
14	VPS_NUT	Video parameter set video_parameter_set_rbsp()	non-VCL
15	SPS_NUT	Sequence parameter set seq_parameter_set_rbsp()	non-VCL
16	PPS_NUT	Picture parameter set pic_parameter_set_rbsp()	non-VCL
17	PREFIX_APS_NUT	Adaptation parameter set	non-VCL
18	SUFFIX_APS_NUT	adaptation_parameter_set_rbsp()	non-VCL
19	PH_NUT	Picture header picture_header_rbsp()	non-VCL
20	AUD_NUT	AU delimiter access_unit_delimiter_rbsp()	non-VCL
21	EOS_NUT	End of sequence end_of_seq_rbsp()	non-VCL
22	EOB_NUT	End of bitstream end_of_bitstream_rbsp()	non-VCL
23	PREFIX_SEI_NUT	Supplemental enhancement information	non-VCL
24	SUFFIX_SEI_NUT	sei_rbsp()	non-VCL
25	FD_NUT	Filler data filler_data_rbsp()	non-VCL
26	RSV_NVCL_26	Reserved non-VCL NAL unit types	non-VCL
27	RSV_NVCL_27		non-VCL
28 . . . 31	UNSPEC_28 . . . UNSPEC_31	Unspecified non-VCL NAL unit types	non-VCL

*indicates a property of a picture when pps_mixed_nalu_types_in_pic_flag is equal to 0 and a property of the subpicture when pps_mixed_nalu_types_in_pic_flag is equal to 1.

19

NOTE—A clean random access (CRA) picture may have associated RASL or RADL pictures present in the bitstream.

NOTE—An instantaneous decoding refresh (IDR) picture having nal_unit_type equal to IDR_N_LP does not have associated leading pictures present in the bitstream. An IDR picture having nal_unit_type equal to IDR_W_RADL does not have associated RASL pictures present in the bitstream, but may have associated RADL pictures in the bitstream.

The value of nal_unit_type shall be the same for all VCL NAL units of a subpicture. A subpicture is referred to as having the same NAL unit type as the VCL NAL units of the subpicture.

For VCL NAL units of any particular picture, the following applies:

If pps_mixed_nalu_types_in_pic_flag is equal to 0, the value of nal_unit_type shall be the same for all VCL NAL units of a picture, and a picture or a PU is referred to as having the same NAL unit type as the coded slice NAL units of the picture or PU.

Otherwise (pps_mixed_nalu_types_in_pic_flag is equal to 1), all of the following constraints apply:

The picture shall have at least two subpictures.

VCL NAL units of the picture shall have two or more different nal_unit_type values.

There shall be no VCL NAL unit of the picture that has nal_unit_type equal to GDR_NUT.

When a VCL NAL unit of the picture has nal_unit_type equal to nalUnitTypeA that is equal to IDR_W_RADL, IDR_N_LP, or CRA_NUT, other VCL NAL units of the picture shall all have nal_unit_type equal to nalUnitTypeA or TRAIL_NUT.

The value of nal_unit_type shall be the same for all pictures in an IRAP or GDR AU.

When sps_video_parameter_set_id is greater than 0, vps_max_tid_il_ref_pics_plus1[i][j] is equal to 0 for j equal to GeneralLayerIdx[nuh_layer_id] and any value of i in the range of j+1 to vps_max_layers_minus1, inclusive, and pps_mixed_nalu_types_in_pic_flag is equal to 1, the value of nal_unit_type shall not be equal to IDR_W_RADL, IDR_N_LP, or CRA_NUT.

It is a requirement of bitstream conformance that the following constraints apply:

When a picture is a leading picture of an IRAP picture, it shall be a RADL or RASL picture.

When a subpicture is a leading subpicture of an IRAP subpicture, it shall be a RADL or RASL subpicture.

When a picture is not a leading picture of an IRAP picture, it shall not be a RADL or RASL picture.

When a subpicture is not a leading subpicture of an IRAP subpicture, it shall not be a RADL or RASL subpicture.

No RASL pictures shall be present in the bitstream that are associated with an IDR picture.

No RASL subpictures shall be present in the bitstream that are associated with an IDR subpicture.

No RADL pictures shall be present in the bitstream that are associated with an IDR picture having nal_unit_type equal to IDR_N_LP.

NOTE—It is possible to perform random access at the position of an IRAP AU by discarding all PUs before the IRAP AU (and to correctly decode the non-RASL pictures in the IRAP AU and all the subsequent AUs in decoding order), provided each parameter set is available (either in the bitstream or by external means not specified in this Specification) when it is referenced.

20

No RADL subpictures shall be present in the bitstream that are associated with an IDR subpicture having nal_unit_type equal to IDR_N_LP.

Any picture, with nuh_layer_id equal to a particular value layerId, that precedes an IRAP picture with nuh_layer_id equal to layerId in decoding order shall precede the IRAP picture in output order and shall precede any RADL picture associated with the IRAP picture in output order.

Any subpicture, with nuh_layer_id equal to a particular value layerId and subpicture index equal to a particular value subpicIdx, that precedes, in decoding order, an IRAP subpicture with nuh_layer_id equal to layerId and subpicture index equal to subpicIdx shall precede, in output order, the IRAP subpicture and all its associated RADL subpictures.

Any picture, with nuh_layer_id equal to a particular value layerId, that precedes a recovery point picture with nuh_layer_id equal to layerId in decoding order shall precede the recovery point picture in output order.

Any subpicture, with nuh_layer_id equal to a particular value layerId and subpicture index equal to a particular value subpicIdx, that precedes, in decoding order, a subpicture with nuh_layer_id equal to layerId and subpicture index equal to subpicIdx in a recovery point picture shall precede that subpicture in the recovery point picture in output order.

Any RASL picture associated with a CRA picture shall precede any RADL picture associated with the CRA picture in output order.

Any RASL subpicture associated with a CRA subpicture shall precede any RADL subpicture associated with the CRA subpicture in output order.

Any RASL picture, with nuh_layer_id equal to a particular value layerId, associated with a CRA picture shall follow, in output order, any IRAP or GDR picture with nuh_layer_id equal to layerId that precedes the CRA picture in decoding order.

Any RASL subpicture, with nuh_layer_id equal to a particular value layerId and subpicture index equal to a particular value subpicIdx, associated with a CRA subpicture shall follow, in output order, any IRAP or GDR subpicture, with nuh_layer_id equal to layerId and subpicture index equal to subpicIdx, that precedes the CRA subpicture in decoding order.

If sps_field_seq_flag is equal to 0, the following applies: when the current picture, with nuh_layer_id equal to a particular value layerId, is a leading picture associated with an IRAP picture, it shall precede, in decoding order, all non-leading pictures that are associated with the same IRAP picture. Otherwise (sps_field_seq_flag is equal to 1), let picA and picB be the first and the last leading pictures, in decoding order, associated with an IRAP picture, respectively, there shall be at most one non-leading picture with nuh_layer_id equal to layerId preceding picA in decoding order, and there shall be no non-leading picture with nuh_layer_id equal to layerId between picA and picB in decoding order.

If sps_field_seq_flag is equal to 0, the following applies: when the current subpicture, with nuh_layer_id equal to a particular value layerId and subpicture index equal to a particular value subpicIdx, is a leading subpicture associated with an IRAP subpicture, it shall precede, in decoding order, all non-leading subpictures that are associated with the same IRAP subpicture. Otherwise (sps_field_seq_flag is equal to 1), let subpicA and subpicB be the first and the last leading subpictures, in

21

decoding order, associated with an IRAP subpicture, respectively, there shall be at most one non-leading subpicture with nuh_layer_id equal to layerId and subpicture index equal to subpicIdx preceding subpicA in decoding order, and there shall be no non-leading picture with nuh_layer_id equal to layerId and subpicture index equal to subpicIdx between picA and picB in decoding order.

As provided in Table 3, a NAL unit may include an supplemental enhancement information (SEI) syntax structure. Table 4 and Table 5 illustrate the supplemental enhancement information (SEI) syntax structure provided in ITU-T H.266.

TABLE 4

sei_rbsp() {	Descriptor
do	
sei_message()	
while(more_rbsp_data())	
rbbsp_trailing_bits()	
}	

TABLE 5

sei_message() {	Descriptor
payloadType = 0	
do {	
payload_type_byte	u(8)
payloadType += payload_type_byte	
} while(payload_type_byte == 0xFF)	
payloadSize = 0	
do {	
payload_size_byte	u(8)
payloadSize += payload_size_byte	
} while(payload_size_byte == 0xFF)	
sei_payload(payloadType, payloadSize)	
}	

22

With respect to Table 4 and Table 5, ITU-T H.266 provides the following semantics:

Each SEI message consists of the variables specifying the type payloadType and size payloadSize of the SEI message payload. SEI message payloads are specified. The derived SEI message payload size payloadSize is specified in bytes and shall be equal to the number of RBSP bytes in the SEI message payload.

NOTE—The NAL unit byte sequence containing the SEI message might include one or more emulation prevention bytes (represented by emulation_prevention_three_byte syntax elements). Since the payload size of an SEI message is specified in RBSP bytes, the quantity of emulation prevention bytes is not included in the size payloadSize of an SEI payload.

payload_type_byte is a byte of the payload type of an SEI message.

payload_size_byte is a byte of the payload size of an SEI message.

It should be noted that ITU-T H.266 defines payload types and “Additional SEI messages for VSEI (Draft 6)” 25th Meeting of ISO/IEC JTC1/SC29/WG11 12-21 Jan. 2022, Teleconference, document JVET-Y2006-v1, which is incorporated by reference herein, and referred to as JVET-Y2006, defines additional payload types. Further, “New level and systems-related supplemental enhancement information for VVC (Draft 5)” 30th Meeting of ISO/IEC JTC1/SC29/WG11 21-28 Apr. 2023, Antalya, TR, document JVET-AD2005-v1, which is incorporated by reference herein, and referred to as JVET-AD2005, defines additional payload types.

Table 6 generally illustrates an sei_payload() syntax structure. That is, Table 6 illustrates the sei_payload() syntax structure, but for the sake of brevity, all of the possible types of payloads are not included in Table 6.

TABLE 6

sei_payload(payloadType, payloadSize) {	Descriptor
SeiExtensionBitsPresentFlag = 0	
if(nal_unit_type == PREFIX_SEI_NUT)	
if(payloadType == 0)	
buffering_period(payloadSize)	
else if(payloadType == 1)	
...	
else if(payloadType == 210) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
nn_post_filter_characteristics(payloadSize)	
else if(payloadType == 211) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
nn_post_filter_activation(payloadSize)	
else if(payloadType == 212) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
phase_indication(payloadSize)	
else /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
reserved_message(payloadSize)	
else /* nal_unit_type == SUFFIX_SEI_NUT */	
if(payloadType == 3) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
filler_payload(payloadSize)	
else if(payloadType == 4) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
user_data_registered_itu_t35(payloadSize)	
else if(payloadType == 5) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
user_data_unregistered(payloadSize)	
else if(payloadType == 22) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
post_filter_hint(payloadSize)	
else if(payloadType == 132) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
decoded_picture_hash(payloadSize)	
else if(payloadType == 133)	
scalable_nesting(payloadSize)	
else if(payloadType == 210) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
nn_post_filter_characteristics(payloadSize)	

TABLE 6-continued

sei_payload(payloadType, payloadSize) {	Descriptor
else if(payloadType == 211) /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
nn_post_filter_activation(payloadSize)	
else /* Specified in Rec. ITU-T H.274 ISO/IEC 23002-7 */	
reserved_message(payloadSize)	
if(SeiExtensionBitsPresentFlag more_data_in_payload()) {	
if(payload_extension_present())	
sei_reserved_payload_extension_data	u(v)
sei_payload_bit_equal_to_one /* equal to 1 */	f(1)
while(!byte_aligned())	
sei_payload_bit_equal_to_zero /* equal to 0 */	f(1)
}	
}	

15

With respect to Table 6, JVET-AD2005 provides the following semantics:

sei_reserved_payload_extension_data shall not be present in bitstreams conforming to this version of this Specification. However, decoders conforming to this version of this Specification shall ignore the presence and value of sei_reserved_payload_extension_data. When present, the length, in bits, of sei_reserved_payload_extension_data is equal to $8 \times \text{payloadSize} - \text{nEarlierBits} - \text{nPayloadZeroBits} - 1$, where nEarlierBits is the number of bits in the sei_payload() syntax structure that precede the sei_reserved_payload_extension_data syntax element, and nPayloadZeroBits is the number of sei_payload_bit_equal_to_zero syntax elements at the end of the sei_payload() syntax structure.

If more_data_in_payload() is TRUE after the parsing of the SEI message syntax structure (e.g., the buffering_period() syntax structure) and nPayloadZeroBits is not equal to 7, PayloadBits is set equal to $8 \times \text{payloadSize} - \text{nPayloadZeroBits} - 1$; otherwise, PayloadBits is set equal to $8 \times \text{payloadSize}$.

sei_payload_bit_equal_to_one shall be equal to 1.

sei_payload_bit_equal_to_zero shall be equal to 0.

NOTE—SEI messages with the same value of payloadType are conceptually the same SEI message regardless of whether they are contained in prefix or suffix SEI NAL units.

NOTE—For SEI messages specified in this Specification and the VSEI specification (ITU-T H.274|ISO/IEC 23002-7), the payloadType values are aligned with similar SEI messages specified in AVC (Rec. ITU-T H.264|ISO/IEC 14496-10) and HEVC (Rec. ITU-T H.265|ISO/IEC 23008-2).

The semantics and persistence scope for each SEI message are specified in the semantics specification for each particular SEI message.

NOTE—Persistence information for SEI messages is informatively summarized.

JVET-AD2005 further provides the following:

The SEI messages having syntax structures identified in [Table 6] that are specified in other specifications, including Rec. ITU-T H.274|ISO/IEC 23002-7, ISO/IEC 23001-11, or ISO/IEC 23090-13, may be used together with bitstreams specified by this Specification.

When any particular SEI message specified in Rec. ITU-T H.274|ISO/IEC 23002-7, ISO/IEC 23001-11, or ISO/IEC 23090-13 is included in a bitstream specified by this Specification, the SEI payload syntax shall be as specified in Rec. ITU-T H.274|ISO/IEC 23002-7, ISO/IEC 23001-11, or ISO/IEC 23090-13, respectively, that syntax shall be included into the sei_payload() syntax structure as specified in clause

D.2.1 and shall use the payloadType value specified in clause D.2.1, the corresponding semantics specified in Rec. ITU-T H.274|ISO/IEC 23002-7, ISO/IEC 23001-11, or ISO/IEC 23090-13 shall apply, and, additionally, any SEI-message-specific constraints, variables, and semantics specified in this annex for that particular SEI message shall apply.

The value of PayloadBits, as specified in clause D.2.2, is passed to the parser of the SEI message syntax structures specified in Rec. ITU-T H.274|ISO/IEC 23002-7, ISO/IEC 23001-11, and ISO/IEC 23090-13.

NOTE—The definition of IRAP picture in the VSEI specification is as follows: A coded picture starting from which all pictures in the same layer in both decoding order and output order can be decoded without first decoding any picture in the same layer earlier in decoding order in the coded video bitstream. Consequently, a GDR picture with ph_recovery_poc_cnt equal to 0 in a VVC bitstream is an IRAP picture according to the IRAP picture definition in the VSEI specification.

JVET-AD2005 further provides the following with respect to the use of neural network post-filter characteristics SEI message:

Let currPic be the cropped decoded output picture for which the neural-network post-processing filter (NNPF) defined by the neural-network post-filter characteristics (NNPFC) SEI message is activated by a neural-network post-filter activation (NNPFA) SEI message and currLayerId be the nuh_layer_id value of currPic.

It is a requirement of bitstream conformance that when a picture unit contains an NNPF SEI message, the value of ph_pic_output_flag in the picture header contained in that picture unit shall be equal to 1.

NOTE—Since only cropped decoded output pictures are used as input pictures of the NNPF, the value of ph_pic_output_flag in the picture header of the coded picture corresponding to each input picture of the NNPF is equal to 1.

The variable pictureRateUpsamplingFlag is set equal to (nnpfc_purpose & 0x08) != 0.

The variable numInputPics is set equal to nnpfc_num_input_pics_minus1 + 1.

The variable numInferences is derived as follows:

If pictureRateUpsamplingFlag is equal to 1, the NNPF SEI message that activated this NNPF has nnpfa_persistence_flag equal to 1, nnpfc_interpolated_pics[i] is greater than 0 only for a single value of i that is greater than 0, and currPic is the last picture of the bitstream in output order that has nuh_layer_id equal to currLayerId, the variable numPostRoll is set equal to the value

60

65

25

of i such that $\text{nnpfc_interpolated_pics}[i]$ is greater than 0 and the variable numInferences is set equal to $1 + \text{numPostRoll}$.

Otherwise, the variable numInferences is set equal to 1. For each value of j in the range of 0 to $\text{numInferences}-1$, inclusive, the following applies:

The arrays $\text{inputPic}[i]$ and $\text{inputPresentFlag}[i]$ for i in the range of 0 to $\text{numInputPics}-1$, inclusive, representing all the input pictures and the presence of input pictures within the bitstream, respectively, are specified as follows:

When j is greater than 0, for each value of k in the range of 0 to $j-1$, inclusive, $\text{inputPic}[k]$ is set to be currPic and $\text{inputPresentFlag}[k]$ is set equal to 0.

The j -th input picture, $\text{inputPic}[j]$, is set equal to be currPic and $\text{inputPresentFlag}[j]$ is set equal to 1.

When numInputPics is greater than 1, the following applies for each value of i in the range of $j+1$ to $\text{numInputPics}-j-1$, inclusive, in increasing order of i :

If $\text{pictureRateUpsamplingFlag}$ is equal to 1, currPic is associated with a frame packing arrangement SEI message with $\text{fp_arrangement_type}$ equal to 5 and a particular value of $\text{fp_current_frame_is_frame0_flag}$, and there is a cropped decoded output picture prevPic that is the last picture in output order among all cropped decoded output pictures that have nuh_layer_id equal to currLayerId , precede $\text{inputPic}[i-1]$ in output order, and are associated with a frame packing arrangement SEI message with $\text{fp_arrangement_type}$ equal to 5 and the same value of $\text{fp_current_frame_is_frame0_flag}$, $\text{inputPic}[i]$ is set to be prevPic and $\text{inputPresentFlag}[i]$ is set equal to 1.

Otherwise, if $\text{pictureRateUpsamplingFlag}$ is equal to 0 and there is a picture prevPic that is the last picture in output order among all cropped decoded output pictures that have nuh_layer_id equal to currLayerId and precede $\text{inputPic}[i-1]$ in output order, $\text{inputPic}[i]$ is set to be prevPic and $\text{inputPresentFlag}[i]$ is set equal to 1.

Otherwise, if currPic is not associated with a frame packing arrangement SEI message with $\text{fp_arrangement_type}$ equal to 5 and there is a cropped decoded output picture prevPic that is the last picture in output order among all cropped decoded output pictures that have nuh_layer_id equal to currLayerId and precede $\text{inputPic}[i-1]$ in output order, $\text{inputPic}[i]$ is set to be prevPic and $\text{inputPresentFlag}[i]$ is set equal to 1.

Otherwise, the following applies:

$\text{inputPic}[i]$ is set to be the same picture as $\text{inputPic}[i-1]$ and $\text{inputPresentFlag}[i]$ is set equal to 0.

It is a requirement of bitstream conformance that $\text{num_interpolated_pics}[i-1]$ shall not be greater than 0.

For purposes of interpretation of the NNPFC SEI message, the following variables are specified:

If numInputPics is greater than 1 and there is a second NNPFC that is defined by at least one NNPFC SEI message, is activated by an NNPFA SEI message for currPic , and has nnpfc_purpose equal to 4, the following applies:

CroppedWidth is set equal to $\text{nnpfc_pic_width_in_luma_samples}$ defined for the second NNPFC.

26

CroppedHeight is set equal to $\text{nnpfc_pic_height_in_luma_samples}$ defined for the second NNPFC.

Otherwise, the following applies:

CroppedWidth is set equal to the value of $\text{pps_pic_width_in_luma_samples} - \text{SubWidthC} * (\text{pps_conf_win_left_offset} + \text{pps_conf_win_right_offset})$ for currPic .

CroppedHeight is set equal to the value of $\text{pps_pic_height_in_luma_samples} - \text{SubHeightC} * (\text{pps_conf_win_top_offset} + \text{pps_conf_win_bottom_offset})$ for currPic .

The luma sample arrays $\text{CroppedYPic}[i]$ and the chroma sample arrays $\text{CroppedCbPic}[i]$ and $\text{CroppedCrPic}[i]$, when present, are derived as follows for each value of i in the range of 0 to $\text{numInputPics}-1$, inclusive:

The variable sourcePic is derived as follows:

If $\text{inputPresentFlag}[i]$ is equal to 1 or $\text{nnpfc_absent_input_pic_zero_flag}$ is equal to 0, sourcePic is set to be $\text{inputPic}[i]$.

Otherwise ($\text{inputPresentFlag}[i]$ is equal to 0 and $\text{nnpfc_absent_input_pic_zero_flag}$ is equal to 1), sourcePic is set to be a picture with a luma sample array of $\text{CroppedWidth} \times \text{CroppedHeight}$ samples equal to 0 and Cb and Cr sample arrays of $(\text{CroppedWidth}/\text{SubWidthC}) \times (\text{CroppedHeight}/\text{SubHeightC})$ samples equal to 0.

If numInputPics is equal to 1, the following applies:

The luma sample array $\text{CroppedYPic}[i]$ and the chroma sample arrays $\text{CroppedCbPic}[i]$ and $\text{CroppedCrPic}[i]$, when present, are set to be the 2-dimensional arrays of decoded sample values of the Y, Cb and Cr components, respectively, of sourcePic .

Otherwise (numInputPics is greater than 1), the following applies:

The variable sourceWidth is set equal to the value of $\text{pps_pic_width_in_luma_samples} - \text{SubWidthC} * (\text{pps_conf_win_left_offset} + \text{pps_conf_win_right_offset})$ for sourcePic .

The variable sourceHeight is set equal to the value of $\text{pps_pic_height_in_luma_samples} - \text{SubHeightC} * (\text{pps_conf_win_top_offset} + \text{pps_conf_win_bottom_offset})$ for sourcePic .

If sourceWidth is equal to CroppedWidth and sourceHeight is equal to CroppedHeight , inputPic is set to be the same as sourcePic .

Otherwise (sourceWidth is not equal to CroppedWidth or sourceHeight is not equal to CroppedHeight), the following applies:

There shall be an NNPFC, hereafter referred to as the super resolution NNPFC, that is defined by at least one NNPFC SEI message, is activated by an NNPFA SEI message for sourcePic , and has nnpfc_purpose equal to 4, $\text{nnpfc_pic_width_in_luma_samples}$ equal to CroppedWidth and $\text{nnpfc_pic_height_in_luma_samples}$ equal to CroppedHeight . resampledPic is set to be the output of the neural-network inference of the super resolution NNPFC with sourcePic being an input.

The luma sample array $\text{CroppedYPic}[i]$ and the chroma sample arrays $\text{CroppedCbPic}[i]$ and $\text{CroppedCrPic}[i]$, when present, are set to be the 2-dimensional arrays of decoded sample values of the Y, Cb and Cr components, respectively, of resampledPic .

27

BitDepth_Y and BitDepth_C are both set equal to Bit-Depth.

ChromaFormatIdc is set equal to sps_chroma_format_idc.

The array StrengthControlVal[i] for all values of i in the range of 0 to numInputPics-1, inclusive, specifying the filtering strength control value for the input pictures for the NNPF, is derived as follows:

StrengthControlVal[i] is set equal to the value of (firstSliceQp_Y+QpBdOffset)+(63+QpBdOffset), where firstSliceQp_Y is equal to SliceQp_Y of the first slice of inputPic[i].

There shall not be more than two NNPF SEI messages present in a picture unit with the same value of nnpfc_id. When there are two NNPF SEI messages present in a picture unit with the same value of nnpfc_id, these SEI messages shall have different content. When two NNPF SEI messages with the same nnpfc_id and different content are present in the same picture unit, both of these NNPF SEI messages shall be in the same SEI NAL unit.

When a picture contains an NNPF, regardless the value of nnpfa_cancel_flag in the NNPF, the picture shall not be a non-output picture.

As described above, JVET-AD2005 provides a NN post-filter supplemental enhancement information messages. In particular, JVET-AD2005 provides a Neural-network post-filter characteristics SEI message (payloadType==210) and a Neural-network post-filter activation SEI message (payloadType==211). Table 7 illustrates the syntax of the Neural-network post-filter characteristics SEI message provided in JVET-AD2006. It should be noted that a Neural-network post-filter characteristics SEI message may be referred to as a NNPF SEI. It should be noted that JVET-AD2006 the following general process for Neural-network post-filters:

Input to this process is a bitstream BitstreamToFilter. Output of this process is a list of NNPF output pictures ListNnpfOutputPics.

First, BitstreamToFilter is decoded, and the list CroppedDecodedPictures is set to be the list of the cropped decoded pictures in output order resulted from decoding BitstreamToFilter.

Second, the filtering process for one picture, as specified, is repeatedly invoked, in output order, for each cropped decoded picture that is in CroppedDecodedPictures and for which one or more NNPFs are activated.

The order of the pictures in ListNnpfOutputPics is in output order.

Within ListNnpfOutputPics there shall be no more than one picture pertaining to any particular output time instance. When for any particular picture in CroppedDecodedPictures there are multiple NNPFs activated and only one the NNPFs is allowed to be chosen to be applied although any of the NNPFs may be chosen, the above constraint shall apply regardless of which NNPF is chosen to be applied to the particular picture.

Further, it should be noted that JVET-AD2006 the following filtering process for one picture using a Neural-network post-filter:

The filtering process specified in this subclause applies to each cropped decoded picture, referred to as the current picture, that is in CroppedDecodedPictures and for which one or more NNPFs are activated.

When applying an NNPF to the current picture, the filtered and/or interpolated pictures are generated by the NNPF by applying the NNPF process specified in the semantics of the NNPF SEI message, in a patch-wise manner, to the current picture.

28

When applying an NNPF to the current picture, the order of the pictures generated by the NNPF by applying the NNPF process being stored into the output tensor of the NNPF is in output order.

When the applied NNPF is the last NNPF that is applied to the current picture, the pictures generated by the NNPF and output by the NNPF process are included into ListNnpfOutputPics, in the same order as when the pictures are stored into the output tensor of the NNPF.

TABLE 7

nn_post_filter_characteristics(payloadSize) {	Descriptor
nnpfc_purpose	u(16)
nnpfc_id	ue(v)
nnpfc_base_flag	u(1)
nnpfc_mode_idc	ue(v)
if(nnpfc_mode_idc == 1) {	
while(!byte_aligned())	
nnpfc_reserved_zero_bit_a	u(1)
nnpfc_tag_uri	st(v)
nnpfc_uri	st(v)
}	
nnpfc_property_present_flag	u(1)
if(nnpfc_property_present_flag) {	
/* input and output formatting */	
nnpfc_num_input_pics_minus1	ue(v)
if(nnpfc_num_input_pics_minus1 > 0) {	
for(i = 0; i <= nnpfc_num_input_pics_minus1; i++)	
nnpfc_input_pic_output_flag[i]	u(1)
nnpfc_absent_input_pic_zero_flag	u(1)
}	
if(chromaUpsamplingFlag)	
nnpfc_out_sub_c_flag	u(1)
if(colourizationFlag)	
nnpfc_out_colour_format_idc	u(2)
if(resolutionResamplingFlag) {	
nnpfc_pic_width_num_minus1	ue(v)
nnpfc_pic_width_denom_minus1	ue(v)
nnpfc_pic_height_num_minus1	ue(v)
nnpfc_pic_height_denom_minus1	ue(v)
}	
if(pictureRateUpsamplingFlag)	
for(i = 0; i < nnpfc_num_input_pics_minus1; i++)	
nnpfc_interpolated_pics[i]	ue(v)
nnpfc_component_last_flag	u(1)
nnpfc_inp_format_idc	ue(v)
nnpfc_auxiliary_inp_idc	ue(v)
nnpfc_inp_order_idc	ue(v)
if(nnpfc_inp_format_idc == 1) {	
if(nnpfc_inp_order_idc != 1)	
nnpfc_inp_tensor_luma_bitdepth_minus8	ue(v)
if(nnpfc_inp_order_idc != 0)	
nnpfc_inp_tensor_chroma_bitdepth_minus8	ue(v)
}	
nnpfc_out_format_idc	ue(v)
nnpfc_out_order_idc	ue(v)
if(nnpfc_out_format_idc == 1) {	
if(nnpfc_out_order_idc != 1)	
nnpfc_out_tensor_luma_bitdepth_minus8	ue(v)
if(nnpfc_out_order_idc != 0)	
nnpfc_out_tensor_chroma_bitdepth_minus8	ue(v)
}	
nnpfc_separate_colour_description_present_flag	u(1)
if(nnpfc_separate_colour_description_present_flag) {	
nnpfc_colour_primaries	u(8)
nnpfc_transfer_characteristics	u(8)
if(nnpfc_out_format_idc == 1) {	
nnpfc_matrix_coeffs	u(8)
nnpfc_full_range_flag	u(1)
}	
}	
nnpfc_chroma_loc_info_present_flag	u(1)
if(nnpfc_chroma_loc_info_present_flag)	
nnpfc_chroma_sample_loc_type_frame	ue(v)
nnpfc_overlap	ue(v)
nnpfc_constant_patch_size_flag	u(1)
if(nnpfc_constant_patch_size_flag) {	
nnpfc_patch_width_minus1	ue(v)

TABLE 7-continued

nn_post_filter_characteristics(payloadSize) {	Descriptor	
nnpfc_patch_height_minus1	ue(v)	5
} else {		
nnpfc_extended_patch_width_cd_delta_minus1	ue(v)	
nnpfc_extended_patch_height_cd_delta_minus1	ue(v)	
}		
nnpfc_padding_type	ue(v)	10
if(nnpfc_padding_type == 4) {		
if(nnpfc_inp_order_idc != 1)		
nnpfc_luma_padding_val	ue(v)	
if(nnpfc_inp_order_idc != 0) {		
nnpfc_cb_padding_val	ue(v)	
nnpfc_cr_padding_val	ue(v)	
}		
}		
nnpfc_complexity_info_present_flag	u(1)	15
if(nnpfc_complexity_info_present_flag) {		
nnpfc_parameter_type_idc	u(2)	
if(nnpfc_parameter_type_idc != 2)		
nnpfc_log2_parameter_bit_length_minus3	u(2)	
nnpfc_num_parameters_idc	u(6)	
nnpfc_num_kmac_operations_idc	ue(v)	20
nnpfc_total_kilobyte_size	ue(v)	
}		
nnpfc_metadata_extension_num_bits	ue(v)	25
if(nnpfc_metadata_extension_num_bits > 0)		
nnpfc_reserved_metadata_extension	u(v)	
}		
/* ISO/IEC 15938-17 bitstream */		
if(nnpfc_mode_idc == 0) {		
while(!byte_aligned())		
nnpfc_reserved_zero_bit_b	u(1)	
for(i = 0; more_data_in_payload(); i++)		
nnpfc_payload_byte[i]	b(8)	30
}		
}		

With respect to Table 7, JVET-AD2006 provides the following semantics:

The neural-network post-filter characteristics (NNPFC) SEI message specifies a neural network that may be used as a post-processing filter. The use of specified neural-network post-processing filters (NNPFs) for specific pictures is indicated with neural-network post-filter activation (NNPFA) SEI messages.

Use of this SEI message requires the definition of the following variables:

Input picture width and height in units of luma samples, denoted herein by *CroppedWidth* and *CroppedHeight*, respectively.

Luma sample array *CroppedYPic[idx]* and chroma sample arrays *CroppedCbPic[idx]* and *CroppedCrPic[idx]*, when present, of the input pictures with index *idx* in the range of 0 to *numInputPics*−1, inclusive, that are used as input for the NNPF.

Bit depth *BitDepth_Y* for the luma sample array of the input pictures.

Bit depth *BitDepth_C* for the chroma sample arrays, if any, of the input pictures.

A chroma format indicator, denoted herein by *ChromaFormatIdc*, as described in subclause 7.3.

When *nnpfc_auxiliary_inp_idc* is equal to 1, a filtering strength control value array *StrengthControlVal[idx]* that shall contain real numbers in the range of 0 to 1, inclusive, of the input pictures with index *idx* in the range of 0 to *numInputPics*−1, inclusive.

Input picture with index 0 corresponds to the picture for which the NNPF defined by this NNPFC SEI message is activated by an NNPFA SEI message. Input picture with index *i* in the range of 1 to *numInputPics*−1, inclusive, precedes the input picture with index *i*−1 in output order.

The variables *SubWidthC* and *SubHeightC* are derived from *ChromaFormatIdc* as specified by Table 1.

NOTE—More than one NNPFC SEI message can be present for the same picture. When more than one NNPFC SEI message with different values of *nnpfc_id* is present or activated for the same picture, they can have the same or different values of *nnpfc_purpose* and *nnpfc_mode_idc*.

nnpfc_purpose indicates the purpose of the NNPF as specified in Table 8, where (*nnpfc_purpose* & *bitMask*) not equal to 0 indicates that the NNPF has the purpose associated with the *bitMask* value in Table 8. When *nnpfc_purpose* is greater than 0 and (*nnpfc_purpose* & *bitMask*) is equal to 0, the purpose associated with the *bitMask* value is not applicable to the NNPF. When *nnpfc_purpose* is equal to 0, the NNPF may be used as determined by the application. The value of *nnpfc_purpose* shall be in the range of 0 to 63, inclusive, in bitstreams conforming to this edition of this document. Values of 64 to 65 535, inclusive, for *nnpfc_purpose* are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SEI messages with *nnpfc_purpose* in the range of 64 to 65 535, inclusive.

TABLE 8

bitMask	Interpretation
0x01	General visual quality improvement
0x02	Chroma upsampling (from the 4:2:0 chroma format to the 4:2:2 or 4:4:4 chroma format, or from the 4:2:2 chroma format to the 4:4:4 chroma format)
0x04	Resolution resampling (increasing or decreasing the width or height)
0x08	Picture rate upsampling
0x10	Bit depth upsampling (increasing the luma bit depth or the chroma bit depth)
0x20	Colourization

The variables *chromaUpsamplingFlag*, *resolutionResamplingFlag*, *pictureRateUpsamplingFlag*, and *colourizationFlag*, specifying whether *nnpfc_purpose* indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

$$\text{chromaUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0 \times 02) > 0) ? 1:0$$

$$\text{resolutionResamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0 \times 04) > 0) ? 1:0$$

$$\text{pictureRateUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0 \times 08) > 0) ? 1:0$$

$$\text{bitDepthUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0 \times 10) > 0) ? 1:0$$

$$\text{colourizationFlag} = ((\text{nnpfc_purpose} \ \& \ 0 \times 20) > 0) ? 1:0$$

NOTE—When a reserved value of *nnpfc_purpose* is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by *nnpfc_purpose* being equal to that value.

When *ChromaFormatIdc* is equal to 3, *chromaUpsamplingFlag* shall be equal to 0.

When *ChromaFormatIdc* or *chromaUpsamplingFlag* is not equal to 0, *colourizationFlag* shall be equal to 0.

31

When `pictureRateUpsamplingFlag` is equal to 1 and the input picture with index 0 is associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5, all input pictures are associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5 and the same value of `fp_current_frame_is_frame0_flag`.

`nnpfc_id` contains an identifying number that may be used to identify an NNPF. The value of `nnpfc_id` shall be in the range of 0 to $2^{32}-2$, inclusive. Values of `nnpfc_id` from 256 to 511, inclusive, and from 2^{31} to $2^{32}-2$, inclusive, are reserved for future use by ITU-T/ISO/IEC. Decoders conforming to this edition of this document encountering an NNPF SEI message with `nnpfc_id` in the range of 256 to 511, inclusive, or in the range of 2^{31} to $2^{32}-2$, inclusive, shall ignore the SEI message.

When an NNPF SEI message is the first NNPF SEI message, in decoding order, that has a particular `nnpfc_id` value within the current CLVS, the following applies:

This SEI message specifies a base NNPF.

This SEI message pertains to the current decoded picture and all subsequent decoded pictures of the current layer, in output order, until the end of the current CLVS. `nnpfc_base_flag` equal to 1 specifies that the SEI message specifies the base NNPF. `nnpfc_base_flag` equal to 0 specifies that the SEI message specifies an update relative to the base NNPF.

The following constraints apply to the value of `nnpfc_base_flag`:

When an NNPF SEI message is the first NNPF SEI message, in decoding order, that has a particular `nnpfc_id` value within the current CLVS, the value of `nnpfc_base_flag` shall be equal to 1.

When an NNPF SEI message `nnpfcB` is not the first NNPF SEI message, in decoding order, that has a particular `nnpfc_id` value within the current CLVS and the value of `nnpfc_base_flag` is equal to 1, the NNPF SEI message shall be a repetition of the first NNPF SEI message `nnpfcA` with the same `nnpfc_id` value, in decoding order, i.e., the payload content of `nnpfcB` shall be the same as that of `nnpfcA`.

When `nnpfc_base_flag` is equal to 0, the following applies:

This SEI message defines an update relative to the preceding base NNPF in decoding order with the same `nnpfc_id` value. Updates are not cumulative but rather each update is applied on the base NNPF, which is the NNPF specified by the first NNPF SEI message, in decoding order, that has a particular `nnpfc_id` value within the current CLVS. The NNPF defined by this SEI message is obtained by applying the update defined by this SEI message relative to the base NNPF with the same `nnpfc_id` value.

This SEI message pertains to the current decoded picture and all subsequent decoded pictures of the current layer, in output order, until the end of the current CLVS or up to but excluding the decoded picture that follows the current decoded picture in output order within the current CLVS and is associated with a subsequent NNPF SEI message, in decoding order, having `nnpfc_base_flag` equal to 0 and that particular `nnpfc_id` value within the current CLVS, whichever is earlier.

`nnpfc_mode_idc` equal to 0 indicates that this SEI message contains an ISO/IEC 15938-17 bitstream that specifies a base NNPF (when `nnpfc_base_flag` is equal

32

to 1) or is an update relative to the base NNPF with the same `nnpfc_id` value (when `nnpfc_base_flag` is equal to 0).

When `nnpfc_base_flag` is equal to 1, `nnpfc_mode_idc` equal to 1 specifies that the base NNPF associated with the `nnpfc_id` value is a neural network identified by the URI indicated by `nnpfc_uri` with the format identified by the tag URI `nnpfc_tag_uri`.

When `nnpfc_base_flag` is equal to 0, `nnpfc_mode_idc` equal to 1 specifies that an update relative to the base NNPF with the same `nnpfc_id` value is defined by the URI indicated by `nnpfc_uri` with the format identified by the tag URI `nnpfc_tag_uri`.

The value of `nnpfc_mode_idc` shall be in the range of 0 to 1, inclusive, in bitstreams conforming to this edition of this document. Values of 2 to 255, inclusive, for `nnpfc_mode_idc` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages with `nnpfc_mode_idc` in the range of 2 to 255, inclusive. Values of `nnpfc_mode_idc` greater than 255 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

`nnpfc_reserved_zero_bit_a` shall be equal to 0 in bitstreams conforming to this edition of this document. Decoders shall ignore NNPF SEI messages in which `nnpfc_reserved_zero_bit_a` is not equal to 0.

`nnpfc_tag_uri` contains a tag URI with syntax and semantics as specified in IETF RFC 4151 identifying the format and associated information about the neural network used as a base NNPF or an update relative to the base NNPF with the same `nnpfc_id` value specified by `nnpfc_uri`.

NOTE—`nnpfc_tag_uri` enables uniquely identifying the format of neural network data specified by `nnpfc_uri` without needing a central registration authority.

`nnpfc_tag_uri` equal to “tag:iso.org,2023:15938-17” indicates that the neural network data identified by `nnpfc_id` conforms to ISO/IEC 15938-17.

`nnpfc_uri` contains a URI with syntax and semantics as specified in IETF Internet Standard 66 identifying the neural network used as a base NNPF or an update relative to the base NNPF with the same `nnpfc_id` value.

`nnpfc_property_present_flag` equal to 1 specifies that syntax elements related to the filter purpose, input formatting, output formatting, and complexity are present. `nnpfc_property_present_flag` equal to 0 specifies that no syntax elements related to the filter purpose, input formatting, output formatting, and complexity are present.

When `nnpfc_base_flag` is equal to 1, `nnpfc_property_present_flag` shall be equal to 1.

When `nnpfc_property_present_flag` is equal to 0, the values of all syntax elements that may be present only when `nnpfc_property_present_flag` is equal to 1 are inferred to be equal to their corresponding syntax elements, respectively, in the NNPF SEI message that contains the base NNPF for which this SEI message provides an update.

When an NNPF SEI message `nnpfcCurr` is not the first NNPF SEI message, in decoding order, that has a particular `nnpfc_id` value within the current CLVS, is not a repetition of the first NNPF SEI message with that particular `nnpfc_id` (i.e., the value of `nnpfc_base_flag` is equal to 0),

and the value of `nnpfc_property_present_flag` is equal to 1, the following constraints apply:

The value of `nnpfc_purpose` in the NNPFC SEI message shall be the same as the value of `nnpfc_purpose` in the first NNPFC SEI message, in decoding order, that has that particular `nnpfc_id` value within the current CLVS. The values of syntax elements following `nnpfc_property_present_flag` and preceding `nnpfc_complexity_info_present_flag`, in decoding order, in the NNPFC SEI message shall be the same as the values of corresponding syntax elements in the first NNPFC SEI message, in decoding order, that has that particular `nnpfc_id` value within the current CLVS.

Either `nnpfc_complexity_info_present_flag` shall be equal to 0 or both `nnpfc_complexity_info_present_flag` shall be equal to 1 in the first NNPFC SEI message, in decoding order, that has that particular `nnpfc_id` value within the current CLVS (denoted as `nnpfcBase` below) and all the following apply:

`nnpfc_parameter_type_idc` in `nnpfcCurr` shall be equal to `nnpfc_parameter_type_idc` in `nnpfcBase`.

`nnpfc_log_2_parameter_bit_length_minus3` in `nnpfcCurr`, when present, shall be less than or equal to `nnpfc_log_2_parameter_bit_length_minus3` in `nnpfcBase`.

If `nnpfc_num_parameters_idc` in `nnpfcBase` is equal to 0, `nnpfc_num_parameters_idc` in `nnpfcCurr` shall be equal to 0.

Otherwise (`nnpfc_num_parameters_idc` in `nnpfcBase` is greater than 0), `nnpfc_num_parameters_idc` in `nnpfcCurr` shall be greater than 0 and less than or equal to `nnpfc_num_parameters_idc` in `nnpfcBase`.

If `nnpfc_num_kmac_operations_idc` in `nnpfcBase` is equal to 0, `nnpfc_num_kmac_operations_idc` in `nnpfcCurr` shall be equal to 0.

Otherwise (`nnpfc_num_kmac_operations_idc` in `nnpfcBase` is greater than 0), `nnpfc_num_kmac_operations_idc` in `nnpfcCurr` shall be greater than 0 and less than or equal to `nnpfc_num_kmac_operations_idc` in `nnpfcBase`.

If `nnpfc_total_kilobyte_size` in `nnpfcBase` is equal to 0, `nnpfc_total_kilobyte_size` in `nnpfcCurr` shall be equal to 0.

Otherwise (`nnpfc_total_kilobyte_size` in `nnpfcBase` is greater than 0), `nnpfc_total_kilobyte_size` in `nnpfcCurr` shall be greater than 0 and less than or equal to `nnpfc_total_kilobyte_size` in `nnpfcBase`.

`nnpfc_num_input_pics_minus1` plus 1 specifies the number of pictures used as input for the NNPFC. The value of `nnpfc_num_input_pics_minus1` shall be in the range of 0 to 63, inclusive. When `pictureRateUpsamplingFlag` is equal to 1, the value of `nnpfc_num_input_pics_minus1` shall be greater than 0.

The variable `numInputPics`, specifying the number of pictures used as input for the NNPFC, is derived as follows:

$$\text{numInputPics} = \text{nnpfc_num_input_pics_minus1} + 1$$

`nnpfc_input_pic_output_flag[i]` equal to 1 indicates that for the *i*-th input picture the NNPFC generates a corresponding output picture. `nnpfc_input_pic_output_flag[i]` equal to 0 indicates that for the *i*-th input picture the NNPFC does not generate a corresponding output picture. When `nnpfc_num_input_pics_minus1` is equal to 0, `nnpfc_input_pic_output_flag[0]` is inferred to be

equal to 1. When `pictureRateUpsamplingFlag` is equal to 0 and `nnpfc_num_input_pics_minus1` is greater than 0, `nnpfc_input_pic_output_flag[i]` shall be equal to 1 for at least one value of *i* in the range of 0 to `nnpfc_num_input_pics_minus1`, inclusive.

`nnpfc_absent_input_pic_zero_flag` equal to 1 indicates that the NNPFC expects an input picture that is not present in the bitstream to be represented by sample arrays with sample values equal to 0. `nnpfc_absent_input_pic_flag` equal to 0 indicates that the NNPFC expects an input picture that is not present in the bitstream to be represented by the closest input picture in output order within the bitstream.

`nnpfc_out_sub_c_flag` specifies the values of the variables `outSubWidthC` and `outSubHeightC` when `chromaUpsamplingFlag` is equal to 1. `nnpfc_out_sub_c_flag` equal to 1 specifies that `outSubWidthC` is equal to 1 and `outSubHeightC` is equal to 1. `nnpfc_out_sub_c_flag` equal to 0 specifies that `outSubWidthC` is equal to 2 and `outSubHeightC` is equal to 1. When `ChromaFormatIdc` is equal to 2 and `nnpfc_out_sub_c_flag` is present, the value of `nnpfc_out_sub_c_flag` shall be equal to 1.

`nnpfc_out_colour_format_idc`, when `colourizationFlag` is equal to 1, specifies the colour format of the NNPFC output and consequently the values of the variables `outSubWidthC` and `outSubHeightC`. `nnpfc_out_colour_format_idc` equal to 1 specifies that the colour format of the NNPFC output is the 4:2:0 format and `outSubWidthC` and `outSubHeightC` are both equal to 2. `nnpfc_out_colour_format_idc` equal to 2 specifies that the colour format of the NNPFC output is the 4:2:2 format and `outSubWidthC` is equal to 2 and `outSubHeightC` is equal to 1. `nnpfc_out_colour_format_idc` equal to 3 specifies that the colour format of the NNPFC output is the 4:4:4 format and `outSubWidthC` and `outSubHeightC` are both equal to 1. The value of `nnpfc_out_colour_format_idc` shall not be equal to 0.

When `chromaUpsamplingFlag` and `colourizationFlag` are both equal to 0, `outSubWidthC` and `outSubHeightC` are inferred to be equal to `SubWidthC` and `SubHeightC`, respectively.

`nnpfc_pic_width_num_minus1` plus 1 and `nnpfc_pic_width_denom_minus1` plus 1 specify the numerator and denominator, respectively, for the resampling ratio of the NNPFC output picture width relative to `CroppedWidth`. The value of $(\text{nnpfc_pic_width_num_minus1} + 1) \div (\text{nnpfc_pic_width_denom_minus1} + 1)$ shall be in the range of 1 to 16, inclusive. When `nnpfc_pic_width_num_minus1` and `nnpfc_pic_width_denom_minus1` are not present, the values of `nnpfc_pic_width_num_minus1` and `nnpfc_pic_width_denom_minus1` are both inferred to be equal to 0. The variable `nnpfcOutputPicWidth`, representing the width of the luma sample arrays of the picture(s) resulting from applying the NNPFC identified by `nnpfc_id` to the input picture(s), is derived as follows:

$$\text{nnpfcOutputPicWidth} =$$

$$\text{Ceil}(\text{CroppedWidth} * (\text{nnpfc_pic_width_num_minus1} + 1) \div$$

$$(\text{nnpfc_pic_width_denom_minus1} + 1))$$

35

It is a requirement of bitstream conformance that the value of `nnpfcOutputPicWidth % outSubWidthC` shall be equal to 0.

`nnpfc_pic_height_num_minus1` plus 1 and `nnpfc_pic_height_denom_minus1` plus 1 specify the numerator and denominator, respectively, for the resampling ratio of the NNPF output picture height relative to Cropped-Height. The value of $(\text{nnpfc_pic_height_num_minus1} + 1) \div (\text{nnpfc_pic_height_denom_minus1} + 1)$ shall be in the range of $1 \div 16$ to 16, inclusive. When `nnpfc_pic_height_num_minus1` and `nnpfc_pic_height_denom_minus1` are not present, the values of `nnpfc_pic_height_num_minus1` and `nnpfc_pic_height_denom_minus1` are both inferred to be equal to 0. The variable `nnpfcOutputPicHeight`, representing the height of the luma sample arrays of the picture(s) resulting from applying the NNPF identified by `nnpfc_id` to the input picture(s), is derived as follows:

`nnpfcOutputPicHeight =`

$$\text{Ceil}(\text{CroppedHeight} * (\text{nnpfc_pic_height_num_minus1} + 1) \div (\text{nnpfc_pic_height_denom_minus1} + 1))$$

It is a requirement of bitstream conformance that the value of `nnpfcOutputPicHeight % outSubHeightC` shall be equal to 0.

When `nnpfc_pic_width_num_minus1`, `nnpfc_pic_width_denom_minus1`, `nnpfc_pic_height_num_minus1`, and `nnpfc_pic_height_denom_minus1` are present, at least one the following shall be true:

The value of `nnpfcOutputPicWidth` is not equal to `CroppedWidth`.

The value of `nnpfcOutputPicHeight` is not equal to `CroppedHeight`.

`nnpfc_interpolated_pics[i]` specifies the number of interpolated pictures generated by the NNPF between the *i*-th and the (*i*+1)-th picture used as input for the NNPF. The value of `nnpfc_interpolated_pics[i]` shall be in the range of 0 to 63, inclusive. The value of `nnpfc_interpolated_pics[i]` shall be greater than 0 for at least one value of *i* in the range of 0 to `nnpfc_num_input_pics_minus1`, inclusive.

The variables `NumInpPicsInOutputTensor`, specifying the number of pictures that have a corresponding input picture and are present in the output tensor of the NNPF, `InpIdx[idx]` specifying the input picture index of the *idx*-th picture that is present in the output tensor of the NNPF and has a corresponding input picture, and `numOutputPics`, specifying the total number of pictures present in the output tensor of the NNPF, are derived as follows:

```

for( i = 0, numOutputPics = 0; i < numInputPics; i++ )
  if( nnpfc_input_pic_output_flag[ i ] ) {
    InpIdx[ numOutputPics ] = i
    numOutputPics++
  }
NumInpPicsInOutputTensor = numOutputPics
if( pictureRateUpsamplingFlag )
  for( i = 0; i <= numInputPics - 2; i++ )
    numOutputPics += nnpfc_interpolated_pics[ i ]

```

`nnpfc_component_last_flag` equal to 1 indicates that the last dimension in the input tensor `inputTensor` to the NNPF and the output tensor `outputTensor` resulting

36

from the NNPF is used for a current channel. `nnpfc_component_last_flag` equal to 0 indicates that the third dimension in the input tensor `inputTensor` to the NNPF and the output tensor `outputTensor` resulting from the NNPF is used for a current channel.

NOTE—The first dimension in the input tensor and in the output tensor is used for the batch index, which is a practice in some neural network frameworks. While formulae in the semantics of this SEI message use the batch size corresponding to the batch index equal to 0, it is up to the post-processing implementation to determine the batch size used as input to the neural network inference.

NOTE—For example, when `nnpfc_inp_order_idc` is equal to 3 and `nnpfc_auxiliary_inp_idc` is equal to 1, there are 7 channels in the input tensor, including four luma matrices, two chroma matrices, and one auxiliary input matrix. In this case, the process `DeriveInputTensors( )` would derive each of these 7 channels of the input tensor one by one, and when a particular channel of these channels is processed, that channel is referred to as the current channel during the process.

`nnpfc_inp_format_idc` indicates the method of converting a sample value of the input picture to an input value to the NNPF. When `nnpfc_inp_format_idc` is equal to 0, the input values to the NNPF are real numbers and the functions `InpY( )` and `InpC( )` are specified as follows:

$$\text{InpY}(x) = x \div ((1 \ll \text{BitDepth}_Y) - 1)$$

$$\text{InpC}(x) = x \div ((1 \ll \text{BitDepth}_C) - 1)$$

When `nnpfc_inp_format_idc` is equal to 1, the input values to the NNPF are unsigned integer numbers and the functions `InpY( )` and `InpC( )` are specified as follows:

```

shiftY = BitDepthY - inpTensorBitDepthY
if( inpTensorBitDepthY >= BitDepthY )
  InpY( x ) = x << ( inpTensorBitDepthY - BitDepthY )
else
  InpY( x ) = Clip3( 0, ( 1 << inpTensorBitDepthY ) - 1, ( x + ( 1 <<
    ( shiftY - 1 ) ) ) >> shiftY )
shiftC = BitDepthC - inpTensorBitDepthC
if( inpTensorBitDepthC >= BitDepthC )
  InpC( x ) = x << ( inpTensorBitDepthC - BitDepthC )
else
  InpC( x ) = Clip3( 0, ( 1 << inpTensorBitDepthC ) - 1, ( x + ( 1 <<
    ( shiftC - 1 ) ) ) >> shiftC )

```

The variable `inpTensorBitDepthY` is derived from the syntax element `nnpfc_inp_tensor_luma_bitdepth_minus8` as specified below. The variable `inpTensorBitDepthC` is derived from the syntax element `nnpfc_inp_tensor_chroma_bitdepth_minus8` as specified below.

Values of `nnpfc_inp_format_idc` greater than 1 are reserved for future specification by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages that contain reserved values of `nnpfc_inp_format_idc`.

`nnpfc_auxiliary_inp_idc` greater than 0 indicates that auxiliary input data is present in the input tensor of the NNPF. `nnpfc_auxiliary_inp_idc` equal to 0 indicates that auxiliary input data is not present in the input

37

tensor. `nnpfc_auxiliary_inp_idc` equal to 1 specifies that auxiliary input data is derived as specified.

The value of `nnpfc_auxiliary_inp_idc` shall be in the range of 0 to 1, inclusive, in bitstreams conforming to this edition of this document. Values of 2 to 255, inclusive, for `nnpfc_auxiliary_inp_idc` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SEI messages with `nnpfc_auxiliary_inp_idc` in the range of 2 to 255, inclusive. Values of `nnpfc_auxiliary_inp_idc` greater than 255 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

`nnpfc_inp_order_idc` indicates the method of ordering the sample arrays of an input picture to form an input tensor to the NNPFC.

The value of `nnpfc_inp_order_idc` shall be in the range of 0 to 3, inclusive, in bitstreams conforming to this edition of this document. Values of 4 to 255, inclusive, for `nnpfc_inp_order_idc` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SI messages with `nnpfc_inp_order_idc` in the range of 4 to 255, inclusive. Values of `nnpfc_inp_order_idc` greater than 255 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

When `ChromaFormatIdc` is not equal to 1, `nnpfc_inp_order_idc` shall not be equal to 3.

When `ChromaFormatIdc` is equal to 0, `nnpfc_inp_order_idc` shall be equal to 0.

When `chromaUpsamplingFlag` is equal to 1, `nnpfc_inp_order_idc` shall not be equal to 0.

Table 9 contains an informative description of `nnpfc_inp_order_idc` values.

TABLE 9

<code>nnpfc_inp_order_idc</code> Description	
0	If <code>nnpfc_auxiliary_inp_idc</code> is equal to 0, one luma matrix is present in the input tensor for each input picture, and the number of channels is 1. Otherwise, when <code>nnpfc_auxiliary_inp_idc</code> is equal to 1, one luma matrix and one auxiliary input matrix are present, and the number of channels is 2.
1	If <code>nnpfc_auxiliary_inp_idc</code> is equal to 0, two chroma matrices are present in the input tensor, and the number of channels is 2. Otherwise, when <code>nnpfc_auxiliary_inp_idc</code> is equal to 1, two chroma matrices and one auxiliary input matrix are present, and the number of channels is 3.
2	If <code>nnpfc_auxiliary_inp_idc</code> is equal to 0, one luma and two chroma matrices are present in the input tensor, and the number of channels is 3. Otherwise, when <code>nnpfc_auxiliary_inp_idc</code> is equal to 1, one luma matrix, two chroma matrices and one auxiliary input matrix are present, and the number of channels is 4.
3	If <code>nnpfc_auxiliary_inp_idc</code> is equal to 0, four luma matrices and two chroma matrices are present in the input tensor, and the number of channels is 6. Otherwise, when <code>nnpfc_auxiliary_inp_idc</code> is equal to 1, four luma matrices, two chroma matrices, and one auxiliary input matrix are present in the input tensor, and the number of channels is 7. The luma channels are derived in an interleaved manner as illustrated in FIG. 8. This <code>nnpfc_inp_order_idc</code> can only be used when the input chroma format is 4:2:0.
4 . . . 255	Reserved

38

`nnpfc_inp_tensor_luma_bitdepth_minus8` plus 8 specifies the bit depth of luma sample values in the input integer tensor. The value of `inpTensorBitDepthY` is derived as follows:

$$\text{inpTensorBitDepth}_Y = \text{nnpfc_inp_tensor_luma_bitdepth_minus8} + 8$$

It is a requirement of bitstream conformance that the value of `nnpfc_inp_tensor_luma_bitdepth_minus8` shall be in the range of 0 to 24, inclusive.

`nnpfc_inp_tensor_chroma_bitdepth_minus8` plus 8 specifies the bit depth of chroma sample values in the input integer tensor. The value of `inpTensorBitDepthC` is derived as follows:

$$\text{inpTensorBitDepth}_C = \text{nnpfc_inp_tensor_chroma_bitdepth_minus8} + 8$$

It is a requirement of bitstream conformance that the value of `nnpfc_inp_tensor_chroma_bitdepth_minus8` shall be in the range of 0 to 24, inclusive.

When `nnpfc_auxiliary_inp_idc` is equal to 1, the variable `strengthControlScaledVal` is derived as follows:

```

for( i = 0; i < numInputPics; i++ )
  if( nnpfc_inp_format_idc == 1 )
    if( nnpfc_inp_order_idc == 0 || nnpfc_inp_order_idc == 2 ||
        nnpfc_inp_order_idc == 3 )
      strengthControlScaledVal[ i ] =
        Floor ( StrengthControlVal[ i ] * ( ( 1 <<
          inpTensorBitDepthY ) - 1 ) )
    else if( nnpfc_inp_order_idc == 1 )
      strengthControlScaledVal[ i ] =
        Floor ( StrengthControlVal[ i ] * ( ( 1 <<
          inpTensorBitDepthC ) - 1 ) )

```

```

else
    strengthControlScaledVal[ i ] = StrengthControlVal[ i ]

```

A patch is a rectangular array of samples from a component (e.g., a luma or chroma component) of a picture.

The process DeriveInputTensors(), for deriving the input tensor inputTensor for a given vertical sample coordinate cTop and a horizontal sample coordinate cLeft specifying the top-left sample location for the patch of samples included in the input tensor, is specified as follows:

```

for( i = 0; i < numInputPics; i++ ) {
    if( nnpfc_inp_order_idc == 0 )
        for( yP = -nnpfc_overlap; yP < inpPatchHeight + nnpfc_overlap; yP++ )
            for( xP = -nnpfc_overlap; xP < inpPatchWidth + nnpfc_overlap; xP++ ) {
                inpVal = InpY( InpSampleVal( cTop + yP, cLeft + xP, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                yPovlp = yP + nnpfc_overlap
                xPovlp = xP + nnpfc_overlap
                if( !nnpfc_component_last_flag )
                    inputTensor[ 0 ][ i ][ 0 ][ yPovlp ][ xPovlp ] = inpVal
                else
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 0 ] = inpVal
            }
        if( nnpfc_auxiliary_inp_idc == 1 )
            if( !nnpfc_component_last_flag )
                inputTensor[ 0 ][ i ][ 1 ][ yPovlp ][ xPovlp ] = strengthControlScaledVal[ i ]
            else
                inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 1 ] = strengthControlScaledVal[ i ]
    }
    else if( nnpfc_inp_order_idc == 1 )
        for( yP = -nnpfc_overlap; yP < inpPatchHeight + nnpfc_overlap; yP++ )
            for( xP = -nnpfc_overlap; xP < inpPatchWidth + nnpfc_overlap; xP++ ) {
                inpCbVal = InpC( InpSampleVal( cTop + yP, cLeft + xP, CroppedHeight / SubHeightC,
                    CroppedWidth / SubWidthC, CroppedCbPic[ i ], 1 ) )
                inpCrVal = InpC( InpSampleVal( cTop + yP, cLeft + xP, CroppedHeight / SubHeightC,
                    CroppedWidth / SubWidthC, CroppedCrPic[ i ], 2 ) )
                yPovlp = yP + nnpfc_overlap
                xPovlp = xP + nnpfc_overlap
                if( !nnpfc_component_last_flag ) {
                    inputTensor[ 0 ][ i ][ 0 ][ yPovlp ][ xPovlp ] = inpCbVal
                    inputTensor[ 0 ][ i ][ 1 ][ yPovlp ][ xPovlp ] = inpCrVal
                } else {
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 0 ] = inpCbVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 1 ] = inpCrVal
                }
            }
        if( nnpfc_auxiliary_inp_idc == 1 )
            if( !nnpfc_component_last_flag )
                inputTensor[ 0 ][ i ][ 2 ][ yPovlp ][ xPovlp ] = strengthControlScaledVal[ i ]
            else
                inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 2 ] = strengthControlScaledVal[ i ]
    }
    else if( nnpfc_inp_order_idc == 2 )
        for( yP = -nnpfc_overlap; yP < inpPatchHeight + nnpfc_overlap; yP++ )
            for( xP = -nnpfc_overlap; xP < inpPatchWidth + nnpfc_overlap; xP++ ) {
                yY = cTop + yP
                xY = cLeft + xP
                yC = yY / SubHeightC
                xC = xY / SubWidthC
                inpYVal = InpY( InpSampleVal( yY, xY, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                inpCbVal = InpC( InpSampleVal( yC, xC, CroppedHeight / SubHeightC,
                    CroppedWidth / SubWidthC, CroppedCbPic[ i ], 1 ) )
                inpCrVal = InpC( InpSampleVal( yC, xC, CroppedHeight / SubHeightC,
                    CroppedWidth / SubWidthC, CroppedCrPic[ i ], 2 ) )
                yPovlp = yP + nnpfc_overlap
                xPovlp = xP + nnpfc_overlap
                if( !nnpfc_component_last_flag ) {
                    inputTensor[ 0 ][ i ][ 0 ][ yPovlp ][ xPovlp ] = inpYVal
                    inputTensor[ 0 ][ i ][ 1 ][ yPovlp ][ xPovlp ] = inpCbVal
                    inputTensor[ 0 ][ i ][ 2 ][ yPovlp ][ xPovlp ] = inpCrVal
                } else {
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 0 ] = inpYVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 1 ] = inpCbVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 2 ] = inpCrVal
                }
            }
        if( nnpfc_auxiliary_inp_idc == 1 )
            if( !nnpfc_component_last_flag )
                inputTensor[ 0 ][ i ][ 3 ][ yPovlp ][ xPovlp ] = strengthControlScaledVal[ i ]

```

```

        else
            inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 3 ] = strengthControlScaledVal[ i ]
        }
    }
    else if( nnpfc_inp_order_idc == 3 )
        for( yP = -nnpfc_overlap; yP < inpPatchHeight + nnpfc_overlap; yP++ )
            for( xP = -nnpfc_overlap; xP < inpPatchWidth + nnpfc_overlap; xP++ ) {
                yTL = cTop + yP * 2
                xTL = cLeft + xP * 2
                yBR = yTL + 1
                xBR = xTL + 1
                yC = cTop / 2 + yP
                xC = cLeft / 2 + xP
                inpTLVal = InpY( InpSampleVal( yTL, xTL, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                inpTRVal = InpY( InpSampleVal( yTL, xBR, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                inpBLVal = InpY( InpSampleVal( yBR, xTL, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                inpBRVal = InpY( InpSampleVal( yBR, xBR, CroppedHeight,
                    CroppedWidth, CroppedYPic[ i ], 0 ) )
                inpCbVal = InpC( InpSampleVal( yC, xC, CroppedHeight / 2,
                    CroppedWidth / 2, CroppedCbPic[ i ], 1 ) )
                inpCrVal = InpC( InpSampleVal( yC, xC, CroppedHeight / 2,
                    CroppedWidth / 2, CroppedCrPic[ i ], 2 ) )
                yPovlp = yP + nnpfc_overlap
                xPovlp = xP + nnpfc_overlap
                if( !nnpfc_component_last_flag ) {
                    inputTensor[ 0 ][ i ][ 0 ][ yPovlp ][ xPovlp ] = inpTLVal
                    inputTensor[ 0 ][ i ][ 1 ][ yPovlp ][ xPovlp ] = inpTRVal
                    inputTensor[ 0 ][ i ][ 2 ][ yPovlp ][ xPovlp ] = inpBLVal
                    inputTensor[ 0 ][ i ][ 3 ][ yPovlp ][ xPovlp ] = inpBRVal
                    inputTensor[ 0 ][ i ][ 4 ][ yPovlp ][ xPovlp ] = inpCbVal
                    inputTensor[ 0 ][ i ][ 5 ][ yPovlp ][ xPovlp ] = inpCrVal
                } else {
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 0 ] = inpTLVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 1 ] = inpTRVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 2 ] = inpBLVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 3 ] = inpBRVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 4 ] = inpCbVal
                    inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 5 ] = inpCrVal
                }
            }
        if( nnpfc_auxiliary_inp_idc == 1 )
            if( !nnpfc_component_last_flag )
                inputTensor[ 0 ][ i ][ 6 ][ yPovlp ][ xPovlp ] = strengthControlScaledVal[ i ]
            else
                inputTensor[ 0 ][ i ][ yPovlp ][ xPovlp ][ 6 ] = strengthControlScaledVal[ i ]
    }
}

```

nnpfc_out_format_idc equal to 0 indicates that the sample values output by the NNPf are real numbers where the value range of 0 to 1, inclusive, maps linearly to the unsigned integer value range of 0 to $(1 \ll \text{bitDepth}) - 1$, inclusive, for any desired bit depth bitDepth for subsequent post-processing or displaying.

nnpfc_out_format_idc equal to 1 indicates that the luma sample values output by the NNPf are unsigned integer numbers in the range of 0 to $(1 \ll \text{outTensorBitDepth}_Y) - 1$, inclusive, and the chroma sample values output by the NNPf are unsigned integer numbers in the range of 0 to $(1 \ll \text{outTensorBitDepth}_C) - 1$, inclusive.

Values of nnpfc_out_format_idc greater than 1 are reserved for future specification by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPfC SEI messages that contain reserved values of nnpfc_out_format_idc.

nnpfc_out_order_idc indicates the output order of samples resulting from the NNPf.

The value of nnpfc_out_order_idc shall be in the range of 0 to 3, inclusive, in bitstreams conforming to this edition of this document. Values of 4 to 255, inclusive, for nnpfc_out_order_idc are reserved for future use by ITU-T/ISO/IEC

and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPfC SEI messages with nnpfc_out_order_idc in the range of 4 to 255, inclusive. Values of nnpfc_out_order_idc greater than 255 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

When chromaUpsamplingFlag is equal to 1, nnpfc_out_order_idc shall not be equal to 0 or 3.

When colourizationFlag is equal to 1, nnpfc_out_order_idc shall not be equal to 0.

Table 10 contains an informative description of nnpfc_out_order_idc values.

TABLE 10

nnpfc_out_order_idc	Description
0	Only the luma matrix is present in the output tensor, thus the number of channels is 1.
1	Only the chroma matrices are present in the output tensor, thus the number of channels is 2.
2	The luma and chroma matrices are present in the output tensor, thus the number of channels is 3.

TABLE 10-continued

nnpfc_out_order_idc	Description
3	Four luma matrices and two chroma matrices are present in the output tensor, thus the number of channels is 6. This nnpfc_out_order_idc can only be used when the output chroma format is 4:2:0.
4 . . . 255	Reserved

nnpfc_out_tensor_luma_bitdepth_minus8 plus 8 specifies the bit depth of luma sample values in the output integer tensor. The value of nnpfc_out_tensor_luma_bitdepth_minus8 shall be in the range of 0 to 24, inclusive. The value of outTensorBitDepth_Y is derived as follows:

$$\text{outTensorBitDepth}_Y = \text{nnpfc_out_tensor_luma_bitdepth_minus8} + 8$$

nnpfc_out_tensor_chroma_bitdepth_minus8 plus 8 specifies the bit depth of chroma sample values in the output integer tensor. The value of nnpfc_out_tensor_chroma_bitdepth_minus8 shall be in the range of 0 to 24, inclusive. The value of outTensorBitDepth_C is derived as follows:

$$\text{outTensorBitDepth}_C = \text{nnpfc_out_tensor_chroma_bitdepth_minus8} + 8$$

When bitDepthUpsamplingFlag is equal to 1, the value of nnpfc_out_format_idc shall be equal to 1 and at least one of the following conditions shall be true:

nnpfc_out_tensor_luma_bitdepth_minus8 is present and outTensorBitDepth_Y is greater than BitDepth_Y.

nnpfc_out_tensor_chroma_bitdepth_minus8 is present and outTensorBitDepth_C is greater than BitDepth_C.

When nnpfc_inp_tensor_luma_bitdepth_minus8, nnpfc_inp_tensor_chroma_bitdepth_minus8, nnpfc_out_tensor_luma_bitdepth_minus8, and nnpfc_out_tensor_chroma_bitdepth_minus8 are present and outTensorBitDepth_Y is greater than inpTensorBitDepth_Y, outTensorBitDepth_C shall not be less than inpTensorBitDepth_C. When nnpfc_inp_tensor_luma_bitdepth_minus8, nnpfc_inp_tensor_chroma_bitdepth_minus8, nnpfc_out_tensor_luma_bitdepth_minus8, and nnpfc_out_tensor_chroma_bitdepth_minus8 are present and outTensorBitDepth_C is greater than inpTensorBitDepth_C, outTensorBitDepth_Y shall not be less than inpTensorBitDepth_Y.

The process StoreOutputTensors(), for deriving sample values in the filtered output sample arrays FilteredYPic, FilteredCbPic, and FilteredCrPic from the output tensor outputTensor for a given vertical sample coordinate cTop and a horizontal sample coordinate cLeft specifying the top-left sample location for the patch of samples included in the input tensor, is specified as follows:

```

for( i = 0; i < numOutputPics; i++ ) {
  if( nnpfc_out_order_idc == 0 )
    for( yP = 0; yP < outPatchHeight; yP++ )
      for( xP = 0; xP < outPatchWidth; xP++ ) {
        yY = cTop * outPatchHeight / inpPatchHeight + yP
        xY = cLeft * outPatchWidth / inpPatchWidth + xP
        if ( yY < nnpfcOutputPicHeight && xY < nnpfcOutputPicWidth )
          if( !nnpfc_component_last_flag )
            FilteredYPic[ i ][ xY ][ yY ] = outputTensor[ 0 ][ i ][ 0 ][ yP ][ xP ]
          else
            FilteredYPic[ i ][ xY ][ yY ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 0 ]
        }
      }
  else if( nnpfc_out_order_idc == 1 )
    for( yP = 0; yP < outPatchCHeight; yP++ )
      for( xP = 0; xP < outPatchCWidth; xP++ ) {
        xSrc = cLeft * horCScaling + xP
        ySrc = cTop * verCScaling + yP
        if ( ySrc < nnpfcOutputPicHeight / outSubHeightC &&
            xSrc < nnpfcOutputPicWidth / outSubWidthC )
          if( !nnpfc_component_last_flag ) {
            FilteredCbPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ 0 ][ yP ][ xP ]
            FilteredCrPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ 1 ][ yP ][ xP ]
          } else {
            FilteredCbPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 0 ]
            FilteredCrPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 1 ]
          }
        }
      }
  else if( nnpfc_out_order_idc == 2 )
    for( yP = 0; yP < outPatchHeight; yP++ )
      for( xP = 0; xP < outPatchWidth; xP++ ) {
        yY = cTop * outPatchHeight / inpPatchHeight + yP
        xY = cLeft * outPatchWidth / inpPatchWidth + xP
        yC = yY / outSubHeightC
        xC = xY / outSubWidthC
        yPc = ( yP / outSubHeightC ) * outSubHeightC
        xPc = ( xP / outSubWidthC ) * outSubWidthC
        if ( yY < nnpfcOutputPicHeight && xY < nnpfcOutputPicWidth )
          if( !nnpfc_component_last_flag ) {
            FilteredYPic[ i ][ xY ][ yY ] = outputTensor[ 0 ][ i ][ 0 ][ yP ][ xP ]
            FilteredCbPic[ i ][ xC ][ yC ] = outputTensor[ 0 ][ i ][ 1 ][ yPc ][ xPc ]
            FilteredCrPic[ i ][ xC ][ yC ] = outputTensor[ 0 ][ i ][ 2 ][ yPc ][ xPc ]
          } else {
            FilteredYPic[ i ][ xY ][ yY ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 0 ]

```

```

        FilteredCbPic[ i ][ xC ][ yC ] = outputTensor[ 0 ][ i ][ yPc ][ xPc ][ 1 ]
        FilteredCrPic[ i ][ xC ][ yC ] = outputTensor[ 0 ][ i ][ yPc ][ xPc ][ 2 ]
    }
}
else if( nnpfc_out_order_idc == 3 )
for( yP = 0; yP < outPatchHeight; yP++ )
    for( xP = 0; xP < outPatchWidth; xP++ ) {
        ySrc = cTop / 2 * outPatchHeight / inpPatchHeight + yP
        xSrc = cLeft / 2 * outPatchWidth / inpPatchWidth + xP
        if ( ySrc < nnpfcOutputPicHeight / 2 &&
            xSrc < nnpfcOutputPicWidth / 2 )
            if( !nnpfc_component_last_flag ) {
                FilteredYPic[ i ][ xSrc * 2 ][ ySrc * 2 ] = outputTensor[ 0 ][ i ][ 0 ][ yP ][ xP ]
                FilteredYPic[ i ][ xSrc * 2 + 1 ][ ySrc * 2 ] = outputTensor[ 0 ][ i ][ 1 ][ yP ][ xP ]
                FilteredYPic[ i ][ xSrc * 2 ][ ySrc * 2 + 1 ] = outputTensor[ 0 ][ i ][ 2 ][ yP ][ xP ]
                FilteredYPic[ i ][ xSrc * 2 + 1 ][ ySrc * 2 + 1 ] = outputTensor[ 0 ][ i ][ 3 ][ yP ][ xP ]
                FilteredCbPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ 4 ][ yP ][ xP ]
                FilteredCrPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ 5 ][ yP ][ xP ]
            } else {
                FilteredYPic[ i ][ xSrc * 2 ][ ySrc * 2 ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 0 ]
                FilteredYPic[ i ][ xSrc * 2 + 1 ][ ySrc * 2 ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 1 ]
                FilteredYPic[ i ][ xSrc * 2 ][ ySrc * 2 + 1 ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 2 ]
                FilteredYPic[ i ][ xSrc * 2 + 1 ][ ySrc * 2 + 1 ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 3 ]
                FilteredCbPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 4 ]
                FilteredCrPic[ i ][ xSrc ][ ySrc ] = outputTensor[ 0 ][ i ][ yP ][ xP ][ 5 ]
            }
    }
}

```

nnpfc_separate_colour_description_present_flag equal to 1 indicates that a distinct combination of colour primaries, transfer characteristics, matrix coefficients, and scaling and offset values applied in association with the matrix coefficients for the picture resulting from the NNPf is specified in the SEI message syntax structure. nnpfc_separate_colour_description_present_flag equal to 0 indicates that the combination of colour primaries, transfer characteristics, matrix coefficients, and scaling and offset values applied in association with the matrix coefficients for the picture resulting from the NNPf is the same as indicated in VUI parameters for the CLVS. nnpfc_colour_primaries has the same semantics as specified for the vui_colour_primaries syntax element, which are as follows: vui_colour_primaries indicates the chromaticity coordinates of the source colour primaries. Its semantics are as specified for the ColourPrimaries parameter in Rec. ITU-T H.273|ISO/IEC 23091-2. When the vui_colour_primaries syntax element is not present, the value of vui_colour_primaries is inferred to be equal to 2 (the chromaticity is unknown or unspecified or determined by other means not specified in this Specification). Values of vui_colour_primaries that are identified as reserved for future use in Rec. ITU-T H.273|ISO/IEC 23091-2 shall not be present in bitstreams conforming to this version of this Specification. Decoders shall interpret reserved values of vui_colour_primaries as equivalent to the value 2.

Except as follows:

nnpfc_colour_primaries specifies the colour primaries of the picture resulting from applying the NNPf specified in the SEI message, rather than the colour primaries used for the CLVS.

When nnpfc_colour_primaries is not present in the NNPFC SEI message, the value of nnpfc_colour_primaries is inferred to be equal to vui_colour_primaries.

nnpfc_transfer_characteristics has the same semantics as specified for the vui_transfer_characteristics syntax element, which are as follows: vui_transfer_characteristics indicates the transfer characteristics function of

the colour representation. Its semantics are as specified for the TransferCharacteristics parameter in Rec. ITU-T H.273|ISO/IEC 23091-2. When the vui_transfer_characteristics syntax element is not present, the value of vui_transfer_characteristics is inferred to be equal to 2 (the transfer characteristics are unknown or unspecified or determined by other means not specified in this Specification). Values of vui_transfer_characteristics that are identified as reserved for future use in Rec. ITU-T H.273|ISO/IEC 23091-2 shall not be present in bitstreams conforming to this version of this Specification. Decoders shall interpret reserved values of vui_transfer_characteristics as equivalent to the value 2.

Except as follows:

nnpfc_transfer_characteristics specifies the transfer characteristics of the picture resulting from applying the NNPf specified in the SEI message, rather than the transfer characteristics used for the CLVS.

When nnpfc_transfer_characteristics is not present in the NNPFC SEI message, the value of nnpfc_transfer_characteristics is inferred to be equal to vui_transfer_characteristics.

nnpfc_matrix_coeffs describes the equations used in deriving luma and chroma signals from the green, blue, and red, or Y, Z, and X primaries. Its semantics apply to the pictures resulting from applying the NNPf specified in this SEI message and are as specified for MatrixCoefficients in Rec. ITU-T H.273|ISO/IEC 23091-2 with BitDepth_Y and BitDepth_C being equal to outTensorBitDepth_Y and outTensorBitDepth_C, respectively.

When nnpfc_matrix_coeffs is not present in the NNPFC SEI message, the value of nnpfc_matrix_coeffs is inferred to be equal to vui_matrix_coeffs.

nnpfc_matrix_coeffs shall not be equal to 0 unless both of the following conditions are true:

nnpfc_out_tensor_chroma_bitdepth_minus8 is equal to nnpfc_out_tensor_luma_bitdepth_minus8.

nnpfc_out_order_idc is equal to 2, outSubHeightC is equal to 1, and outSubWidthC is equal to 1.
 nnpfc_matrix_coeffs shall not be equal to 8 unless one of the following conditions is true:
 nnpfc_out_tensor_chroma_bitdepth_minus8 is equal to 5
 nnpfc_out_tensor_luma_bitdepth_minus8.
 nnpfc_out_tensor_chroma_bitdepth_minus8 is equal to
 nnpfc_out_tensor_luma_bitdepth_minus8+1, nnpfc-
 c_out_order_idc is equal to 2, outSubHeightC is
 equal to 1, and outSubWidthC is equal to 1. 10
 nnpfc_full_range_flag indicates the scaling and offset
 values applied in association with the matrix coeffi-
 cients as specified by nnpfc_matrix_coeffs. Its seman-
 tics are as specified for the VideoFullRangeFlag param-
 eter in Rec. ITU-T H.273|ISO/IEC 23091-2. When not
 present, the value of nnpfc_full_range_flag is inferred
 to be equal to 0.
 nnpfc_chroma_loc_info_present_flag equal to 1 indicates
 the presence of the nnpfc_chroma_sample-
 loc_type_frame syntax element in the NNPF SEI
 message. nnpfc_chroma_loc_info_present_flag equal
 to 0 indicates the absence of the nnpfc_chroma-
 sample_loc_type_frame syntax element in the
 NNPF SEI message. When colourizationFlag is equal
 to 0 or nnpfc_out_colour_format_idc is not equal to 1,
 the value of nnpfc_chroma_loc_info_present_flag shall
 be equal to 0.
 nnpfc_chroma_sample_loc_type_frame, when not equal
 to 6 and nnpfc_out_colour_format_idc is equal to 1,
 specifies the location of chroma samples of the output
 pictures, nnpfc_chroma_sample_loc_type_frame equal
 to 6 and nnpfc_out_colour_format_idc equal to 1 indi-
 cates that the location of the chroma samples is
 unknown or unspecified or specified by other means not
 specified in this document. The value of nnpfc_chroma-
 sample_loc_type_frame shall be in the range of 0
 to 6, inclusive.
 nnpfc_overlap indicates the overlapping horizontal and
 vertical sample counts of adjacent input tensors of the
 NNPF. The value of nnpfc_overlap shall be in the range
 of 0 to 16 383, inclusive.
 nnpfc_constant_patch_size_flag equal to 1 indicates that
 the NNPF accepts exactly the patch size indicated by
 nnpfc_patch_width_minus1 and nnpfc_patch-
 height_minus1 as input. nnpfc_constant_patch_size_f-
 lag equal to 0 indicates that the NNPF accepts as input
 any patch size with width inpPatchWidth and height
 inpPatchHeight such that the width of an extended
 patch (i.e., a patch plus the overlapping area), which is
 equal to inpPatchWidth+2*nnpfc_overlap, is a positive
 integer multiple of nnpfc_extended_patch_width-
 cd_delta_minus1+1+2*nnpfc_overlap, and the height
 of the extended patch, which is equal to inpPatch-
 Height+2*nnpfc_overlap, is a positive integer multiple
 of nnpfc_extended_patch_height_cd_delta_minus1+
 1+2*nnpfc_overlap.
 nnpfc_patch_width_minus1 plus 1, when nnpfc-
 constant_patch_size_flag equal to 1, indicates the hori-
 zontal sample counts of the patch size required for the
 input to the NNPF. The value of nnpfc_patch-
 width_minus1 shall be in the range of 0 to Min(32 766,
 CroppedWidth-1), inclusive.
 nnpfc_patch_height_minus1 plus 1, when nnpfc-
 constant_patch_size_flag equal to 1, indicates the vertical
 sample counts of the patch size required for the input to

the NNPF. The value of nnpfc_patch_height_minus1
 shall be in the range of 0 to Min(32 766, Cropped-
 Height-1), inclusive.
 nnpfc_extended_patch_width_cd_delta_minus1 plus 1
 plus 2*nnpfc_overlap, when nnpfc_constant-
 patch_size_flag equal to 0, indicates a common divisor
 of all allowed values of the width of an extended patch
 required for the input to the NNPF. The value of
 nnpfc_extended_patch_width_cd_delta_minus1 shall
 be in the range of 0 to Min(32 766, CroppedWidth-1),
 inclusive.
 nnpfc_extended_patch_height_cd_delta_minus1 plus 1
 plus 2*nnpfc_overlap, when nnpfc_constant-
 patch_size_flag equal to 0, indicates a common divisor
 of all allowed values of the height of an extended patch
 required for the input to the NNPF. The value of
 nnpfc_extended_patch_height_cd_delta_minus1 shall
 be in the range of 0 to Min(32 766, CroppedHeight-1),
 inclusive.
 Let the variables inpPatchWidth and inpPatchHeight be
 the patch size width and the patch size height, respectively.
 If nnpfc_constant_patch_size_flag is equal to 0, the fol-
 lowing applies:
 The values of inpPatchWidth and inpPatchHeight are
 either provided by external means not specified in this
 document or set by the post-processor itself.
 The value of inpPatchWidth+2*nnpfc_overlap shall be a
 positive integer multiple of nnpfc_extended-
 patch_width_cd_delta_minus1+1+2*nnpfc_overlap
 and inpPatchWidth shall be less than or equal to
 CroppedWidth. The value of inpPatchHeight+2*nnpf-
 c_overlap shall be a positive integer multiple of
 nnpfc_extended_patch_height_cd_delta_minus1+1+
 2*nnpfc_overlap and inpPatchHeight shall be less than
 or equal to CroppedHeight.
 Otherwise (nnpfc_constant_patch_size_flag is equal to 1),
 the value of inpPatchWidth is set equal to nnpfc-
 patch_width_minus1+1 and the value of inpPatchHeight is
 set equal to nnpfc_patch_height_minus1+1.
 The variables outPatchWidth, outPatchHeight, horCScaling,
 verCScaling, outPatchCWidth, and outPatchCHeight
 are derived as follows:

$$\begin{aligned} outPatchWidth &= (nnpfcOutputPicWidth * inpPatchWidth) / CroppedWidth \\ outPatchHeight &= \\ & (nnpfcOutputPicHeight * inpPatchHeight) / CroppedHeight \\ horCScaling &= SubWidthC / outSubWidthC \\ verCScaling &= SubHeightC / outSubHeightC \\ outPatchCWidth &= outPatchWidth * horCScaling \\ outPatchCHeight &= outPatchHeight * verCScaling \end{aligned}$$

It is a requirement of bitstream conformance that
 outPatchWidth*CroppedWidth shall be equal to
 nnpfcOutputPicWidth*inpPatchWidth and
 outPatchHeight*CroppedHeight shall be equal to
 nnpfcOutputPicHeight*inpPatchHeight.

nnpfc_padding_type indicates the process of padding
 when referencing sample locations outside the bound-
 aries of the input picture as described in Table 11. The
 value of nnpfc_padding_type shall be in the range of 0
 to 4, inclusive, in bitstreams conforming to this edition
 of this document. Values of 5 to 15, inclusive, for

nnpfc_padding_type are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SEI messages with nnpfc_padding_type in the range of 5 to 15, inclusive. Values of nnpfc_padding_type greater than 15 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

TABLE 11

nnpfc_padding_type	Description
0	Zero padding
1	Replication padding
2	Reflection padding
3	Wrap-around padding
4	Fixed padding
5 . . . 15	reserved

nnpfc_luma_padding_val indicates the luma value to be used for padding when nnpfc_padding_type is equal to 4. The value of nnpfc_luma_padding_val shall be in the range of 0 to $(1 \ll \text{BitDepth}_Y) - 1$, inclusive.

nnpfc_cb_padding_val indicates the Cb value to be used for padding when nnpfc_padding_type is equal to 4. The value of nnpfc_cb_padding_val shall be in the range of 0 to $(1 \ll \text{BitDepth}_C) - 1$, inclusive.

nnpfc_cr_padding_val indicates the Cr value to be used for padding when nnpfc_padding_type is equal to 4. The value of nnpfc_cr_padding_val shall be in the range of 0 to $(1 \ll \text{BitDepth}_C) - 1$, inclusive.

The function InpSampleVal(y, x, picHeight, picWidth, croppedPic, cldx) with inputs being a vertical sample location y, a horizontal sample location x, a picture height picHeight, a picture width picWidth, sample array croppedPic, and component index cldx (equal to 0 for luma, 1 for Cb, and 2 for Cr) returns the value of sampleVal derived as follows:

NOTE—For the inputs to the function InpSampleVal(), the vertical location is listed before the horizontal location for compatibility with input tensor conventions of some inference engines.

```

if( nnpfc_padding_type == 0 )
    if( y < 0 || x < 0 || y >= picHeight || x >= picWidth )
        sampleVal = 0
    else
        sampleVal = croppedPic[ x ][ y ]
else if( nnpfc_padding_type == 1 )
    sampleVal = croppedPic[ Clip3( 0, picWidth - 1, x ) ][ Clip3( 0,
        picHeight - 1, y ) ]
else if( nnpfc_padding_type == 2 )
    sampleVal = croppedPic[ Reflect( picWidth - 1, x ) ][ Reflect( picHeight -
        1, y ) ]
else if( nnpfc_padding_type == 3 )
    if( y >= 0 && y < picHeight )
        sampleVal = croppedPic[ Wrap( picWidth - 1, x ) ][ y ]
else if( nnpfc_padding_type == 4 )
    if( y < 0 || x < 0 || y >= picHeight || x >= picWidth )
        sampleVal = ( cldx == 0 ? nnpfc_luma_padding_val :
            ( cldx == 1 ? nnpfc_cb_padding_val :
                nnpfc_cr_padding_val ) )
    else
        sampleVal = croppedPic[ x ][ y ]

```

An NNPF PostProcessingFilter() is the target NNPF as derived in the semantics of the NNPF SEI message. The following example process may be used, with the NNPF

PostProcessingFilter(), to generate, in a patch-wise manner, the filtered and/or interpolated picture(s), which contain Y, Cb, and Cr sample arrays FilteredYPic, FilteredCbPic, and FilteredCrPic, respectively, as indicated by nnpfc_out_order_idc:

```

if( nnpfc_inp_order_idc == 0 || nnpfc_inp_order_idc == 2 )
    for( cTop = 0; cTop < CroppedHeight; cTop += inpPatchHeight )
        for( cLeft = 0; cLeft < CroppedWidth; cLeft += inpPatchWidth ) {
            DeriveInputTensors( )
            outputTensor = PostProcessingFilter( inputTensor )
            StoreOutputTensors( )
        }
else if( nnpfc_inp_order_idc == 1 )
    for( cTop = 0; cTop < CroppedHeight / SubHeightC; cTop +=
        inpPatchHeight )
        for( cLeft = 0; cLeft < CroppedWidth / SubWidthC; cLeft +=
            inpPatchWidth ) {
            DeriveInputTensors( )
            outputTensor = PostProcessingFilter( inputTensor )
            StoreOutputTensors( )
        }
else if( nnpfc_inp_order_idc == 3 )
    for( cTop = 0; cTop < CroppedHeight; cTop += inpPatchHeight * 2 )
        for( cLeft = 0; cLeft < CroppedWidth; cLeft += inpPatchWidth *
            2 ) {
            DeriveInputTensors( )
            outputTensor = PostProcessingFilter( inputTensor )
            StoreOutputTensors( )
        }

```

An NNPF-generated picture with index i contains sample arrays FilteredYPic[i], FilteredCbPic[i], and FilteredCrPic[i], when present, that are derived as specified. An NNPF-generated picture does not include the overlap regions.

The NNPF process consists of the process defined as specified followed by outputting NNPF-generated pictures in their increasing index order, where all NNPF-generated pictures that were interpolated by the NNPF are output and those NNPF-generated pictures that correspond to any input pictures to the NNPF are output as specified in the semantics of the NNPF SEI message.

nnpfc_complexity_info_present_flag equal to 1 specifies that one or more syntax elements that indicate the complexity of the NNPF associated with the nnpfc_id are present. nnpfc_complexity_info_present_flag equal to 0 specifies that no syntax elements that indicate the complexity of the NNPF associated with the nnpfc_id are present.

nnpfc_parameter_type_idc equal to 0 indicates that the neural network uses only integer parameters. nnpfc_parameter_type_flag equal to 1 indicates that the neural network may use floating point or integer parameters. nnpfc_parameter_type_idc equal to 2 indicates that the neural network uses only binary parameters. nnpfc_parameter_type_idc equal to 3 is reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SEI messages with nnpfc_parameter_type_idc equal to 3.

nnpfc_log_2_parameter_bit_length_minus3 equal to 0, 1, 2, and 3 indicates that the neural network does not use parameters of bit length greater than 8, 16, 32, and 64, respectively. When nnpfc_parameter_type_idc is present and nnpfc_log_2_parameter_bit_length_minus3 is not present, the neural network does not use parameters of bit length greater than 1.

nnpfc_num_parameters_idc indicates the maximum number of neural network parameters for the NNPF in units

51

of a power of 2 048. `nnpfc_num_parameters_idc` equal to 0 indicates that the maximum number of neural network parameters is unknown. The value `nnpfc_num_parameters_idc` shall be in the range of 0 to 52, inclusive. Values of `nnpfc_num_parameters_idc` greater than 52 are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPFC SEI messages with `nnpfc_num_parameters_idc` greater than 52.

If the value of `nnpfc_num_parameters_idc` is greater than zero, the variable `maxNumParameters` is derived as follows:

$$\text{maxNumParameters} = (2\,048 \ll \text{nnpfc_num_parameters_idc}) - 1$$

It is a requirement of bitstream conformance that the number of neural network parameters of the NNPFC shall be less than or equal to `maxNumParameters`.

`nnpfc_num_kmac_operations_idc` greater than 0 indicates that the maximum number of multiply-accumulate operations per sample of the NNPFC is less than or equal to `nnpfc_num_kmac_operations_idc * 1 000`. `nnpfc_num_kmac_operations_idc` equal to 0 indicates that the maximum number of multiply-accumulate operations of the network is unknown. The value of `nnpfc_num_kmac_operations_idc` shall be in the range of 0 to $2^{32}-2$, inclusive.

`nnpfc_total_kilobyte_size` greater than 0 indicates a total size in kilobytes required to store the uncompressed parameters for the neural network. The total size in bits is a number equal to or greater than the sum of bits used to store each parameter. `nnpfc_total_kilobyte_size` is the total size in bits divided by 8 000, rounded up. `nnpfc_total_kilobyte_size` equal to 0 indicates that the total size required to store the parameters for the neural network is unknown. The value of `nnpfc_total_kilobyte_size` shall be in the range of 0 to $2^{32}-2$, inclusive.

`nnpfc_metadata_extension_num_bits` equal to 0 specifies that `nnpfc_reserved_metadata_extension` is not present. `nnpfc_metadata_extension_num_bits` greater than 0 specifies the length, in bits, of `nnpfc_reserved_metadata_extension`. `nnpfc_metadata_extension_num_bits` shall be equal to 0 in this edition of this document. Values in the range of 1 to 2 048, inclusive, for `nnpfc_metadata_extension_num_bits` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall allow any value of `nnpfc_metadata_extension_num_bits` in the range of 0 to 2 048, inclusive. Values of `nnpfc_metadata_extension_num_bits` greater than 2 048 shall not be present in bitstreams conforming to this edition of this document and are not reserved for future use.

`nnpfc_reserved_metadata_extension` shall not be present in bitstreams conforming to this edition of this document. However, decoders conforming to this edition of this document shall ignore the presence and value of `nnpfc_reserved_metadata_extension`. When present, the length, in bits, of `nnpfc_reserved_metadata_extension` is equal to `nnpfc_metadata_extension_num_bits`. `nnpfc_reserved_zero_bit_b` shall be equal to 0 in bitstreams conforming to this edition of this document.

52

Decoders shall ignore NNPFC SEI messages in which `nnpfc_reserved_zero_bit_b` is not equal to 0.

`nnpfc_payload_byte[i]` contains the *i*-th byte of a bitstream conforming to ISO/IEC 15938-17. The byte sequence `nnpfc_payload_byte[i]` for all present values of *i* shall be a complete bitstream that conforms to ISO/IEC 15938-17.

Table 12 illustrates the syntax of the Neural-network post-filter activation SEI message provided in JVET-AD2006.

TABLE 12

nn_post_filter_activation(payloadSize) {	Descriptor
<code>nnpfa_target_id</code>	ue(v)
<code>nnpfa_cancel_flag</code>	u(1)
if(! <code>nnpfa_cancel_flag</code>) {	
<code>nnpfa_target_base_flag</code>	u(1)
<code>nnpfa_persistence_flag</code>	u(1)
<code>nnpfa_num_output_entries</code>	ue(v)
for(<i>i</i> = 0; <i>i</i> < <code>nnpfa_num_output_entries</code> ; <i>i</i> ++)	
<code>nnpfa_output_flag[i]</code>	u(1)
}	
}	

With respect to Table 12, JVET-AD2006 provides the following semantics:

The neural-network post-filter activation (NNPFA) SEI message activates or de-activates the possible use of the target neural-network post-processing filter (NNPF), identified by `nnpfa_target_id` and `nnpfa_target_base_flag`, for post-processing filtering of a set of pictures. For a particular picture for which the NNPFC is activated, the target NNPFC is derived as follows:

If `nnpfa_target_base_flag` is equal to 1, the target NNPFC is the base NNPFC with `nnpfc_id` equal to `nnpfa_target_id`.

Otherwise (`nnpfa_target_base_flag` is equal to 0), the target NNPFC is the NNPFC specified by the last NNPFC SEI message with `nnpfc_id` equal to `nnpfa_target_id` that precedes the first VCL NAL unit of the current picture in decoding order and is not a repetition of the NNPFC SEI message that contains the base NNPFC.

NOTE—There can be several NNPFA SEI messages present for the same picture, for example, when the NNPFCs are meant for different purposes or for filtering of different colour components.

`nnpfa_target_id` indicates the target NNPFC, which is specified by one or more NNPFC SEI messages that pertain to the current picture and have `nnpfc_id` equal to `nnpfa_target_id`. The value of `nnpfa_target_id` shall be in the range of 0 to $2^{32}-2$, inclusive.

An NNPFA SEI message with a particular value of `nnpfa_target_id` shall not be present in a current PU unless one or both of the following conditions are true:

Within the current CLVS there is an NNPFC SEI message with `nnpfc_id` equal to the particular value of `nnpfa_target_id` present in a PU preceding the current PU in decoding order.

There is an NNPFC SEI message with `nnpfc_id` equal to the particular value of `nnpfa_target_id` in the current PU.

When a PU contains both an NNPFC SEI message with a particular value of `nnpfc_id` and an NNPFA SEI message with `nnpfa_target_id` equal to the particular value of `nnpfc_id`, the NNPFC SEI message shall precede the NNPFA SEI message in decoding order.

nnpfa_cancel_flag equal to 1 indicates that the persistence of the target NNPF established by any previous NNPA SEI message with the same nnpfa_target_id as the current SEI message is cancelled, i.e., the target NNPF is no longer used unless it is activated by another NNPA SEI message with the same nnpfa_target_id as the current SEI message and nnpfa_cancel_flag equal to 0. nnpfa_cancel_flag equal to 0 indicates that the nnpfa_target_base_flag, nnpfa_persistence_flag, and nnpfa_num_output_entries follow.

nnpfa_target_base_flag equal to 1 specifies that the target NNPF is the base NNPF with nnpfc_id equal to nnpfa_target_id. nnpfa_target_base_flag equal to 0 specifies that the target NNPF is the NNPF specified by the last NNPF SEI message with nnpfc_id equal to nnpfa_target_id that precedes the first VCL NAL unit of the current picture in decoding order and is not a repetition of the NNPF SEI message that contains the base NNPF.

nnpfa_persistence_flag specifies the persistence of the target NNPF for the current layer.

nnpfa_persistence_flag equal to 0 specifies that the target NNPF may be used for post-processing filtering for the current picture only.

nnpfa_persistence_flag equal to 1 specifies that the target NNPF may be used for post-processing filtering for the current picture and all subsequent pictures of the current layer in output order until one or more of the following conditions are true:

A new CLVS of the current layer begins.

The bitstream ends.

A picture in the current layer associated with a NNPA SEI message with the same nnpfa_target_id as the current SEI message and nnpfa_cancel_flag equal to 1 is output that follows the current picture in output order.

NOTE—The target NNPF is not applied for this subsequent picture in the current layer associated with a NNPA SEI message with the same nnpfa_target_id as the current SEI message and nnpfa_cancel_flag equal to 1.

Let the nnpfcTargetPictures be the set of pictures to which the last NNPF SEI message with nnpfc_id equal to nnpfa_target_id that precedes the current NNPA SEI message in decoding order pertains. Let nnpfaTargetPictures be the set of pictures for which the target NNPF is activated by the current NNPA SEI message. It is a requirement of bitstream conformance that any picture included in nnpfaTargetPictures shall also be included in nnpfcTargetPictures.

nnpfa_num_output_entries specifies the number of nnpfa_output_flag[i] syntax elements present in the NNPA SEI message. The value of nnpfa_num_output_entries shall be in the range of 0 to NumInpPicsInOutputTensor, inclusive.

nnpfa_output_flag[i] equal to 1 specifies that the NNPF-generated picture that corresponds to the input picture having index InpIdx[i] is output by the NNPF process activated by this NNPA SEI message, where the NNPF process is specified in the semantics of the NNPF SEI message. nnpfa_output_flag[i] equal to 0 specifies that the NNPF-generated picture that corresponds to the input picture having index InpIdx[i] is not output by the NNPF process activated by this NNPA SEI message. When nnpfa_num_output_entries is less than NumInpPicsInOutputTensor, nnpfa_output_flag[i]

is inferred to be equal to 1 for each value of i in the range of nnpfa_num_output_entries to NumInpPicsInOutputTensor-1, inclusive.

The Neural-network post-filter characteristics SEI message provided in JVET-AD2006 may be less than ideal. In particular, for example, in JVET-AD2006, when syntax element nnpfc_purpose is equal to 0, the NNPF may be used as determined by the application and no information about the application specific purpose is provided. According to the techniques herein, a mechanism is provided to provide further information about an application-specific purpose.

FIG. 1 is a block diagram illustrating an example of a system that may be configured to code (i.e., encode and/or decode) video data according to one or more techniques of this disclosure. System 100 represents an example of a system that may encapsulate video data according to one or more techniques of this disclosure. As illustrated in FIG. 1, system 100 includes source device 102, communications medium 110, and destination device 120. In the example illustrated in FIG. 1, source device 102 may include any device configured to encode video data and transmit encoded video data to communications medium 110. Destination device 120 may include any device configured to receive encoded video data via communications medium 110 and to decode encoded video data. Source device 102 and/or destination device 120 may include computing devices equipped for wired and/or wireless communications and may include, for example, set top boxes, digital video recorders, televisions, desktop, laptop or tablet computers, gaming consoles, medical imaging devices, and mobile devices, including, for example, smartphones, cellular telephones, personal gaming devices.

Communications medium 110 may include any combination of wireless and wired communication media, and/or storage devices. Communications medium 110 may include coaxial cables, fiber optic cables, twisted pair cables, wireless transmitters and receivers, routers, switches, repeaters, base stations, or any other equipment that may be useful to facilitate communications between various devices and sites. Communications medium 110 may include one or more networks. For example, communications medium 110 may include a network configured to enable access to the World Wide Web, for example, the Internet. A network may operate according to a combination of one or more telecommunication protocols. Telecommunications protocols may include proprietary aspects and/or may include standardized telecommunication protocols. Examples of standardized telecommunications protocols include Digital Video Broadcasting (DVB) standards, Advanced Television Systems Committee (ATSC) standards, Integrated Services Digital Broadcasting (ISDB) standards, Data Over Cable Service Interface Specification (DOCSIS) standards, Global System Mobile Communications (GSM) standards, code division multiple access (CDMA) standards, 3rd Generation Partnership Project (3GPP) standards, European Telecommunications Standards Institute (ETSI) standards, Internet Protocol (IP) standards, Wireless Application Protocol (WAP) standards, and Institute of Electrical and Electronics Engineers (IEEE) standards.

Storage devices may include any type of device or storage medium capable of storing data. A storage medium may include a tangible or non-transitory computer-readable media. A computer readable medium may include optical discs, flash memory, magnetic memory, or any other suitable digital storage media. In some examples, a memory device or portions thereof may be described as non-volatile memory and in other examples portions of memory devices

55

may be described as volatile memory. Examples of volatile memories may include random access memories (RAM), dynamic random access memories (DRAM), and static random access memories (SRAM). Examples of non-volatile memories may include magnetic hard discs, optical discs, floppy discs, flash memories, or forms of electrically programmable memories (EPROM) or electrically erasable and programmable (EEPROM) memories. Storage device(s) may include memory cards (e.g., a Secure Digital (SD) memory card), internal/external hard disk drives, and/or internal/external solid state drives. Data may be stored on a storage device according to a defined file format.

FIG. 4 is a conceptual drawing illustrating an example of components that may be included in an implementation of system 100. In the example implementation illustrated in FIG. 4, system 100 includes one or more computing devices 402A-402N, television service network 404, television service provider site 406, wide area network 408, local area network 410, and one or more content provider sites 412A-412N. The implementation illustrated in FIG. 4 represents an example of a system that may be configured to allow digital media content, such as, for example, a movie, a live sporting event, etc., and data and applications and media presentations associated therewith to be distributed to and accessed by a plurality of computing devices, such as computing devices 402A-402N. In the example illustrated in FIG. 4, computing devices 402A-402N may include any device configured to receive data from one or more of television service network 404, wide area network 408, and/or local area network 410. For example, computing devices 402A-402N may be equipped for wired and/or wireless communications and may be configured to receive services through one or more data channels and may include televisions, including so-called smart televisions, set top boxes, and digital video recorders. Further, computing devices 402A-402N may include desktop, laptop, or tablet computers, gaming consoles, mobile devices, including, for example, "smart" phones, cellular telephones, and personal gaming devices.

Television service network 404 is an example of a network configured to enable digital media content, which may include television services, to be distributed. For example, television service network 404 may include public over-the-air television networks, public or subscription-based satellite television service provider networks, and public or subscription-based cable television provider networks and/or over the top or Internet service providers. It should be noted that although in some examples television service network 404 may primarily be used to enable television services to be provided, television service network 404 may also enable other types of data and services to be provided according to any combination of the telecommunication protocols described herein. Further, it should be noted that in some examples, television service network 404 may enable two-way communications between television service provider site 406 and one or more of computing devices 402A-402N. Television service network 404 may comprise any combination of wireless and/or wired communication media. Television service network 404 may include coaxial cables, fiber optic cables, twisted pair cables, wireless transmitters and receivers, routers, switches, repeaters, base stations, or any other equipment that may be useful to facilitate communications between various devices and sites. Television service network 404 may operate according to a combination of one or more telecommunication protocols. Telecommunications protocols may include proprietary aspects and/or may include standardized telecommunication protocols.

56

Examples of standardized telecommunications protocols include DVB standards, ATSC standards, ISDB standards, DTMB standards, DMB standards, Data Over Cable Service Interface Specification (DOCSIS) standards, HbbTV standards, W3C standards, and UPnP standards.

Referring again to FIG. 4, television service provider site 406 may be configured to distribute television service via television service network 404. For example, television service provider site 406 may include one or more broadcast stations, a cable television provider, or a satellite television provider, or an Internet-based television provider. For example, television service provider site 406 may be configured to receive a transmission including television programming through a satellite uplink/downlink. Further, as illustrated in FIG. 4, television service provider site 406 may be in communication with wide area network 408 and may be configured to receive data from content provider sites 412A-412N. It should be noted that in some examples, television service provider site 406 may include a television studio and content may originate therefrom.

Wide area network 408 may include a packet based network and operate according to a combination of one or more telecommunication protocols. Telecommunications protocols may include proprietary aspects and/or may include standardized telecommunication protocols. Examples of standardized telecommunications protocols include Global System Mobile Communications (GSM) standards, code division multiple access (CDMA) standards, 3rd Generation Partnership Project (3GPP) standards, European Telecommunications Standards Institute (ETSI) standards, European standards (EN), IP standards, Wireless Application Protocol (WAP) standards, and Institute of Electrical and Electronics Engineers (IEEE) standards, such as, for example, one or more of the IEEE 802 standards (e.g., Wi-Fi). Wide area network 408 may comprise any combination of wireless and/or wired communication media. Wide area network 408 may include coaxial cables, fiber optic cables, twisted pair cables, Ethernet cables, wireless transmitters and receivers, routers, switches, repeaters, base stations, or any other equipment that may be useful to facilitate communications between various devices and sites. In one example, wide area network 408 may include the Internet. Local area network 410 may include a packet based network and operate according to a combination of one or more telecommunication protocols. Local area network 410 may be distinguished from wide area network 408 based on levels of access and/or physical infrastructure. For example, local area network 410 may include a secure home network.

Referring again to FIG. 4, content provider sites 412A-412N represent examples of sites that may provide multimedia content to television service provider site 406 and/or computing devices 402A-402N. For example, a content provider site may include a studio having one or more studio content servers configured to provide multimedia files and/or streams to television service provider site 406. In one example, content provider sites 412A-412N may be configured to provide multimedia content using the IP suite. For example, a content provider site may be configured to provide multimedia content to a receiver device according to Real Time Streaming Protocol (RTSP), HTTP, or the like. Further, content provider sites 412A-412N may be configured to provide data, including hypertext based content, and the like, to one or more of receiver devices computing devices 402A-402N and/or television service provider site 406 through wide area network 408. Content provider sites

412A-412N may include one or more web servers. Data provided by data provider site 412A-412N may be defined according to data formats.

Referring again to FIG. 1, source device 102 includes video source 104, video encoder 106, data encapsulator 107, and interface 108. Video source 104 may include any device configured to capture and/or store video data. For example, video source 104 may include a video camera and a storage device operably coupled thereto. Video encoder 106 may include any device configured to receive video data and generate a compliant bitstream representing the video data. A compliant bitstream may refer to a bitstream that a video decoder can receive and reproduce video data therefrom. Aspects of a compliant bitstream may be defined according to a video coding standard. When generating a compliant bitstream video encoder 106 may compress video data. Compression may be lossy (discernible or indiscernible to a viewer) or lossless. FIG. 5 is a block diagram illustrating an example of video encoder 500 that may implement the techniques for encoding video data described herein. It should be noted that although example video encoder 500 is illustrated as having distinct functional blocks, such an illustration is for descriptive purposes and does not limit video encoder 500 and/or sub-components thereof to a particular hardware or software architecture. Functions of video encoder 500 may be realized using any combination of hardware, firmware, and/or software implementations.

Video encoder 500 may perform intra prediction coding and inter prediction coding of picture areas, and, as such, may be referred to as a hybrid video encoder. In the example illustrated in FIG. 5, video encoder 500 receives source video blocks. In some examples, source video blocks may include areas of picture that has been divided according to a coding structure. For example, source video data may include macroblocks, CTUs, CBs, sub-divisions thereof, and/or another equivalent coding unit. In some examples, video encoder 500 may be configured to perform additional sub-divisions of source video blocks. It should be noted that the techniques described herein are generally applicable to video coding, regardless of how source video data is partitioned prior to and/or during encoding. In the example illustrated in FIG. 5, video encoder 500 includes summer 502, transform coefficient generator 504, coefficient quantization unit 506, inverse quantization and transform coefficient processing unit 508, summer 510, intra prediction processing unit 512, inter prediction processing unit 514, filter unit 516, and entropy encoding unit 518. As illustrated in FIG. 5, video encoder 500 receives source video blocks and outputs a bitstream.

In the example illustrated in FIG. 5, video encoder 500 may generate residual data by subtracting a predictive video block from a source video block. The selection of a predictive video block is described in detail below. Summer 502 represents a component configured to perform this subtraction operation. In one example, the subtraction of video blocks occurs in the pixel domain. Transform coefficient generator 504 applies a transform, such as a discrete cosine transform (DCT), a discrete sine transform (DST), or a conceptually similar transform, to the residual block or sub-divisions thereof (e.g., four 8x8 transforms may be applied to a 16x16 array of residual values) to produce a set of residual transform coefficients. Transform coefficient generator 504 may be configured to perform any and all combinations of the transforms included in the family of discrete trigonometric transforms, including approximations thereof. Transform coefficient generator 504 may output transform coefficients to coefficient quantization unit 506.

Coefficient quantization unit 506 may be configured to perform quantization of the transform coefficients. The quantization process may reduce the bit depth associated with some or all of the coefficients. The degree of quantization may alter the rate-distortion (i.e., bit-rate vs. quality of video) of encoded video data. The degree of quantization may be modified by adjusting a quantization parameter (QP). A quantization parameter may be determined based on slice level values and/or CU level values (e.g., CU delta QP values). QP data may include any data used to determine a QP for quantizing a particular set of transform coefficients. As illustrated in FIG. 5, quantized transform coefficients (which may be referred to as level values) are output to inverse quantization and transform coefficient processing unit 508. Inverse quantization and transform coefficient processing unit 508 may be configured to apply an inverse quantization and an inverse transformation to generate reconstructed residual data. As illustrated in FIG. 5, at summer 510, reconstructed residual data may be added to a predictive video block. In this manner, an encoded video block may be reconstructed and the resulting reconstructed video block may be used to evaluate the encoding quality for a given prediction, transformation, and/or quantization. Video encoder 500 may be configured to perform multiple coding passes (e.g., perform encoding while varying one or more of a prediction, transformation parameters, and quantization parameters). The rate-distortion of a bitstream or other system parameters may be optimized based on evaluation of reconstructed video blocks. Further, reconstructed video blocks may be stored and used as reference for predicting subsequent blocks.

Referring again to FIG. 5, intra prediction processing unit 512 may be configured to select an intra prediction mode for a video block to be coded. Intra prediction processing unit 512 may be configured to evaluate a frame and determine an intra prediction mode to use to encode a current block. As described above, possible intra prediction modes may include planar prediction modes, DC prediction modes, and angular prediction modes. Further, it should be noted that in some examples, a prediction mode for a chroma component may be inferred from a prediction mode for a luma prediction mode. Intra prediction processing unit 512 may select an intra prediction mode after performing one or more coding passes. Further, in one example, intra prediction processing unit 512 may select a prediction mode based on a rate-distortion analysis. As illustrated in FIG. 5, intra prediction processing unit 512 outputs intra prediction data (e.g., syntax elements) to entropy encoding unit 518 and transform coefficient generator 504. As described above, a transform performed on residual data may be mode dependent (e.g., a secondary transform matrix may be determined based on a prediction mode).

Referring again to FIG. 5, inter prediction processing unit 514 may be configured to perform inter prediction coding for a current video block. Inter prediction processing unit 514 may be configured to receive source video blocks and calculate a motion vector for PUs of a video block. A motion vector may indicate the displacement of a prediction unit of a video block within a current video frame relative to a predictive block within a reference frame. Inter prediction coding may use one or more reference pictures. Further, motion prediction may be uni-predictive (use one motion vector) or bi-predictive (use two motion vectors). Inter prediction processing unit 514 may be configured to select a predictive block by calculating a pixel difference determined by, for example, sum of absolute difference (SAD), sum of square difference (SSD), or other difference metrics.

59

As described above, a motion vector may be determined and specified according to motion vector prediction. Inter prediction processing unit **514** may be configured to perform motion vector prediction, as described above. Inter prediction processing unit **514** may be configured to generate a predictive block using the motion prediction data. For example, inter prediction processing unit **514** may locate a predictive video block within a frame buffer (not shown in FIG. 5). It should be noted that inter prediction processing unit **514** may further be configured to apply one or more interpolation filters to a reconstructed residual block to calculate sub-integer pixel values for use in motion estimation. Inter prediction processing unit **514** may output motion prediction data for a calculated motion vector to entropy encoding unit **518**.

Referring again to FIG. 5, filter unit **516** receives reconstructed video blocks and coding parameters and outputs modified reconstructed video data. Filter unit **516** may be configured to perform deblocking and/or Sample Adaptive Offset (SAO) filtering. SAO filtering is a non-linear amplitude mapping that may be used to improve reconstruction by adding an offset to reconstructed video data. It should be noted that as illustrated in FIG. 5, intra prediction processing unit **512** and inter prediction processing unit **514** may receive modified reconstructed video block via filter unit **216**. Entropy encoding unit **518** receives quantized transform coefficients and predictive syntax data (i.e., intra prediction data and motion prediction data). It should be noted that in some examples, coefficient quantization unit **506** may perform a scan of a matrix including quantized transform coefficients before the coefficients are output to entropy encoding unit **518**. In other examples, entropy encoding unit **518** may perform a scan. Entropy encoding unit **518** may be configured to perform entropy encoding according to one or more of the techniques described herein. In this manner, video encoder **500** represents an example of a device configured to generate encoded video data according to one or more techniques of this disclosure.

Referring again to FIG. 1, data encapsulator **107** may receive encoded video data and generate a compliant bitstream, e.g., a sequence of NAL units according to a defined data structure. A device receiving a compliant bitstream can reproduce video data therefrom. Further, as described above, sub-bitstream extraction may refer to a process where a device receiving a compliant bitstream forms a new compliant bitstream by discarding and/or modifying data in the received bitstream. It should be noted that the term conforming bitstream may be used in place of the term compliant bitstream. In one example, data encapsulator **107** may be configured to generate syntax according to one or more techniques described herein. It should be noted that data encapsulator **107** need not necessarily be located in the same physical device as video encoder **106**. For example, functions described as being performed by video encoder **106** and data encapsulator **107** may be distributed among devices illustrated in FIG. 4.

As described above, the signaling provided in JVET-AD2006 may be less than ideal. In particular, in JVET-AD2006, when syntax element `nnpfc_purpose` is equal to 0, the NNPF may be used as determined by the application and no information about the application specific purpose is provided. In one example, according to the techniques herein, a syntax element, (e.g., `nnpfc_application_purpose_tag_uri`) may be signaled when `nnpfc_purpose` is equal to 0 (i.e., when NNPF purpose indicates application specific or application determined usage) to provide more details about an application-specific purpose in a decentralized manner.

60

Table 13A and Table 13B illustrate examples of relevant syntax of a Neural-network post-filter characteristics SEI message, where a syntax element is signaled to provide more details about an application-specific purpose, according to the techniques herein.

TABLE 13A

nn_post_filter_characteristics(payloadSize) {		Descriptor
nnpfc_purpose		u(16)
nnpfc_id		ue(v)
nnpfc_base_flag		u(1)
nnpfc_mode_idc		ue(v)
if(nnpfc_mode_idc == 1) {		
while(!byte_aligned())		
nnpfc_reserved_zero_bit_a		u(1)
nnpfc_tag_uri		st(v)
nnpfc_uri		st(v)
}		
if(nnpfc_purpose == 0)		
nnpfc_application_purpose_tag_uri		st(v)
nnpfc_property_present_flag		u(1)
if(nnpfc_property_present_flag) {		
/* input and output formatting */		
...		
}		

TABLE 13B

nn_post_filter_characteristics(payloadSize) {		Descriptor
nnpfc_purpose		u(16)
...		
nnpfc_metadata_extension_num_bits		ue(v)
if(nnpfc_metadata_extension_num_bits > 0) {		
if(nnpfc_purpose == 0)		
nnpfc_application_purpose_tag_uri		st(v)
nnpfc_reserved_metadata_extension		u(v)
}		
/* ISO/IEC 15938-17 bitstream */		
if(nnpfc_mode_idc == 0) {		
while(!byte_aligned())		
nnpfc_reserved_zero_bit_b		u(1)
for(i = 0; more_data_in_payload(); i++)		
nnpfc_payload_byte[i]		b(8)
}		

With respect to Table 13A and Table 13B, in one example, the semantics may be based on the semantics provided above and the following:

`nnpfc_purpose` indicates the purpose of the NNPF as specified in Table 8, where (`nnpfc_purpose` & `bitMask`) not equal to 0 indicates that the NNPF has the purpose associated with the `bitMask` value in Table 8. When `nnpfc_purpose` is greater than 0 and (`nnpfc_purpose` & `bitMask`) is equal to 0, the purpose associated with the `bitMask` value is not applicable to the NNPF. When `nnpfc_purpose` is equal to 0, the NNPF may be used as determined by the application and as specified by the `nnpfc_application_purpose_tag_uri`.

The value of `nnpfc_purpose` shall be in the range of 0 to 63, inclusive, in bitstreams conforming to this edition of this document. Values of 64 to 65 535, inclusive, for `nnpfc_purpose` are reserved for future use by JTV-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages with `nnpfc_purpose` in the range of 64 to 65 535, inclusive.

The variables `chromaUpsamplingFlag`, `resolutionResamplingFlag`, `pictureRateUpsamplingFlag`, `bitDepthUpsam-`

61

plingFlag, and colourizationFlag, specifying whether nnpfc_purpose indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

```
chromaUpsamplingFlag = ((nnpfc_purpose & 0x02) > 0) ? 1 : 0
resolutionResamplingFlag = ((nnpfc_purpose & 0x04) > 0) ? 1 : 0
pictureRateUpsamplingFlag = ((nnpfc_purpose & 0x08) > 0) ? 1 : 0
bitDepthUpsamplingFlag = ((nnpfc_purpose & 0x10) > 0) ? 1 : 0
colourizationFlag = ((nnpfc_purpose & 0x20) > 0) ? 1 : 0
```

NOTE—When a reserved value of nnpfc_purpose is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by nnpfc_purpose being equal to that value.

When ChromaFormatIdc is equal to 3, chromaUpsamplingFlag shall be equal to 0.

When ChromaFormatIdc or chromaUpsamplingFlag is not equal to 0, colourizationFlag shall be equal to 0.

When pictureRateUpsamplingFlag is equal to 1 and the input picture with index 0 is associated with a frame packing arrangement SEI message with fp_arrangement_type equal to 5, all input pictures are associated with a frame packing arrangement SEI message with fp_arrangement_type equal to 5 and the same value of fp_current_frame_is_frame0_flag.

nnpfc_application_purpose_tag_uri specifies a tag URI with syntax and semantics as specified in IETF RFC 4151 identifying the application-specific or application determined purpose of the NNPF, when nnpfc_purpose is equal to 0 (i.e., when NNPF purpose indicates application specific or application determined usage).

NOTE—nnpfc_application_purpose_tag_uri enables uniquely identifying the application-specific or application determined purpose of NNPF without needing a central registration authority.

With respect to Table 13B, in one example, the semantics may be based on the semantics provided above and the following:

nnpfc_reserved_metadata_extension shall not be present in bitstreams conforming to this edition of this document. However, decoders conforming to this edition of this document shall ignore the presence and value of nnpfc_reserved_metadata_extension. When present, the length, in bits, of nnpfc_reserved_metadata_extension is equal to nnpfc_metadata_extension_num_bits—Length of (nnpfc_application_purpose_tag_uri) when nnpfc_purpose is equal to 0 and is equal to nnpfc_metadata_extension_num_bits otherwise.

In one example, according to the techniques herein, more than one nnpfc_application_purpose_tag_uri[i] values may be specified to allow defining multiple application-specific purposes of the NNPF. Table 14A and Table 14B illustrate examples of relevant syntax of a Neural-network post-filter characteristics SEI message, where more than one syntax elements may be signaled to provide more details about an application-specific purpose, according to the techniques herein.

62

TABLE 14A

nn_post_filter_characteristics(payloadSize) {		Descriptor
5	nnpfc_purpose	u(16)
	nnpfc_id	ue(v)
	nnpfc_base_flag	u(1)
	nnpfc_mode_idc	ue(v)
	if(nnpfc_mode_idc == 1) {	
	while(!byte_aligned())	
	nnpfc_reserved_zero_bit_a	u(1)
10	nnpfc_tag_uri	st(v)
	nnpfc_uri	st(v)
	}	
	if(nnpfc_purpose == 0) {	
	nnpfc_num_application_purposes_minus1	ue(v)
	for(i = 0; i < nnpfc_num_application_purposes_minus1;	
	i++)	
15	nnpfc_application_purpose_tag_uri[i]	st(v)
	}	
	nnpfc_property_present_flag	u(1)
	if(nnpfc_property_present_flag) {	
	/* input and output formatting */	
20	... }	

TABLE 14B

nn_post_filter_characteristics(payloadSize) {		Descriptor
25	nnpfc_purpose	u(16)
	...	
	nnpfc_metadata_extension_num_bits	ue(v)
	if(nnpfc_metadata_extension_num_bits > 0) {	
30	if(nnpfc_purpose == 0) {	
	nnpfc_num_application_purposes_minus1	ue(v)
	for(i = 0; i <	
	nnpfc_num_application_purposes_minus1; i++)	
	nnpfc_application_purpose_tag_uri[i]	st(v)
35	}	
	}	
	nnpfc_reserved_metadata_extension	u(v)
	}	
	/* ISO/IEC 15938-17 bitstream */	
	if(nnpfc_mode_idc == 0) {	
	while(!byte_aligned())	
40	nnpfc_reserved_zero_bit_b	u(1)
	for(i = 0; more_data_in_payload(); i++)	
	nnpfc_payload_byte[i]	b(8)
	}	
	}	

With respect to Table 14A and Table 14B, in one example, the semantics may be based on the semantics provided above and the following:

nnpfc_purpose indicates the purpose of the NNPF as specified in Table 8, where (nnpfc_purpose & bitMask) not equal to 0 indicates that the NNPF has the purpose associated with the bitMask value in Table 8. When nnpfc_purpose is greater than 0 and (nnpfc_purpose & bitMask) is equal to 0, the purpose associated with the bitMask value is not applicable to the NNPF. When nnpfc_purpose is equal to 0, the NNPF may be used as determined by the application and as specified by the nnpfc_application_purpose_tag_uri.

The value of nnpfc_purpose shall be in the range of 0 to 63, inclusive, in bitstreams conforming to this edition of this document. Values of 64 to 65 535, inclusive, for nnpfc_purpose are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages with nnpfc_purpose in the range of 64 to 65 535, inclusive.

The variables chromaUpsamplingFlag, resolutionResamplingFlag, pictureRateUpsamplingFlag, bitDepthUpsam-

plingFlag, and colourizationFlag, specifying whether nnpfc_purpose indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

$$\begin{aligned} \text{chromaUpsamplingFlag} &= ((\text{nnpfc_purpose} \& 0x02) > 0) ? 1 : 0 \\ \text{resolutionResamplingFlag} &= ((\text{nnpfc_purpose} \& 0x04) > 0) ? 1 : 0 \\ \text{pictureRateUpsamplingFlag} &= ((\text{nnpfc_purpose} \& 0x08) > 0) ? 1 : 0 \\ \text{bitDepthUpsamplingFlag} &= ((\text{nnpfc_purpose} \& 0x10) > 0) ? 1 : 0 \\ \text{colourizationFlag} &= ((\text{nnpfc_purpose} \& 0x20) > 0) ? 1 : 0 \end{aligned}$$

NOTE—When a reserved value of nnpfc_purpose is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by nnpfc_purpose being equal to that value.

When ChromaFormatIdc is equal to 3, chromaUpsamplingFlag shall be equal to 0.

When ChromaFormatIdc or chromaUpsamplingFlag is not equal to 0, colourizationFlag shall be equal to 0.

When pictureRateUpsamplingFlag is equal to 1 and the input picture with index 0 is associated with a frame packing arrangement SEI message with fp_arrangement_type equal to 5, all input pictures are associated with a frame packing arrangement SEI message with fp_arrangement_type equal to 5 and the same value of fp_current_frame_is_frame0_flag.

nnpfc_num_application_purposes_minus1 plus 1 specifies the number of nnpfc_application_purpose_tag_uri[i] value signalled specifying the number of application-specific or application determined purposes of the NNPF. The value of nnpfc_num_application_purposes_minus1 shall be in the range of 0 to 63 (or 127 or 255 or 1024 or some other upper bound), inclusive.

nnpfc_application_purpose_tag_uri[i] specifies i-th tag URI with syntax and semantics as specified in IETF RFC 4151 identifying the i-th application-specific or application determined purpose of the NNPF, when nnpfc_purpose is equal to 0 (i.e., when NNPF purpose indicates application specific or application determined usage).

NOTE—nnpfc_application_purpose_tag_uri[i] enables uniquely identifying the application-specific or application determined purpose of NNPF without needing a central registration authority. An NNPF with multiple nnpfc_application_purpose_tag_uri[i] provides functionality specified by all the individual nnpfc_application_purpose_tag_uri[i] values.

With respect to Table 14B, in one example, the semantics may be based on the semantics provided above and the following:

nnpfc_reserved_metadata_extension shall not be present in bitstreams conforming to this edition of this document. However, decoders conforming to this edition of this document shall ignore the presence and value of nnpfc_reserved_metadata_extension. When present, the length, in bits, of nnpfc_reserved_metadata_extension is equal to nnpfc_metadata_extension_num_bits—Length of (nnpfc_num_application_purposes_minus1)—sum of Length of (nnpfc_application_purpose_tag_uri[i]) for i in the range of 0 to nnpfc_num_application_purposes_minus1, inclusive, when nnpfc_purpose is equal to 0 and is equal to nnpfc_metadata_extension_num_bits otherwise.

nus1, inclusive, when nnpfc_purpose is equal to 0 and is equal to nnpfc_metadata_extension_num_bits otherwise.

As described above, in JVET-AD2006, when syntax element nnpfc_purpose is equal to 0, the NNPF may be used as determined by the application and no information about the application specific purpose is provided. It should be noted that in this case, in JVET-AD2006, the defined NNPF purposes (e.g., general visual quality improvement, chroma upsampling, resolution upsampling, picture rate upsampling, bit depth upsampling, colourization) cannot be specified together with an application specific purpose. For example, a NNPF may support one or more of the currently defined NNPF purposes and include additional purposes which may be application specific. In one example, according to the techniques herein, a bit is used to specify an additional purpose of the NNPF as determined by the application. This allows using an application determined purpose together with the other NNPF purposes which are already defined or which may be defined in future. In one example, according to the techniques herein, the semantic of syntax element nnpfc_purpose may be based on the following:

nnpfc_purpose indicates the purpose of the NNPF as specified in Table 15, where (nnpfc_purpose & bitMask) not equal to 0 indicates that the NNPF has the purpose associated with the bitMask value in Table 15. When nnpfc_purpose is greater than 0 and (nnpfc_purpose & bitMask) is equal to 0, the purpose associated with the bitMask value is not applicable to the NNPF. nnpfc_purpose value equal to 0 is reserved.

The value of nnpfc_purpose shall be in the range of 0 to 127, inclusive, in bitstreams conforming to this edition of this document. Values of 128 to 65 535, inclusive, for nnpfc_purpose are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages with nnpfc_purpose in the range of 64 to 65 535, inclusive.

TABLE 15

bitMask	Interpretation
0x01	General visual quality improvement
0x02	Chroma upsampling (from the 4:2:0 chroma format to the 4:2:2 or 4:4:4 chroma format, or from the 4:2:2 chroma format to the 4:4:4 chroma format)
0x04	Resolution resampling (increasing or decreasing the width or height)
0x08	Picture rate upsampling
0x10	Bit depth upsampling (increasing the luma bit depth or the chroma bit depth)
0x20	Colourization
0x40	Application determined or application-specific purpose

The variables chromaUpsamplingFlag, resolutionResamplingFlag, pictureRateUpsamplingFlag, bitDepthUpsamplingFlag, and colourizationFlag, specifying whether nnpfc_purpose indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

$$\begin{aligned} \text{chromaUpsamplingFlag} &= ((\text{nnpfc_purpose} \& 0x02) > 0) ? 1 : 0 \\ \text{resolutionResamplingFlag} &= ((\text{nnpfc_purpose} \& 0x04) > 0) ? 1 : 0 \end{aligned}$$

65

-continued

```

pictureRateUpsamplingFlag = ((nnpfc_purpose & 0x08) > 0) ? 1 : 0
bitDepthUpsamplingFlag = ((nnpfc_purpose & 0x10) > 0) ? 1 : 0
colourizationFlag = ((nnpfc_purpose & 0x20) > 0) ? 1 : 0
applicationPurposeFlag = ((nnpfc_purpose & 0x40) > 0) ? 1 : 0

```

NOTE—When a reserved value of `nnpfc_purpose` is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by `nnpfc_purpose` being equal to that value.

When `ChromaFormatIdc` is equal to 3, `chromaUpsamplingFlag` shall be equal to 0.

When `ChromaFormatIdc` or `chromaUpsamplingFlag` is not equal to 0, `colourizationFlag` shall be equal to 0.

When `pictureRateUpsamplingFlag` is equal to 1 and the input picture with index 0 is associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5, all input pictures are associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5 and the same value of `fp_current_frame_is_frame0_flag`.

In one example, the semantics of `nnpfc_purpose` provided above may be used with example syntax and semantics provide above with respect to Tables 13A-14B. That is, in each of Tables 13A-14B the condition “if(`nnpfc_purpose`==0)” may be replaced with the condition “if(applicationPurposeFlag).”

Further, it should be noted that although in the example semantics a particular bit (i.e., 7th LSB) set to 1 is used to derive `applicationPurposeFlag` equal to 1, in other examples, the meaning of this bit may be flipped (i.e., it being equal to 0 may be used to derive `applicationPurposeFlag` equal to 1). Further, in other examples, instead of 7th LSB, another bit may be used to indicate the application determined or application-specific purpose. In another example, more than 1 bit (e.g., 2 bits) may be used to specify the application determined or application-specific purpose.

In one example, according to the techniques herein, a bit of syntax element `nnpfc_purpose` which specifies an application specified purpose may be signaled for those cases where the application specified purpose may be indicated in addition to one of the other defined NNPF purposes. However, for cases where entire NNPF purpose is application specified, a special value of `nnpfc_purpose` being equal to 0 is also still defined. In one example, according to the techniques herein, the semantic of syntax element `nnpfc_purpose` may be based on the following:

`nnpfc_purpose` indicates the purpose of the NNPF as specified in Table 15, where (`nnpfc_purpose` & `bitMask`) not equal to 0 indicates that the NNPF has the purpose associated with the `bitMask` value in Table 15. When `nnpfc_purpose` is greater than 0 and (`nnpfc_purpose` & `bitMask`) is equal to 0, the purpose associated with the `bitMask` value is not applicable to the NNPF. When `nnpfc_purpose` is equal to 0, the NNPF may be used as entirely determined by the application.

The value of `nnpfc_purpose` shall be in the range of 0 to 127, inclusive, in bitstreams conforming to this edition of this document. Values of 128 to 65 535, inclusive, for `nnpfc_purpose` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this

66

edition of this document shall ignore NNPF SEI messages with `nnpfc_purpose` in the range of 64 to 65 535, inclusive.

The variables `chromaUpsamplingFlag`, `resolutionResamplingFlag`, `pictureRateUpsamplingFlag`, `bitDepthUpsamplingFlag`, and `colourizationFlag`, specifying whether `nnpfc_purpose` indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

```

chromaUpsamplingFlag = ((nnpfc_purpose & 0x02) > 0) ? 1 : 0
resolutionResamplingFlag = ((nnpfc_purpose & 0x04) > 0) ? 1 : 0
pictureRateUpsamplingFlag = ((nnpfc_purpose & 0x08) > 0) ? 1 : 0
bitDepthUpsamplingFlag = ((nnpfc_purpose & 0x10) > 0) ? 1 : 0
colourizationFlag = ((nnpfc_purpose & 0x20) > 0) ? 1 : 0
if (nnpfc_purpose > 0)
    applicationPurposeFlag = ((nnpfc_purpose & 0x40) > 0) ? 1 : 0

```

NOTE—When a reserved value of `nnpfc_purpose` is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by `nnpfc_purpose` being equal to that value.

When `ChromaFormatIdc` is equal to 3, `chromaUpsamplingFlag` shall be equal to 0.

When `ChromaFormatIdc` or `chromaUpsamplingFlag` is not equal to 0, `colourizationFlag` shall be equal to 0.

In one example, the bit of syntax element `nnpfc_purpose` which indicates an application determined or application-specific purpose may only be used when at least one of the other defined `nnpfc_purpose` bits is equal to 1. This prevents using only the bit to indicate `nnpfc_purpose`, which is entirely application determined or application-specific. In one example, this may be accomplished by adding the following constraint to the example semantics of `nnpfc_purpose` provided above:

It may be a requirement of bitstream conformance that when `applicationPurposeFlag` is equal to 1, at least one of `chromaUpsamplingFlag`, `resolutionResamplingFlag`, `pictureRateUpsamplingFlag`, `bitDepthUpsamplingFlag`, `colourizationFlag` (or other pre-defined NNPF purpose flags) shall be equal to 1.

In another example, the derivation of `applicationPurposeFlag` in the example semantics of `nnpfc_purpose` may instead be as follows:

```

if (nnpfc_purpose == 0)
    applicationPurposeFlag = 1
else
    applicationPurposeFlag = ((nnpfc_purpose & 0x40) > 0) ? 1 : 0

```

In this case, `applicationPurposeFlag` is set equal to 1 when `nnpfc_purpose` is equal to 0 or when the application determined or application-specific bit of `nnpfc_purpose` is set equal to 1, (i.e. if (`nnpfc_purpose` & 0x40) is greater than 0).

In another example, a flag (e.g., `nnpfc_externally_specified_purpose_flag`) may be signaled, where if the flag has a value 1, then the NNPF purpose is completely determined by the application and syntax element `nnpfc_purpose` is not signaled. Further, if the flag has a value equal to 0, the syntax element `nnpfc_purpose` is signaled. Table 16 illustrates an

67

example of relevant syntax of a Neural-network post-filter characteristics SEI message, where a flag may be signaled to provide more details about an application-specific purpose, according to the techniques herein.

TABLE 16

<code>nn_post_filter_characteristics(payloadSize) {</code>	Descriptor
<code>nnpfc_externally_specified_purpose_flag</code>	<code>u(1)</code>
<code>if(!nnpfc_externally_specified_purpose_flag)</code>	
<code> nnpfc_purpose</code>	<code>u(16)</code>
<code> nnpfc_id</code>	<code>ue(v)</code>
<code> nnpfc_base_flag</code>	<code>u(1)</code>
<code>...</code>	

With respect to Table 16, in one example, the semantics may be based on the semantics provided above and the following:

`nnpfc_externally_specified_purpose_flag` equal to 1 specifies that syntax elements element `nnpfc_purpose` is not present and the purpose of the NNPF is specified externally or is determined by the application or specified by the application.

`nnpfc_externally_specified_purpose_flag` equal to 0 specifies that the syntax element `nnpfc_purpose` is present and specifies the purpose of the NNPF.

In one example, the semantics of `nnpfc_purpose` provided above may be used with example syntax and semantics provide above with respect to Tables 13A-14B. That is, in each of Tables 13A-14B the condition “`if(nnpfc_purpose==0)`” may be replaced with the condition “`if(nnpfc_externally_specified_purpose_flag)`.”

With respect to Table 16, the semantics of `nnpfc_purpose` may be based on the semantics provided above with respect to Table 7, the semantics of `nnpfc_purpose` provided above including Table 15, or the following:

`nnpfc_purpose` indicates the purpose of the NNPF as specified in Table 8, where $(\text{nnpfc_purpose} \ \& \ \text{bitMask})$ not equal to 0 indicates that the NNPF has the purpose associated with the `bitMask` value in Table 8. When `nnpfc_purpose` is greater than 0 and $(\text{nnpfc_purpose} \ \& \ \text{bitMask})$ is equal to 0, the purpose associated with the `bitMask` value is not applicable to the NNPF. `nnpfc_purpose` value equal to 0 is reserved.

The value of `nnpfc_purpose` shall be in the range of 0 to 63, inclusive, in bitstreams conforming to this edition of this document. Values of 64 to 65 535, inclusive, for `nnpfc_purpose` are reserved for future use by ITU-T/ISO/IEC and shall not be present in bitstreams conforming to this edition of this document. Decoders conforming to this edition of this document shall ignore NNPF SEI messages with `nnpfc_purpose` in the range of 64 to 65 535, inclusive.

The variables `chromaUpsamplingFlag`, `resolutionResamplingFlag`, `pictureRateUpsamplingFlag`, `bitDepthUpsamplingFlag`, and `colourizationFlag`, specifying whether `nnpfc_purpose` indicates the purpose of the NNPF to include chroma upsampling, resolution resampling, picture rate upsampling, bit depth upsampling, and colourization, respectively, are derived as follows:

$$\text{chromaUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0x02) > 0) ? 1 : 0$$

$$\text{resolutionResamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0x04) > 0) ? 1 : 0$$

$$\text{pictureRateUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0x08) > 0) ? 1 : 0$$

68

-continued

$$\text{bitDepthUpsamplingFlag} = ((\text{nnpfc_purpose} \ \& \ 0x10) > 0) ? 1 : 0$$

$$\text{colourizationFlag} = ((\text{nnpfc_purpose} \ \& \ 0x20) > 0) ? 1 : 0$$

NOTE—When a reserved value of `nnpfc_purpose` is taken into use in the future by ITU-T/ISO/IEC, the syntax of this SEI message could be extended with syntax elements whose presence is conditioned by `nnpfc_purpose` being equal to that value.

When `ChromaFormatIdc` is equal to 3, `chromaUpsamplingFlag` shall be equal to 0.

When `ChromaFormatIdc` or `chromaUpsamplingFlag` is not equal to 0, `colourizationFlag` shall be equal to 0.

When `pictureRateUpsamplingFlag` is equal to 1 and the input picture with index 0 is associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5, all input pictures are associated with a frame packing arrangement SEI message with `fp_arrangement_type` equal to 5 and the same value of `fp_current_frame_is_frame0_flag`.

In this manner, video encoder 500 represents an example of a device configured to signal a neural network post-filter characteristics message, signal a first syntax element in the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and signal a second syntax element in the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

Referring again to FIG. 1, interface 108 may include any device configured to receive data generated by data encapsulator 107 and transmit and/or store the data to a communications medium. Interface 108 may include a network interface card, such as an Ethernet card, and may include an optical transceiver, a radio frequency transceiver, or any other type of device that can send and/or receive information. Further, interface 108 may include a computer system interface that may enable a file to be stored on a storage device. For example, interface 108 may include a chipset supporting Peripheral Component Interconnect (PCI) and Peripheral Component Interconnect Express (PCIe) bus protocols, proprietary bus protocols, Universal Serial Bus (USB) protocols, I²C, or any other logical and physical structure that may be used to interconnect peer devices.

Referring again to FIG. 1, destination device 120 includes interface 122, data decapsulator 123, video decoder 124, and display 126. Interface 122 may include any device configured to receive data from a communications medium. Interface 122 may include a network interface card, such as an Ethernet card, and may include an optical transceiver, a radio frequency transceiver, or any other type of device that can receive and/or send information. Further, interface 122 may include a computer system interface enabling a compliant video bitstream to be retrieved from a storage device. For example, interface 122 may include a chipset supporting PCI and PCIe bus protocols, proprietary bus protocols, USB protocols, I²C, or any other logical and physical structure that may be used to interconnect peer devices. Data decapsulator 123 may be configured to receive and parse any of the example syntax structures described herein.

Video decoder 124 may include any device configured to receive a bitstream (e.g., a sub-bitstream extraction) and/or acceptable variations thereof and reproduce video data therefrom. Display 126 may include any device configured to display video data. Display 126 may comprise one of a

variety of display devices such as a liquid crystal display (LCD), a plasma display, an organic light emitting diode (OLED) display, or another type of display. Display 126 may include a High Definition display or an Ultra High Definition display. It should be noted that although in the example illustrated in FIG. 1, video decoder 124 is described as outputting data to display 126, video decoder 124 may be configured to output video data to various types of devices and/or sub-components thereof. For example, video decoder 124 may be configured to output video data to any communication medium, as described herein.

FIG. 6 is a block diagram illustrating an example of a video decoder that may be configured to decode video data according to one or more techniques of this disclosure (e.g., the decoding process for reference-picture list construction described above). In one example, video decoder 600 may be configured to decode transform data and reconstruct residual data from transform coefficients based on decoded transform data. Video decoder 600 may be configured to perform intra prediction decoding and inter prediction decoding and, as such, may be referred to as a hybrid decoder. Video decoder 600 may be configured to parse any combination of the syntax elements described above in Tables 1-16. Video decoder 600 may decode video based on or according to the processes described above, and further based on parsed values in Tables 1-16.

In the example illustrated in FIG. 6, video decoder 600 includes an entropy decoding unit 602, inverse quantization unit 604, inverse transform coefficient processing unit 606, intra prediction processing unit 608, inter prediction processing unit 610, summer 612, post filter unit 614, and reference buffer 616. Video decoder 600 may be configured to decode video data in a manner consistent with a video coding system. It should be noted that although example video decoder 600 is illustrated as having distinct functional blocks, such an illustration is for descriptive purposes and does not limit video decoder 600 and/or sub-components thereof to a particular hardware or software architecture. Functions of video decoder 600 may be realized using any combination of hardware, firmware, and/or software implementations.

As illustrated in FIG. 6, entropy decoding unit 602 receives an entropy encoded bitstream. Entropy decoding unit 602 may be configured to decode syntax elements and quantized coefficients from the bitstream according to a process reciprocal to an entropy encoding process. Entropy decoding unit 602 may be configured to perform entropy decoding according any of the entropy coding techniques described above. Entropy decoding unit 602 may determine values for syntax elements in an encoded bitstream in a manner consistent with a video coding standard. As illustrated in FIG. 6, entropy decoding unit 602 may determine a quantization parameter, quantized coefficient values, transform data, and prediction data from a bitstream. In the example illustrated in FIG. 6, inverse quantization unit 604 and inverse transform coefficient processing unit 606 receive quantized coefficient values from entropy decoding unit 602 and output reconstructed residual data.

Referring again to FIG. 6, reconstructed residual data may be provided to summer 612. Summer 612 may add reconstructed residual data to a predictive video block and generate reconstructed video data. A predictive video block may be determined according to a predictive video technique (i.e., intra prediction and inter frame prediction). Intra prediction processing unit 608 may be configured to receive intra prediction syntax elements and retrieve a predictive video block from reference buffer 616. Reference buffer 616

may include a memory device configured to store one or more frames of video data. Intra prediction syntax elements may identify an intra prediction mode, such as the intra prediction modes described above. Inter prediction processing unit 610 may receive inter prediction syntax elements and generate motion vectors to identify a prediction block in one or more reference frames stored in reference buffer 616. Inter prediction processing unit 610 may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used for motion estimation with sub-pixel precision may be included in the syntax elements. Inter prediction processing unit 610 may use interpolation filters to calculate interpolated values for sub-integer pixels of a reference block. Post filter unit 614 may be configured to perform filtering on reconstructed video data. For example, post filter unit 614 may be configured to perform deblocking and/or Sample Adaptive Offset (SAO) filtering, e.g., based on parameters specified in a bitstream. Further, it should be noted that in some examples, post filter unit 614 may be configured to perform proprietary discretionary filtering (e.g., visual enhancements, such as, mosquito noise reduction). As illustrated in FIG. 6, a reconstructed video block may be output by video decoder 600. In this manner, video decoder 600 represents an example of a device configured to receive a neural network post-filter characteristics message, parse a first syntax element from the neural network post-filter characteristics message indicating whether a purpose of a neural network post-filter is specified by an application, and parse a second syntax element from the neural network post-filter characteristics message specifying a universal resource identifier indicating the application specified purpose.

In one or more examples, the functions described may be implemented in hardware, software, firmware, or any combination thereof. If implemented in software, the functions may be stored on or transmitted over as one or more instructions or code on a computer-readable medium and executed by a hardware-based processing unit. Computer-readable media may include computer-readable storage media, which corresponds to a tangible medium such as data storage media, or communication media including any medium that facilitates transfer of a computer program from one place to another, e.g., according to a communication protocol. In this manner, computer-readable media generally may correspond to (1) tangible computer-readable storage media which is non-transitory or (2) a communication medium such as a signal or carrier wave. Data storage media may be any available media that can be accessed by one or more computers or one or more processors to retrieve instructions, code and/or data structures for implementation of the techniques described in this disclosure. A computer program product may include a computer-readable medium.

By way of example, and not limitation, such computer-readable storage media can comprise RAM, ROM, EEPROM, CD-ROM or other optical disk storage, magnetic disk storage, or other magnetic storage devices, flash memory, or any other medium that can be used to store desired program code in the form of instructions or data structures and that can be accessed by a computer. Also, any connection is properly termed a computer-readable medium. For example, if instructions are transmitted from a website, server, or other remote source using a coaxial cable, fiber optic cable, twisted pair, digital subscriber line (DSL), or wireless technologies such as infrared, radio, and microwave, then the coaxial cable, fiber optic cable, twisted pair, DSL, or wireless technologies such as infrared, radio, and

71

microwave are included in the definition of medium. It should be understood, however, that computer-readable storage media and data storage media do not include connections, carrier waves, signals, or other transitory media, but are instead directed to non-transitory, tangible storage media. Disk and disc, as used herein, includes compact disc (CD), laser disc, optical disc, digital versatile disc (DVD), floppy disk and Blu-ray disc where disks usually reproduce data magnetically, while discs reproduce data optically with lasers. Combinations of the above should also be included within the scope of computer-readable media.

Instructions may be executed by one or more processors, such as one or more digital signal processors (DSPs), general purpose microprocessors, application specific integrated circuits (ASICs), field programmable logic arrays (FPGAs), or other equivalent integrated or discrete logic circuitry. Accordingly, the term "processor," as used herein may refer to any of the foregoing structure or any other structure suitable for implementation of the techniques described herein. In addition, in some aspects, the functionality described herein may be provided within dedicated hardware and/or software modules configured for encoding and decoding, or incorporated in a combined codec. Also, the techniques could be fully implemented in one or more circuits or logic elements.

The techniques of this disclosure may be implemented in a wide variety of devices or apparatuses, including a wireless handset, an integrated circuit (IC) or a set of ICs (e.g., a chip set). Various components, modules, or units are described in this disclosure to emphasize functional aspects of devices configured to perform the disclosed techniques, but do not necessarily require realization by different hardware units. Rather, as described above, various units may be combined in a codec hardware unit or provided by a collection of interoperative hardware units, including one or more processors as described above, in conjunction with suitable software and/or firmware.

Moreover, each functional block or various features of the base station device and the terminal device used in each of the aforementioned embodiments may be implemented or executed by a circuitry, which is typically an integrated circuit or a plurality of integrated circuits. The circuitry designed to execute the functions described in the present specification may comprise a general-purpose processor, a digital signal processor (DSP), an application specific or general application integrated circuit (ASIC), a field programmable gate array (FPGA), or other programmable logic devices, discrete gates or transistor logic, or a discrete hardware component, or a combination thereof. The general-purpose processor may be a microprocessor, or alternatively, the processor may be a conventional processor, a controller, a microcontroller or a state machine. The general-purpose processor or each circuit described above may be configured by a digital circuit or may be configured by an analogue circuit. Further, when a technology of making into an integrated circuit superseding integrated circuits at the present time appears due to advancement of a semiconductor technology, the integrated circuit by this technology is also able to be used.

72

Various examples have been described. These and other examples are within the scope of the following claims.

What is claimed is:

1. A method of performing neural network filtering for video data, the method comprising:
 - receiving a neural network post-filter characteristics message;
 - parsing a first syntax element in the neural network post-filter characteristics message, wherein the first syntax element is a 16-bit unsigned integer and indicates a purpose of a neural network post-filter, and the first syntax element equal to 0 indicates the neural network post-filter may be used as determined by an application; and
 - parsing a second syntax element in the neural network post-filter characteristics message, based on a condition that a value of the first syntax element is equal to 0, wherein the second syntax element is a string and specifies a tag universal resource identifier identifying the application determined purpose of the neural network post-filter.
2. A device comprising one or more processors configured to:
 - receive a neural network post-filter characteristics message;
 - parse a first syntax element in the neural network post-filter characteristics message, wherein the first syntax element is a 16-bit unsigned integer and indicates a purpose of a neural network post-filter, and the first syntax element equal to 0 indicates the neural network post-filter may be used as determined by an application; and
 - parse a second syntax element in the neural network post-filter characteristics message, based on a condition that a value of the first syntax element is equal to 0, wherein the second syntax element is a string and specifies a tag universal resource identifier identifying the application determined purpose of the neural network post-filter.
3. The device of claim 2, wherein the device includes a video decoder.
4. A device comprising one or more processors configured to:
 - signal a neural network post-filter characteristics message,
 - wherein the neural network post-filter characteristics message includes:
 - a first syntax element, wherein the first syntax element is a 16-bit unsigned integer and indicates a purpose of a neural network post-filter, and the first syntax element equal to 0 indicates the neural network post-filter may be used as determined by an application, and
 - a second syntax element in the neural network post-filter characteristics message, based on a condition that a value of the first syntax element is equal to 0, wherein the second syntax element is a string and specifies a tag universal resource identifier identifying the application determined purpose of the neural network post-filter.

* * * * *